TECHNISCHE HOCHSCHULE MITTELHESSEN
Mathematik/ Naturwissenschaften/ Informatik
Informatik
Prof. Dr. Klaus Wüst
Michael Kreutzer

Diplomarbeit

– Einführung in die ARM Cortex™-M3 Mikrocontroller-Progammierung durch eine exemplarische Anwendungsimplementierung unter Einbindung eines Echtzeitbetriebssystems –

vorgelegt von:

Student: Hauke Pfaffelmoser
Studiengang: Informatik
Matrikelnummer: 625210

Kirchhain, den 14.01.2012

Abstract

The Department of Computer and Information Science of the University of Applied Sciences, Central Hessen, Germany, is offering a new internship in the summer term of 2012. It is called 'Internship for Embedded Systems'. This diploma provides an introduction in the range of implementation of software, involving the real-time operating system Keil RTX for a microcontroller based on the Cortex™-M3 architecture (STMicroelectronics STM32S107VC). The programming of the processor and the real-time operating system is described in the context of a fan control as an exemplary application. The implementation occurs by using the libraries ARM Cortex Microcontroller Software Interface Standard (CMSIS) and STM32F10x Standard Peripherals Library. The information is reduced to the essential elements. The implementation is explained step by step, to ease the comprehension of the logic structure of the processor, the data sheets, user manuals and the functioning of the library.

The thesis is divided into several chapters:

Chapter one introduces the structure of the thesis. The individual elements of the hard- and software that was used, their operating mechanisms and the functioning of the pulse width modulation are described in chapter two. In chapter three a template is implemented, which is used as a framework for several different projects as well. The fourth chapter describes the configuration and initialization of the mircocontroller with the help of the STM32F10x Standard Peripherals Library by the implementation of a PWM fan control. Chapter five explains the basic operating mechanisms, the possible configurations, as well as the system calls of the Keil RTX real-time operating system. For a better understanding, the fan control that was implemented before, is being integrated into the real-time operation system, in chapter six. Chapter seven is concerned with the configuration and application of the debugger.

Inhalt

1 EINLEITUNG....1

2 GRUNDLAGEN....5

2.1 HARDWARE....5

2.1.1 STMicroelectronics STM32F107VC ARM Cortex™-M3 Mikrocontroller....5

2.1.2 Keil MCBSTM32C Entwicklungsboard....7

2.1.3 Lüfter....9

2.1.3.1 Ein- und Ausgangssignale....10

2.1.4 Keil ULINK2 Debugger/Programmer....10

2.1.5 Verbinden der Hardware mit dem PC....11

2.2 PULSWEITENMODULATION (PWM)....12

2.3 SOFTWARE....13

2.3.1 Keil µVision4....13

2.3.2 Cortex Microcontroller Software Interface Standard (CMSIS)....13

2.3.3 STM32F10x Standard Peripherals Library....15

3 ERSTELLEN EINES TEMPLATES....17

3.1 EINBINDEN DER STM32F10X STANDARD PERIPHERALS LIBRARY....17

3.2 MINIMALE BEISPIELANWENDUNG....24

3.2.1 Initialisierung und Konfiguration des Mikrocontrollers (Template)....24

3.2.1.1 Konfiguration der Taktfrequenzen....24

3.2.1.2 Konfiguration der GPIO....27

3.2.2 Implementierung der Main-Funktion....28

4 IMPLEMENTIERUNG DER LÜFTERSTEUERUNG....31

4.1 AUFGABENSTELLUNG....31

4.1.1 Vorbereitung und Lösungsvorschlag....32

4.2 INITIALISIERUNG UND KONFIGURATION DES MIKROCONTROLLERS (LÜFTERSTEUERUNG) 32

4.2.1 Konfiguration der für die Peripherie benötigten Takte....33

4.2.2 Konfiguration der benötigten GPIOs und alternativer Funktionen (AFIO)....33

4.2.3 Konfiguration des PWM Timers....34

4.2.4 Konfiguration des DMA Controllers und des ADC....36

4.2.5 Konfiguration des externen Interrupts....38

4.2.6 Konfiguration des Timer 5....39

4.2.7 Konfiguration des Interruptcontrollers....40

4.2.8 Header-Datei der setup.c....41

4.3 IMPLEMENTIERUNG DER INTERRUPT SERVICE ROUTINEN (ISR)....42

4.3.1 Header-Datei der stm32f10x_it.h....44

4.4 IMPLEMENTIERUNG DER MAIN-FUNKTION....45

5 KEILS ECHTZEITBETRIEBSSYSTEM RTX....49

5.1 RTX KERNEL....50

5.2 TASK UND TASKVERWALTUNG....52

5.2.1 Erzeugen von Tasks....55

5.2.2 Priorisierung von Tasks....57

5.2.3 Taskverwaltung....57

5.2.4 Time Management....58

5.2.5 Virtuelle Timer....58

5.3 Intertask Kommunikation....59
5.3.1 Events....59
5.3.2 Interrupt-Behandlung....60
5.3.3 Semaphore....61
5.3.4 Mutex....62
5.3.5 Mailbox....63
5.4 Echtzeitscheduling Verfahren....66
5.4.1 System Timer....66
5.4.2 Round Robin (Zeitscheibenverfahren)....66
5.4.3 Pre-emptive Scheduling....67
5.4.4 Round Robin Pre-emptive Scheduling....68
5.4.5 Co-operatives Multitasking....69

6 EINBINDEN DER LÜFTERSTEUERUNG IN DAS ECHTZEITBETRIEBSSYSTEM....71
6.1 RTX Konfiguration der Entwicklungsumgebung....71
6.2 Anpassen der STM32F10x Standard Peripherals Library....72
6.3 Konfiguration des Echtzeitbetriebssystems....72
6.4 Einbinden der Lüftersteuerung in das Echtzeitbetriebssystem....74
6.4.1 Anpassen der Interrupt Service Routinen....74
6.4.2 Implementieren der main.c....75

7 DEBUGGING UND KONFIGURATION DES PROGRAMMERS/DEBUGGERS....83
7.1 Fehlersuche....83
7.2 Konfiguration des Debuggers/Programmers....84
7.3 Debugging....88
7.3.1 Watch Window....89
7.3.2 Logic Analyzer....90
7.3.3 Exception Tracing....92
7.3.4 Tracing des Echtzeitbetriebssystems und der Tasks....92

8 RESUMEE....95

9 VERZEICHNISSE....97
9.1 Literaturverzeichnis....97
9.2 Verzeichnis der Internetquellen....97

10 INHALT DER CD....101

11 ERKLÄRUNG....103

12 ANHANG....I

Verzeichnis der Abbildungen

Abbildung 1: KEIL MCBSTM32C Entwicklungsboard....8
Abbildung 2: Drehzahl/Spannung bei Ansteuerung über PWM....10
Abbildung 3: Keil ULINK2 Debugger....11
Abbildung 4: PWM Signal mit unterschiedlicher Pulsweite....12
Abbildung 5: Struktur der STM32 Library und der CMSIS....14
Abbildung 6: Neue Projektgruppe erstellen....19
Abbildung 7: µVision4 Projektstruktur....19
Abbildung 8: Anpassen der Suchpfade des Compilers....20
Abbildung 9: Suchpfade der Header-Dateien....20
Abbildung 10: Dateien dem Projekt hinzufügen....21
Abbildung 11: Benötigte Dateien....21
Abbildung 12: Einbinden benötigter Bibliotheks Header-Dateien (stm32f10x_conf.h) 23
Abbildung 13: Auswählen des verwendeten Mikrocontrollers (stm32f10x.h)....23
Abbildung 14: stm32f10x.h Konfiguration der Standard Peripherals Library....24
Abbildung 15: Konfiguration und Initialisierung der Grundtaktfrequenzen (setup.c). 27
Abbildung 16: Einschalten des Taktes für GPIOE (setup.c)....27
Abbildung 17: Fehlerbehandlung bei misslungener Taktinitialisierung (setup.c)....27
Abbildung 18: Konfiguration des GPIOE (setup.c)....28
Abbildung 19: Deklaration der Initialisierungsprototypen (setup.h)....28
Abbildung 20: Aufruf der Initialisierungsroutinen und Setzen der GPIOE-Pins 8 und 9 (main.c)....29
Abbildung 21: Speicherstelle der konvertierten ADC Werte (setup.c)....33
Abbildung 22: Anpassen der Setup_Clock() zum Konfigurieren und Einstellen des Takts (setup.c)....33
Abbildung 23: Anpassen der Setup_GPIO() zum Konfigurieren der GPIO (setup.c)..34
Abbildung 24: Konfiguration des PWM Timers (setup.c)....35
Abbildung 25: ADC1 Konfiguration mit DMA Datenübertragung (setup.c)....38
Abbildung 26: Konfiguration des externen Interrupts (setup.c)....39
Abbildung 27: Konfiguration des Timer 5 (setup.c)....40
Abbildung 28: Konfiguration der Interrupts (setup.c)....41
Abbildung 29: Header-Datei der Prozessorinitialisierung (setup.h)....42
Abbildung 30: Interrupt Service Routinen (stm32f10x_it.c)....44
Abbildung 31: Header-Datei der Interrupt Service Routinen (stm32f10x_it.h)....45
Abbildung 32: Flussdiagramm main.c....46
Abbildung 33: Main-Funktion (main.c)....48
Abbildung 34: Keil RTX Middleware Komponenten....50
Abbildung 35: Aufbau des Keil RTX Kernel....51
Abbildung 36: Mögliche Taskzustände während der Laufzeit....54
Abbildung 37: Deklaration einer Task ID....55
Abbildung 38: Erzeugen von 3 Tasks....55
Abbildung 39: Erzeugen eines Tasks mit Parameterübergabe....56
Abbildung 40: Anpassen der Stack Größe eines Tasks beim Erzeugen....56
Abbildung 41: Anpassen der Stack Größe und Übergabe eines Parameters bei Taskserzeugung....56
Abbildung 42: Priorisierung von Tasks....57
Abbildung 43: Systemaufrufe zur Taskverwaltung....57
Abbildung 44: Systemaufrufe für das Time Management....58

Abbildung 45: Virtueller Timer mit Call-Back Funktion in RTX_Config.c.....59
Abbildung 46: Systemaufrufe der Event Funktionen.....60
Abbildung 47: ISR mit Task zur Bearbeitung der Interruptbehandlung.....61
Abbildung 48: Sperrsynchronisation bei Zugriff auf Temperaturtabelle.....62
Abbildung 49: Mutex Sperrsynchronisation.....63
Abbildung 50: Implementierung eines Mailbox-Systems 1. Teil. Initialisierung.....65
Abbildung 51: Implementierung eines Mailbox-Systems 2. Teil. Versand.....65
Abbildung 52: Implementierung eines Mailbox-Systems 3. Teil. Empfang.....66
Abbildung 53: Round Robin.....67
Abbildung 54: Pre-emptive Scheduling.....68
Abbildung 55: Round Robin Pre-emptive Scheduling.....69
Abbildung 56: Co-operatives Multitasking.....69
Abbildung 57: Konfiguration des Mikrocontrollers für Betrieb mit RTX Kernel.....71
Abbildung 58: Konfiguration des RTX Kernels über den Configuration Wizzard.....73
Abbildung 59: Anpassen der Interrupt Service Routinen zum Betrieb mit dem RTX Kernel (stm32f10x_it.c).....75
Abbildung 60: Definition der benötigten Variablen (main.c).....76
Abbildung 61: Definition der benötigten Task Prototypen und Task-IDs.....77
Abbildung 62: Initialisierung Prozessor und Peripherie (main.c).....77
Abbildung 63: Erzeugen der Programmtasks durch ersten Initialisierungstask (main.c) 78
Abbildung 64: Definition des Tasks zum Schreiben des Displays (main.c).....79
Abbildung 65: Definition des Interrupt Tasks zum Zählen der Tachosignale des Lüfters über externe Interrupts.....80
Abbildung 66: Definition des Interrupt Tasks zum Berechnen der rpm mit Hilfe eines Timer Interrupts (main.c).....80
Abbildung 67: Task zum Einstellen des Tastverhältnisses.....81
Abbildung 68: Options for Target 'STM32F107VC' / Utilities.....85
Abbildung 69: Cortex-M Target Driver Setup / Flash Download.....85
Abbildung 70: Cortex-M Target Driver Setup / Debug.....86
Abbildung 71: Cortex-M Target Driver Setup / Trace.....87
Abbildung 72: Options for Target 'STM32F107VC' / Debug.....88
Abbildung 73: Control Buttons den Debuggers.....89
Abbildung 74: Watch1 Window mit Ausgabe der Lüftervariablen.....89
Abbildung 75: Symbol Window der µVision4 Debug Umgebung.....90
Abbildung 76: Logic Analyzer Window mit grafischer Darstellung des ADC-Wertes und Lüfterdrehzahl.....91
Abbildung 77: Setup Logic Analyzer Window.....91
Abbildung 78: Exeption Trace Window mit Ausgabe der EXTI15_10 und TIM5 Interrupts.....92
Abbildung 79: RTX Task and System Window mit Laufzeitinformationen des Echtzeitbetriebssystems.....94

Verzeichnis der Tabellen

Tabelle 1: Vergleich von Funktion und Task.....53
Tabelle 2: Auflistung der möglichen Task-Zustände des Keil RTX RTOS.....54
Tabelle 3: Mögliche Fehlerquellen.....84

Verzeichnis des Anhangs

Anhang I: Blockdiagramm STMicroelectronics STM32F107VC.....I
Anhang II Schaltplan KEIL MCBSTM32C Evaluationsboard.....II

Verzeichnis der Abkürzungen

ADC	Analog To Digital Converter
AFIO	Alternative Funktion Input/Output
APB	Advanced Peripheral Bus
APSR	Application Program Status Register
CAN	Controller Area Network
CHM	Compiled Help Module
CMSIS	Cortex Microcontroller Software Interface Standard
DMA	Direct Memory Access
EPSR	Execution Program Status Register
EXTI	Externer Interrupt
FIFO	First In First Out
GND	Ground
GPIO	General Purpose Input/Output
HSE	High Speed External Clock Signal
IPSR	Interrupt Program Status Register
IRQ	Interrupt Request
ISR	Interrupt Service Routine
JTAG	Joint Test Action Group
LCD	Liquid Crystal Display
NVIC	Nested Vectored Interrupt Controller
PLL	Phase-Locked Loop
PWM	Pulsweitenmodulation
RCC	Reset And Clock Control
RL-ARM	Keil Real-Time Library
RPM	Revolutions Per Minute
RTOS	Real Time Operating System
RTX	Keil Real-Time Operating System Kernel
SOC	System On A Chip
SPI	Serial Peripheral Interface
SRAM	Static Random Access Memory
SWD	Serial Wire Debug
SWV	Serial Wire Viewer
TCP/IP	Transmission Control Protocol/Internet Protocol
UART	Universal Asynchronous Receiver Transmitter

1 Einleitung

Die Technische Hochschule Mittelhessen wird ab dem Sommersemester 2012 im Fachbereich MNI eine neue Veranstaltung mit dem Titel „Praktikum für eingebettete Systeme“ anbieten.

Nach [MARWEDEL] können eingebettete Systeme als „*informationsverarbeitende Systeme die in ein umgebendes Produkt wie z. B. Autos, Telekommunikationsgeräte oder Produktionsmaschinen eingebettet sind*“ definiert werden. Dabei verrichten diese Systeme weitestgehend unsichtbar ihren Dienst in einer Vielzahl von Geräten in Bereichen wie der Medizin, Telekommunikation, Bordelektronik in Flugzeugen, Unterhaltungselektronik, und militärischen Anwendungen. Als Verarbeitungseinheiten innerhalb dieser Systeme erlangen Mikrocontroller immer größere Bedeutung. Ihre großen Vorteile im Gegensatz zu anwendungsspezifisch integrierten Schaltkreisen mit festverdrahteter Funktionalität (ASICs[1]) sind zum einen die geringeren Anschaffungskosten im Vergleich zu ASICs und zum anderen ihre hohe Flexibilität. Mit ihnen ist es möglich das Verhalten eingebetteter Systeme anzupassen oder zu verändern indem man die auf dem Mikrocontroller laufende Software einfach austauscht. Das ermöglicht Entwurfsfehler zu beseitigen, das System an veränderte Bedingungen anzupassen oder es durch effizientere Algorithmen weiter zu optimieren.

Mikrocontroller sind Mikroprozessoren bei denen Speicher und weitere Peripherie wie z. B. DMA Controller, UART und analoge Ein- und Ausgänge auf einem einzigen Chip vereint wurden. Sie werden auch als System-on-a-Chip oder kurz SoC bezeichnet. Die wesentlichen Unterscheidungsmerkmale zwischen Mikrocontrollern sind die Breite des internen Datenbusses und ihre Architektur. Der hier verwendete Mikrocontroller, STMicroelectronics STM32F107VC, ist ein relativ neuer Controller und basiert auf einer ARM Cortex™-M3 Mikroprozessor Architektur.

Ziel dieser Arbeit ist es, ein Konzept zu entwerfen das dem Leser einen einführenden Überblick in die Implementierung von Software für den Cortex™-M3 Mikrocontroller unter Verwendung eines Echtzeitbetriebssystems zur Verfügung stellt. Dazu gehört die Verwendung der Interrupts, einfacher und spezieller Timer, Analog to digital Wandler, externer Interrupts und die grundlegende Konfiguration des Prozessors.

[1] Application-Specific Integrated Circuits

Heutige Mikrocontroller haben auf Grund der Menge an Funktionalität, Peripherie und dazugehöriger Literatur einen Grad an Komplexität erreicht der es dem Einsteiger im Gebiet der Mikrocontroller Programmierung sehr erschwert, sich in diese einzuarbeiten. Die Fülle an Informationen welche innerhalb der Fachliteratur, Reference Manuals und Datasheets angeboten werden und in der Regel für erfahrene Programmierer verfasst wurden lässt den Studierenden schnell den Überblick über das Wesentliche verlieren.
Deshalb muss die Komplexität des Mikrocontrollers reduziert d. h. ein Grundgerüst zur Verfügung gestellt werden, welches den Studierenden in die Lage versetzt die einzelnen Komponenten des Mikrocontrollers zueinander in Beziehung zu setzen und sich weitere Themen selbstständig erarbeiten zu können.
Um dies zu erreichen soll eine exemplarische Anwendung in Form einer Lüftersteuerung implementiert und anschließend in ein Echtzeitbetriebssystem eingebunden werden. Dazu wird zu Beginn eine Abstraktionsschicht in Form von Bibliotheken eingefügt die Register und Funktionen des Prozessors gut verständlich benennt und eine große Menge an Beispielanwendungen für spätere Projekte zur Verfügung stellt. Des Weiteren werden alle Implementierungen abgebildet und Schritt für Schritt erklärt um die Verwendung der Bibliotheken zu zeigen und ein Verständnis für deren logischen Aufbau zu vermitteln. Außerdem wird in jedem Kapitel auf weiterführende Literaturstellen verwiesen die den Sachverhalt genauer beschreiben.

Um ein möglichst lineares Lesen dieser Arbeit zu ermöglichen ist sie in nachfolgende Kapitel unterteilt.
Im Anschluss an die Einleitung beschäftigt sich Kapitel zwei mit den Grundlagen der für die Softwareimplementierung auf Mikrocontrollern wichtigen Komponenten. Es wird ein Überblick erstellt der alle wichtigen Informationen enthält um der Projektentwicklung folgen zu können. Dazu gehört die hier verwendete Hard- und Software und die Funktionsweise der Pulsweitenmodulation.
Im dritten Kapitel wird durch die Implementierung einer minimalen Beispielanwendung ein Template erstellt welches die Basis für die weiteren Projekte sein soll. Weiterhin wird die STM32F10x Standard Peripherals Library dem Projekt hinzugebunden und konfiguriert.

Aufbauend auf dem zuvor erstellten Template wird in Kapitel vier eine Lüftersteuerung implementiert welche die Initialisierung des Prozessors und seiner Peripherie mit Hilfe der zuvor eingebundenen Bibliothek beschreibt.
Kapitel fünf befasst sich mit der Funktionsweise und Verwendung des KEIL RTX Echtzeitbetriebssystems.
Aufbauend auf den beiden letzten Kapiteln wird in Kapitel sechs die Konfiguration des Keil RTX Echtzeitbetriebssystem erklärt und anschließend die zuvor implementierte Lüftersteuerung in dieses eingebunden.
In Kapitel sieben wird die Konfiguration und Verwendung des Debuggers und Programmers des Mikrocontroller beschrieben.

2 Grundlagen

In diesem Kapitel werden alle zur Realisierung der Lüftersteuerung benötigten Hard- und Software Komponenten und die Funktionsweise der Pulsweitenmodulation beschrieben.

2.1 Hardware

Mikrocontroller, Entwicklungsboard, Lüfter und Debugger/Programmer sind die wesentlichen Hardwarekomponenten, die für die Entwicklung der Lüftersteuerung verwendet werden. Evtl. werden weitere Werkzeuge wie Funktionsgenerator, Oszilloskop und Multimeter benötigt, auf die hier aber nicht weiter eingegangen wird.

2.1.1 STMicroelectronics STM32F107VC ARM Cortex™-M3 Mikrocontroller

Der in dieser Arbeit verwendete Mikrocontroller STM32F107VC wurde nicht von einem einzigen Hersteller[2] entwickelt und produziert. Aus diesem Grund werden während der Softwareentwicklung in dieser Arbeit auch verschiedene Bibliotheken wie der CMSIS[3] von ARM, die STM32F10x Standard Peripheral Library von STMicroelectronics und das Echtzeitbetriebssystem RTX von Keil verwendet.
Der STM32F107VC gehört zur Familie der STM32 Mikrocontroller mit einer 32-Bit ARM Cortex™-M3 Architektur. Ziel der Entwicklung war, einen günstigen und im Vergleich zum ARM 7 weniger komplexen, aber trotzdem schnellen Mikrocontroller zu entwerfen. Der Cortex™-M3 besitzt zur Befehlsverarbeitung eine dreistufige Pipeline: Instruction Fetch, Instruction Decode und Instruction Execute. Diese Pipeline arbeitet bei linear abgearbeitetem Code sehr effizient, da immer parallel zur Ausführung ein neuer Befehl decodiert und ein weiterer aus dem Speicher gelesen werden kann. Weiterhin hat der Prozessor eine integrierte Optimierung, welche beim Erkennen eines Sprungbefehls in der Dekodierungsstufe den Befehl der vermuteten Zieladresse lädt (spekulative instruction fetch). Sollte es zur Ausführung dieses Sprungs kommen, steht

[2] ARM (Entwicklung der Architektur), STMicroelectronics (Produktion und Erweiterung der Peripherie)
[3] Cortex Microcontroller Software Interface Standard

der Befehl innerhalb eines Taktes zur Verfügung. Im anderen Fall würde der nächste Befehl bereits durch den linearen Prefetch in der Pipeline liegen.

Die dem Cortex™-M3 zu Grunde liegende Architektur ist eine Kombination aus der Harvard- und der Von Neuman-Architektur. Sie vereint derer Vorteile: den linearen 4 GB Adressraum der Von Neuman- und die getrennte Busstruktur der Harvard-Architektur. Dabei ist der Flash-Speicher über einen eigenen Instruction Bus mit dem Kern verbunden, welcher nur für das Lesen von Befehlen verwendet wird. Der Zugriff auf die Daten erfolgt über den Datenbus, welcher über eine Busmatrix mit Arbitter[4] die den Prozessor mit DMA[5] Controller, SRAM[6], Flash-Speicher und den Peripheriebussen APB1[7] und APB2 verbindet. Der Prozessorkern und der DMA Controller teilen sich als Busmaster den Datenbus, wobei dem Prozessor, um diesen nicht auszubremsen, 33% der Bandbreite garantiert wird. Die Aufgabe des DMA Controllers ist das Übertragen von Peripherie- und Speicherdaten ohne den Prozessorkern zu belasten.

Der Prozessors arbeitet mit dem Thumb2 Befehlssatz. Dieser ist ein mit 32-Bit Befehlen erweiterter Thumb Befehlssatz; letzterer wird von den älteren ARM 7 Prozessoren verwendet. Die wesentlichen Neuerungen des Thumb2 sind Native Bitfield Manipulation, Hardware Division und If/Then Instructions. Weiterhin soll der Befehlssatz auch auf die Erzeugung eines kleineren Maschinencode optimiert worden sein. Laut Richard Phelan[8] soll sich ein um 26% kleinerer Maschinencode mit dem neuen Befehlssatz generieren lassen.

Ein weiterer großer Vorteil der Cortex™-M3 Architektur ist, dass der Mikrocontroller zwei verschiedene Betriebsmodi zur Verfügung stellt, welche die stabile Nutzung eines Echtzeitbetriebssystems ermöglichen. Es gibt einen Thread- und einen Handler-Mode wobei Letzterem die Interrupt-Behandlung vorbehalten ist. Hinzu kommt, dass der Prozessor in der Lage ist, die Befehle im privilegierten und unprivilegierten Modus auszuführen und dadurch Haupt- von Prozess-Stack zu trennen. Außerdem stehen dem Prozessor im unprivilegierten Modus verschiedene Befehle, wie z. B. die Befehle MRS und MSR für den Zugriff auf die Programmstatusregister[9], nicht mehr zur Verfü-

[4] Funktionseinheit zum Lösen von Zugriffskollisionen bei Bussystemen mit mehreren Busmastern
[5] Engl. Direct Memory Access
[6] Abk. Static Random Access Memory
[7] Abk. Advanced Peripheral Bus
[8] Embedded CPU manager, ARM
[9] Die Programmstatusregister bestehen aus Application Program Status Register (APSR), Interrupt Program Status Register (IPSR) und Execution Program Status Register (EPSR) und enthalten z. B. die Nummer des aktuellen Interrupts

gung. Im Standard-Modus führt ein Cortex™-M3 den Quellcode im Thread und Handler Modus mit privilegierten Rechten aus. Es gibt keine Restriktionen bzgl. des Zugriffs auf Prozessorressourcen und die Prozesse beider Modi teilen sich den Haupt-Stack. Bei der Nutzung eines Echtzeitbetriebssystem ist es jedoch möglich, den Prozessor so zu konfigurieren, dass das Echtzeitbetriebssystem und die Interrupts im privilegierten Handler-Mode laufen und sich den Haupt-Stack teilen. Der eigentliche Applikationscode wird im unprivilegierten Thread Modus ausgeführt und nutzt den Prozess-Stack. Das hat den Vorteil, dass ein Fehler in der Applikation das Betriebssystem nicht zum Abstürzen bringen kann, da System- und Programmspeicher voneinander getrennt wurden.[10]

Die weiteren herausragenden Merkmale des Mikrocontrollers wie der Nested Vectored Interrupt Controller und die integrierte on Chip Debug Schnittstelle werden in späteren Kapiteln genauer beschrieben.

2.1.2 Keil MCBSTM32C Entwicklungsboard

Entwicklungs-, auch Evaluationsboards genannt, werden meist als Starter Kit, Reference Design Kit und System Design Kit von den jeweiligen Herstellern der Mikrocontroller und weiteren Drittanbietern hergestellt und vertrieben. Solche Kits bestehen typischerweise aus einem Debugger, dem Entwicklungsboard und einer Entwicklungsumgebung. Ein Vorteil eines solchen Kits ist, dass die einzelnen Komponenten mit relativ wenig Konfigurationsaufwand sehr gut zusammenarbeiten. Entwicklungsboards wurden oft als Bausatz geliefert, was heute aber auf Grund der großen Pin-Zahlen auf den Mikrocontrollern nur noch schwer möglich ist. Bei heutigen Mikrocontrollern sind ohne weiteres 100 und mehr Pins möglich. Daher werden Entwicklungsboards heute in der Regel fertig bestückt ausgeliefert.

Keil wurde 2005 von ARM Limited übernommen. ARM ist der Hersteller der Mikroprozessorarchitektur Cortex™-M3, auf welcher der auf dem Board verbaute Prozessor beruht. Dieser Prozessor gehört zu den Connectivity Line Devices der auf ARM Cortex™-M3 basierenden Mikrocontrollern, weshalb dieses Board sehr viele Schnittstellen, wie z. B. USB, Can-Bus und Ethernet zur Verfügung stellt.

[10] Vgl. [MARTIN], [STM_REF_MAN], [STM_BLOCKDIA], [EESTASIA], [ELEKTR_NET]

Wesentliche Merkmale des Keil MCBSTM32C[11] Entwicklungsboards sind:

1. 72MHz STM32F107VC ARM Cortex™-M3 basierter Mikrocontroller
2. Externer I²C[12] 8KB Flash Baustein
3. Farb QVGA TFT LCD mit Touchscreen
4. 10/100 Ethernet Port
5. USB 2.0 Full Speed und USB Host
6. MicroSD Card Interface
7. 5 - Positionen Joystick
8. 2 CAN Interfaces
9. 3 - Achsen Beschleunigungssensor
10. Analoge Spannungsversorgung für den ADC[13]
11. 80 GPIO Pins
12. Debug Interface für 20 Pin JTAG/SW[14]

In Abbildung 1 ist das verwendete Entwicklungsboard mit seinen verschiedenen Anschlüssen zu sehen.

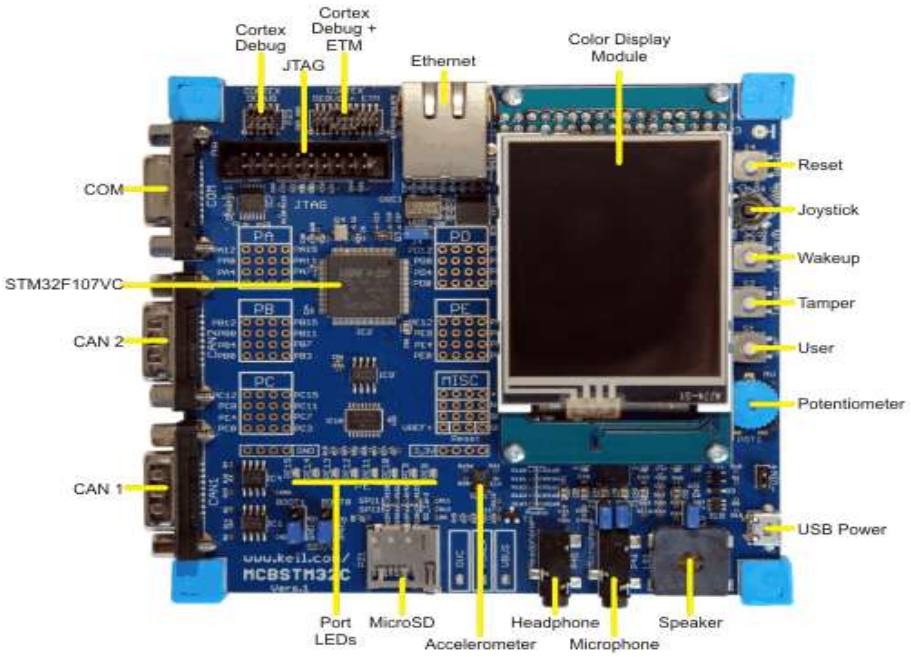


Abbildung 1: KEIL MCBSTM32C Entwicklungsboard

Quelle: www.mikrozone.sk

[11] Weiterführende Literatur [KEIL_EVAL_FEAT]
[12] I²C oder Two Wire Interface ist ein serieller, von Philips Semiconductors entwickelter Datenbus
[13] Abk. Analog-to-Digital Converter
[14] Abk. Joint Test Action Group/Serial Wire

2.1.3 Lüfter

Für den Versuchsaufbau wird ein SilentWings^{PWM} 80mm Standard-Gehäuselüfter verwendet. Dieser wird in der Regel in Heimcomputern verbaut. Er hat 4 Anschlusspins:

- Masse (Schwarzes Kabel)
- +12V (Rotes Kabel)
- Tachoausgang (Gelbes Kabel)
- Drehzahlsteuerung (PWM[15]) (Blaues Kabel)

Der Lüfter besitzt eine eigene Steuerungselektronik (Black Box[16]). Diese setzt das in den Lüfter eingehende PWM Signal um. Der Hersteller gibt bezüglich der Steuerungselektronik keine Informationen an die Hand. Aus diesem Grund wird davon ausgegangen, dass sich der Hersteller an die Vorgabe und Spezifikation[17] der Mainboardhersteller hält.

Technische Daten:

- Abmessungen (L x B x H) 80 mm x 80 mm x 25 mm
- Umdrehungen bei 12V 2000 rpm[18] [19]
- Fördervolumen bei 12V 44,2 m³/h
- Nennspannung 12V
- Anlaufspannung 3,5V
- Arbeitsspannungsbereich 3,5V ~ 14V
- Nennstrom 0,08 A
- Leistungsaufnahme 1,44 Watt
- Drehzahl / Spannung bei PWM Ansteuerung wie in Abbildung 2 zu sehen

[15] Abk. Pulsweitenmodulation
[16] System/Objekt mit definierten Schnittstellen, dessen innerer Aufbau unbekannt ist
[17] Vgl. [PWM_SPEC]
[18] Abk. revolutions per minute
[19] Bei 100% PWM oder bei 12V ohne PWM

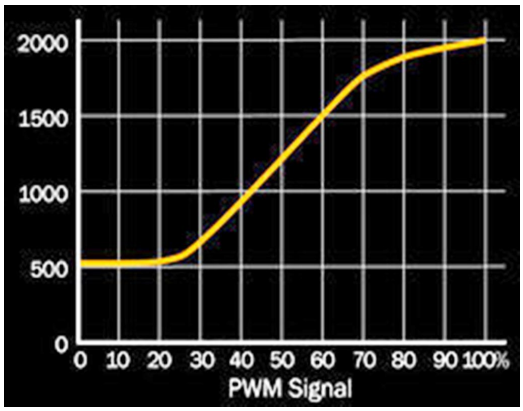


Abbildung 2: Drehzahl/Spannung bei Ansteuerung über PWM

Quelle: www.be-quiet.net

2.1.3.1 Ein- und Ausgangssignale

Der Tacho erzeugt zwei Signale pro Umdrehung. Um diese einzulesen wird ein Pin des Mikrocontrollers als Eingangspin mit aktivem Pullup Widerstand konfiguriert.
Das vom Mikrocontroller generierte PWM-Signal muss laut Spezifikation eine Frequenz zwischen 21KHz und 28KHz haben und der Pin des Mikrocontrollers muss als alternativ funktionierender Open Drain Ausgang konfiguriert werden.

2.1.4 Keil ULINK2 Debugger/Programmer

Der Debugger gehört mit zu den wichtigsten Werkzeugen bei der Entwicklung von Software auf Mikrocontrollern. Der in dieser Arbeit verwendete Keil ULINK2 Debugger ist in Abbildung 3 zu sehen. Seinen Strom bezieht er über einen USB-Port, über welchen er auch die entwickelten Programme in den Flash Speicher des Mikrocontrollers schreibt. Ein wesentliches Merkmal des Debuggers ist die Unterstützung der Serial Wire Debug (SWD)[20] und Serial Wire Viewer (SWV)[21] Schnittstelle des Mikrocontrollers. Diese ermöglichen es dem Debugger während der Laufzeit auf den Systemspeicher zuzugreifen und Daten über den Serial Wire Viewer auszugeben, was auch als Trace oder Tracing bezeichnet wird.[22]

[20] Weiterführende Literatur [ARM_SWD]
[21] Weiterführende Literatur [ARM_SWV]
[22] Vgl. [KEIL_ULINK]

Seine wesentlichen Merkmale sind:

- Unterstützung verschiedener Mikrocontroller
- Programmierung des Mikrocontrollers
- Untersuchen der Speicherregister
- Analyse des Programms über Einzelschritte oder Breakpoints
- Unterstützung von JTAG oder Serial Wire Debug
- Debugging während der Laufzeit



Abbildung 3: Keil ULINK2 Debugger

Quelle: www.arm.com

2.1.5 Verbinden der Hardware mit dem PC

Das Entwicklungsboard wird über einen 20 Pin Pfostenstecker mit dem Debugger ULINK2, der Debugger wiederum über ein USB-Kabel mit dem Rechner verbunden. Für die Stromversorgung des Entwicklungsboards wird ein weiteres USB–Kabel benötigt und mit der Micro–USB Schnittstelle des Entwicklungsboards verbunden.
Der Lüfter mit seinen vier Anschlussleitungen wird wie folgt verkabelt:

- Das **rote** Kabel wird an 12 Volt des Netzteils angeschlossen
- Das **schwarze** Kabel wird an die Masse (GND) des Netzteils angeschlossen
- Das **gelbe** Kabel für den Tachoausgang wird mit Pin **PD11** verbunden
- Das **blaue Kabel** für die Drehzahlsteuerung wird mit Pin **PA8** verbunden

- Ein weiteres Kabel muss die Masse des Entwicklungsboards mit der Masse des Netzteils verbinden.

! Da es im Laufe des Projektes zu Fehlkonfigurationen durch den Programmierer kommen kann, sollte der Lüfter zu Beginn in keinem Fall angeschlossen werden. Dies könnte bei falscher Pin-Konfiguration das Entwicklungsboard zerstören.

2.2 Pulsweitenmodulation (PWM)

Der verwendete Lüfter wird über Pulsweiten-, auch Pulslängenmodulation genannt, gesteuert. Aus diesem Grund sollen in diesem Kapitel einige Grundlagen bezüglich dieser Modulationsart besprochen werden.

PWM wird in der Regel in der Steuerungstechnik zum Ansteuern großer und träger Lasten, wie z. B. Elektromotoren verwendet. In Mikrocontrollern sind daher häufig ein oder mehrere spezielle PWM Ausgänge integriert. Die PWM ist eine digitale Modulationsart, was bedeutet, dass Impulse immer mit voller Spannung an die Last gesendet werden. Dabei wird das Verhältnis der Ein- und Ausschaltzeit innerhalb einer festen Grundfrequenz variiert. Das Verhältnis zwischen dieser Ein- und Ausschaltzeit wird als Tastverhältnis[23] und die Frequenz als Periodendauer bezeichnet.

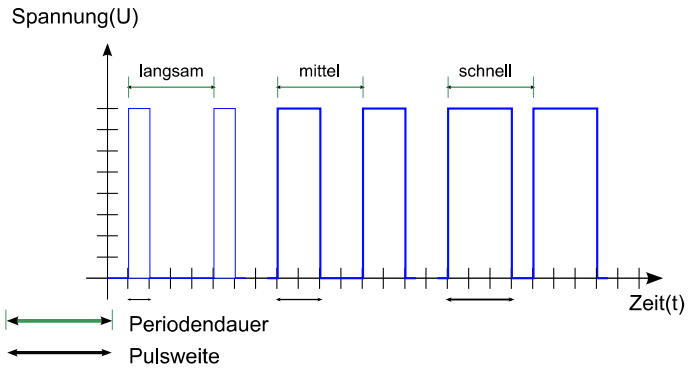


Abbildung 4: PWM Signal mit unterschiedlicher Pulsweite

Quelle: [RN_WISSEN]

Abbildung 4 verdeutlicht, wie der Lüfter in drei verschiedenen Geschwindigkeiten geregelt wird, wobei gut zu erkennen ist, dass die Frequenz konstant bleibt und sich lediglich das Verhältnis der Ein- und Ausschaltzeit innerhalb dieser ändert. Dieses Verhältnis bestimmt auch die mittlere Spannung des PWM Signals.

[23] Engl. Duty Cycle

$U_{mittel} = mittlere\ Spannung$
$U_{ein} = PWM\ Spannung$ 12V
$t_{an} = Zeit\ bei$ 5V
$t_{aus} = Zeit\ bei$ 0V

$$U_{mittel} = U_{ein} * \frac{t_{an}}{t_{an} + t_{aus}}$$

Der STM32F107VC Mikrocontroller verfügt dazu über einen 16 Bit PWM Timer, welcher es ermöglicht das Tastverhältnis in bis zu 2^{16} Einzelschritte zu regeln.[24]

2.3 Software

In diesem Kapitel soll die in dieser Arbeit verwendete Software vorgestellt werden. Sie besteht im Wesentlichen aus der Entwicklungsumgebung und den Bibliotheken Cortex Microcontroller Software Interface Standard und der STM32F10x Standard Peripherals Library. Das RTX Echtzeitbetriebssystem wird in Kapitel 5 gesondert behandelt.

2.3.1 Keil µVision4

Die Entwicklungsumgebung µVision4 gehört zu den Entwicklungstools von Keil zur Programmierung von Mikrocontrollern und vereint folgende Funktionen in einer Anwendung:

- Quellcode Editor und Compiler
- Projekt Management
- Programm Debugger und Analysetools
- Simulator

Auf einzelne Funktionen der Entwicklungsumgebung, wie z. B. das Debugging und die Analysetools wird in späteren Kapiteln genauer eingegangen.[25]

2.3.2 Cortex Microcontroller Software Interface Standard (CMSIS)

Auf Basis des Cortex™-M basierende Mikrocontroller erlangen in der Industrie immer größere Verbreitung. Dies hat zur Folge, dass diese zu einem Industrie-Standard wer-

[24] Vgl. [RN_WISSEN], [MIKROCON_NET_PWM]
[25] Weiterführende Literatur [KEIL_UVISION4_MAN]

den. Der CMSIS ist eine herstellerunabhängige Hardwareabstraktionsschicht für die Cortex™-M-Prozessor-Serie. Er realisiert ein einheitliches Konzept für Schnittstellen zu Peripheriefunktionen, Echtzeit-Betriebssystemen und Middleware-Komponenten. Dies vereinfacht es entwickelte Software wiederzuverwenden, beschleunigt die Einarbeitungszeit in neue Mikrocontroller und verkürzt somit auch die Entwicklungszeit neuer Geräte. Der CMSIS wurde in enger Zusammenarbeit mit einer Reihe wichtiger Halbleiter- und Softwareanbieter entwickelt, worunter auch Keil und STMicroelectronics waren. In der STM32F10x Standard Peripheral Library wird die CMSIS Version 1.30 mitgeliefert und integriert sich, wie in Abbildung 5 zu sehen, in die Abstraktionsschichten.[26]

Dabei liefert der CMSIS folgende Abstraktionen, die sich auf den Zugriff des Prozessorkerns und dessen Peripherie beziehen:

- Namensdefinitionen
- Adressdefinitionen
- Hilfsfunktionen für den Zugriff auf Register
- Hilfsfunktionen für den Zugriff auf Peripherie

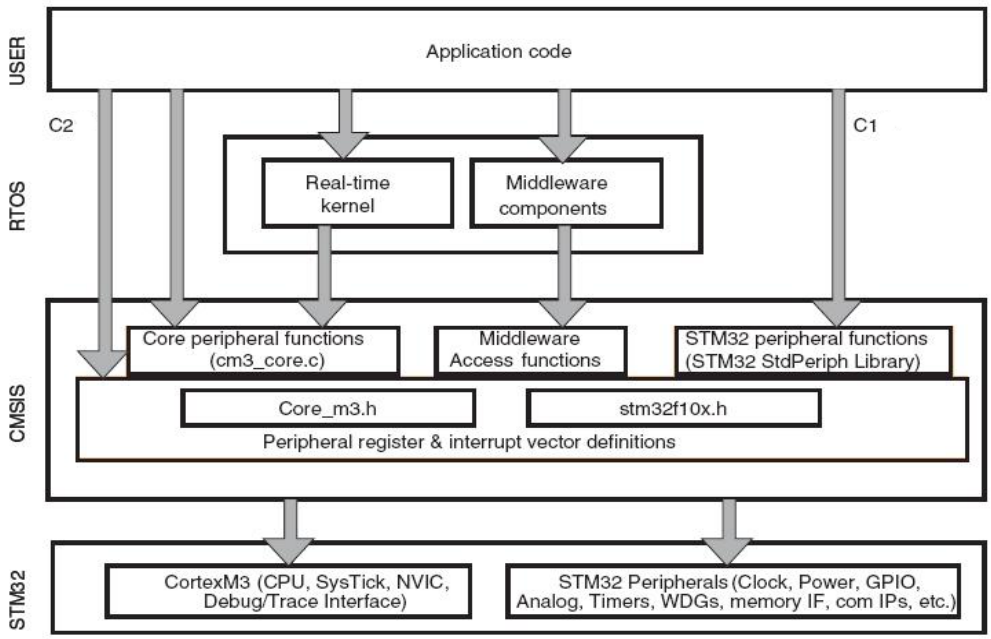


Abbildung 5: Struktur der STM32 Library und der CMSIS

Quelle: www.mikrocontroller.net

[26] Vgl. [ARM_CMSIS]

2.3.3 STM32F10x Standard Peripherals Library

Die STM32F10x Standard Peripherals Library ist eine von STMicroelectronics entwickelte und kostenlos zur Verfügung gestellte C-Bibliothek. Diese umfangreiche und komfortable Bibliothek bietet dem Programmierer Zugriff auf alle Funktionen der Mikrocontroller-Reihe STM32F10x. Für die verschiedenen Peripheriekomponenten stehen jeweils eigene Module zur Verfügung. Die Funktionsweise der einzelnen Module wird an Hand verschiedener Beispiele und einer Beschreibung in Form einer .chm[27] Datei erklärt. Der Hauptgrund, warum diese Bibliothek hier verwendet wurde, ist, dass diese es ermöglicht, mit verhältnismäßig geringen Kenntnissen bezüglich des Prozessors nach kurzer Zeit funktionsfähige Software zu entwickeln. Dies verkürzt die Einarbeitungszeit und hält so beim Anfänger den Grad der Frustration niedrig. Das Einbinden der Bibliothek in ein Projekt und deren Verwendung wird in den Kapiteln 3 und 4 an Hand eines Templates und einer Beispielanwendung demonstriert. Abbildung 5 zeigt die Integration der Bibliothek im Schichtenmodel und deren Co-Existenz mit der CMSIS. Wobei dort abgebildet **C1** den Code basierend auf der Standard Peripheral driver API und **C2** den direkten Zugriff auf die Register der Peripherie darstellt.

[27] Abk. Compiled Help Module

3 Erstellen eines Templates

Ein Template ist eine Mustervorlage oder auch Schablone und wird als Unterbau für das eigentliche Projekt verwendet. Dies bedeutet, dass dieses und alle weiteren Projekte auf diesem Template aufbauen können. Die von der Standard Peripherals Library zur Verfügung gestellten Templates sind leider nicht brauchbar, da diese für die von STMicroelectronics entwickelten Evaluationsboards implementiert wurden. Diese Boards besitzen einen anderen Hardwareaufbau als das hier verwendete Entwicklungsboard von Keil, was zur Folge hat, dass man solch ein Template umschreiben müsste um es verwenden zu können. Aus diesem Grund soll hier ein neues Template erstellt werden welches den Mikrocontroller konfiguriert, initialisiert und zwei GPIOs[28] auf dem Entwicklungsboard einschaltet.

3.1 Einbinden der STM32F10x Standard Peripherals Library

In diesem Kapitel soll die STM32F10x Standard Peripherals Library eingebunden werden. Dazu wird im ersten Schritt folgende Verzeichnisstruktur aufgebaut:

- Mein_Projekt\Project\app\
- Mein_Projekt\Libraries
- Mein_Projekt\Examples

Im Folgenden wird aus der Bibliothek (STM32F10x Standard Peripherals Library[29]) der gesamte Inhalt der beiden Ordner

- ...\STM32F10x_StdPeriph_Lib_V3.5.0**Libraries**
- ...\STM32F10x_StdPeriph_Lib_V3.5.0\Project\STM32F10x_StdPeriph_**Examples**

in die entsprechenden zuvor erstellten Verzeichnisse kopiert. Des Weiteren sollte noch die folgende Hilfsdatei

- ...\STM32F10x_StdPeriph_Lib_V3.5.0**stm32f10x_stdperiph_lib_um.chm**

[28] Abk. General Purpose Input/Output
[29] http://www.st.com/internet/mcu/product/221020.jsp

in den Projektordner kopiert werden. Es hat sich herausgestellt, dass die einzelnen Versionen dieser Bibliothek nicht immer abwärtskompatibel sind. Dies hat zur Folge, dass ältere Projekte mit einer neueren Bibliothek nicht immer funktionieren. Aus diesem Grund sollte jedes Projekt seine eigene Bibliothek enthalten, damit auch für die Zukunft sicher gestellt ist, dass die Projekte noch funktionieren und man trotzdem bei aktuellen Projekten die neuesten Bibliotheken verwenden kann.
Nachdem die Entwicklungsumgebung geöffnet wurde, wird über die Menüleiste unter **Project → New µVision Project...** ein neues Projekt erstellt. Dieses Projekt wird unter dem Namen **mein_template** im Ordner **Mein_Projekt\Project** gespeichert. Daraufhin erscheint ein Pop up Menü, in welchem man den verwendeten Mikrocontroller **STMicroelectronics STM32F107VC** auswählt. Die darauf folgende Frage, ob der „Connectivity Line Startup Code“ dem Projekt hinzugefügt werden soll, wird verneint, da in dem Template der aktuellste Startup Code des Herstellers verwendet werden soll. Nach Verneinen dieser Frage erscheint die Entwicklungsumgebung mit dem Projektfenster auf der linken Seite. Mit einem Klick der rechten Maustaste auf **Target1 → Add Group...** können innerhalb des Projektfensters weitere Untergruppen erstellt werden. Siehe dazu Abbildung 6. Ein Klick mit der rechten Maustaste ermöglicht es auch die Gruppennamen zu ändern. Daraufhin sollte die in Abbildung 7 beschriebene Gruppenstruktur aufgebaut werden. Das hat rein kosmetische Gründe und ermöglicht es dem Programmierer, den Überblick über das Projekt zu behalten da im Laufe des Projekts einige Bibliotheksdateien hinzugebunden werden müssen.

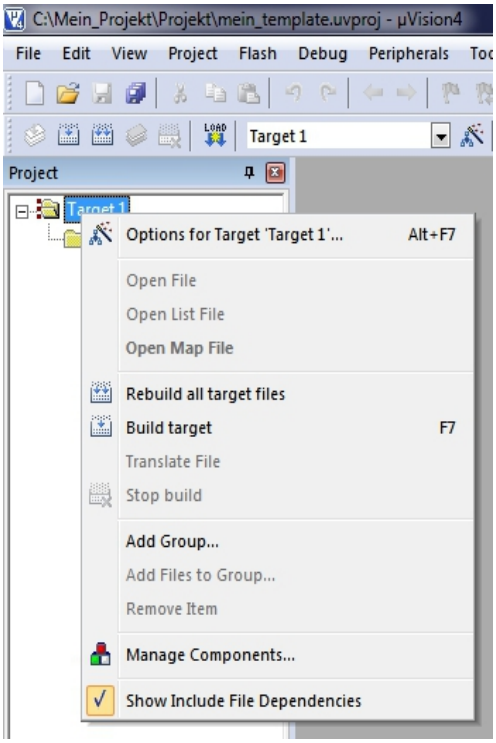


Abbildung 6: Neue Projektgruppe erstellen

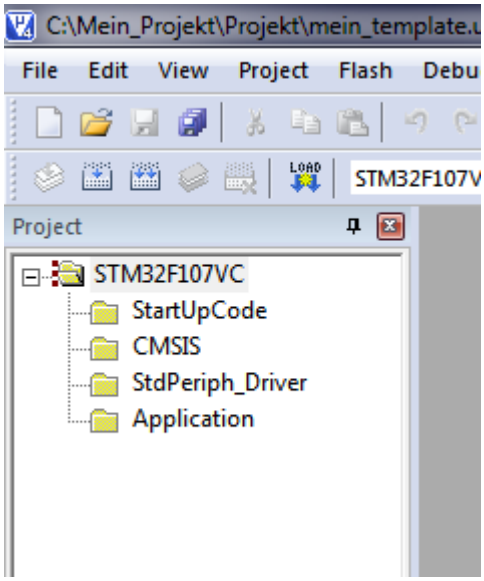


Abbildung 7: µVision4 Projektstruktur

Die Anpassung der Suchpfade des Compilers ist zwingend erforderlich, da die Entwicklungsumgebung eine ältere Version der Standard Peripherals Library zur Verfügung stellt und immer erst dort nach den Bibliotheken sucht. Dies würde zu Mehrfachdefinitionen bei der Compilierung führen. Zum Anpassen der Suchpfade wird in der Menüleiste unter **Flash → Configure Flash Tools...** das Optionsmenü des Projekts geöffnet und das Register **C/C++** gewählt. Abbildung 8 verdeutlicht dies.

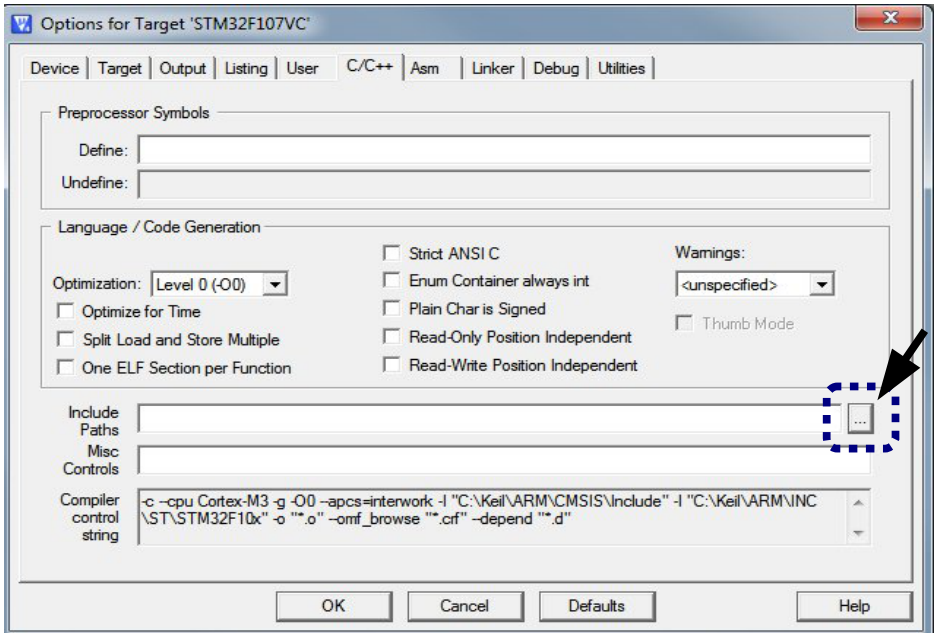


Abbildung 8: Anpassen der Suchpfade des Compilers

Dort werden die in Abbildung 9 zu sehenden Suchpfade angepasst.

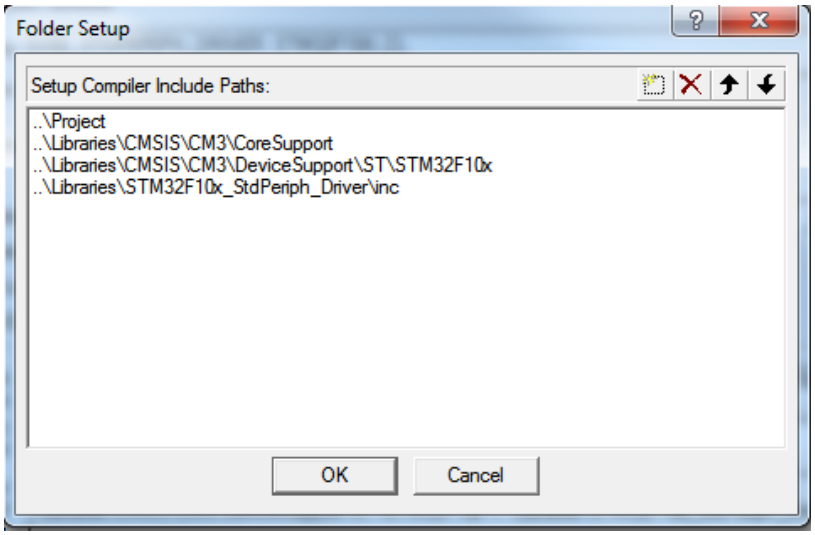


Abbildung 9: Suchpfade der Header-Dateien

Nachdem dies geschehen ist, können die für das Template wichtigen Dateien, wie in Abbildung 10 beschrieben, dem Projekt hinzugefügt und konfiguriert werden. Diese Dateien bestehen aus dem Start-Up Code des Prozessors und einem Minimum an Bibliotheksdateien, welche es ermöglichen mit Hilfe der Bibliotheksfunktionen die GPIO des Mikrocontrollers anzusteuern (siehe dazu auch Abbildung 11). Der Start-Up Code ist für die grundlegende Initialisierung des Prozessors verantwortlich. In ihm wird un-

ter anderem das Clock System konfiguriert, der Programm Counter und Stack Pointer initialisiert und der Betriebsmodus (Handler/Thread) eingestellt.

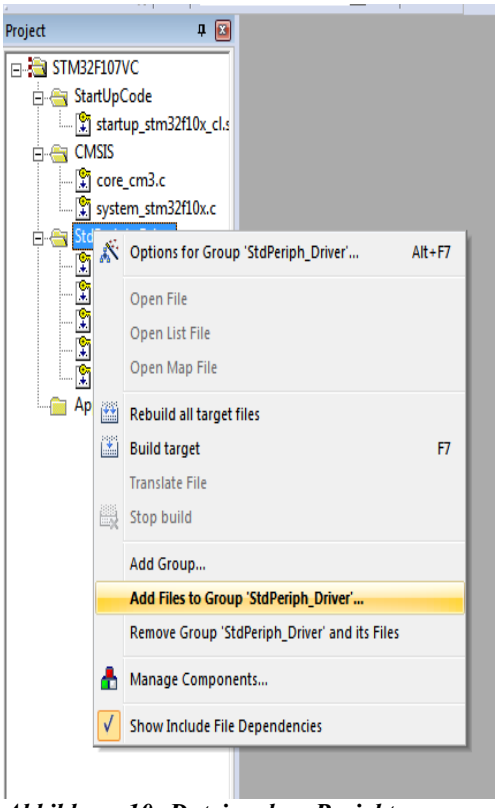


Abbildung 10: Dateien dem Projekt hinzufügen

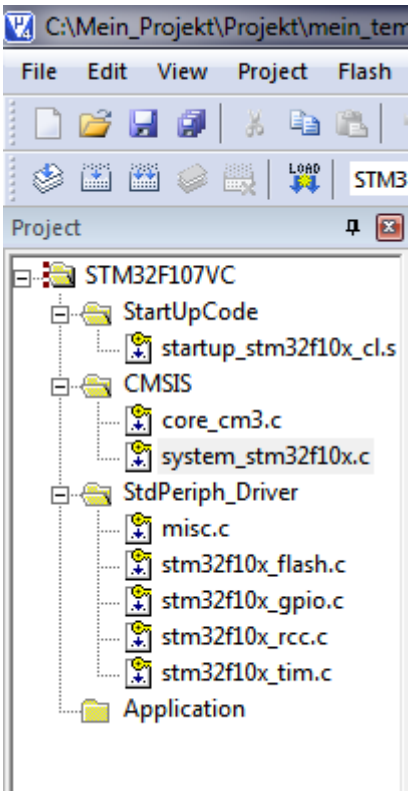


Abbildung 11: Benötigte Dateien

Die Dateien sind in folgenden Ordnern zu finden:

- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x\startup\arm**startup_stm32f10x_cl.s**
- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\CM3\CoreSupport**core_cm3.c**
- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x**system_stm32f10x.c**
- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\STM32F10x_StdPeriph_Driver\src**misc.c**
- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\STM32F10x_StdPeriph_Driver\src**stm32f10x_flash.c**

- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\STM32F10x_StdPeriph_Driver\src**stm32f10x_gpio.c**
- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\STM32F10x_StdPeriph_Driver\src**stm32f10x_rcc.c**
- Mein_Projekt\STM32F10x_StdPeriph_Lib_V3.5.0\Libraries\STM32F10x_StdPeriph_Driver\src**stm32f10x_tim.c**

Abschließend werden noch zwei neue Dateien über die Menüleiste **File → New...** erstellt und diese unter den Namen **setup.c** und **main.c** im Ordner

- Mein_Projekt\Projekt\app

gespeichert. Diese werden dem Projekt, wie schon zuvor beschrieben, hinzugefügt. Das Projekt sollte jetzt über die Menüleiste **Project → Rebuild all target files** kompiliert werden, damit die zur Konfiguration benötigten Header-Dateien dem Projekt hinzugefügt werden. Zur Konfiguration des Mikrocontrollers werden die Header-Dateien **stm32f10x.h** und **stm32f10x_conf.h** benötigt. In ihnen wird durch Einkommentieren einiger Präprozessoranweisungen festgelegt, welcher Mikrocontroller und ob die Bibliothek verwendet werden soll. Des Weiteren wird festgelegt, welche Teile der Bibliothek dem Projekt hinzugefügt werden sollen. Die Konfigurationsdatei **stm32f10x_conf.h** ist kein direkter Bestandteil der Bibliothek, kann aber einem Template im Ordner

- ...\STM32F10x_StdPeriph_Lib_V3.5.0\Project\STM32F10x_StdPeriph_Template

entnommen, angepasst und in den Ordner

- Mein_Projekt\Projekt

kopiert werden. Darauf hin kann die Datei an Abbildung 12 angepasst werden, wodurch der Kompilier Aufwand stark verringert wird, da nur aktuell benötigte Header-Dateien eingebunden werden.

```
26 /* Includes ------------------------
27 /* Uncomment/Comment the line belo
28 //#include "stm32f10x_adc.h"
29 //#include "stm32f10x_bkp.h"
30 //#include "stm32f10x_can.h"
31 //#include "stm32f10x_cec.h"
32 //#include "stm32f10x_crc.h"
33 //#include "stm32f10x_dac.h"
34 //#include "stm32f10x_dbgmcu.h"
35 //#include "stm32f10x_dma.h"
36 //#include "stm32f10x_exti.h"
37 #include "stm32f10x_flash.h"
38 //#include "stm32f10x_fsmc.h"
39 #include "stm32f10x_gpio.h"
40 //#include "stm32f10x_i2c.h"
41 //#include "stm32f10x_iwdg.h"
42 //#include "stm32f10x_pwr.h"
43 #include "stm32f10x_rcc.h"
44 //#include "stm32f10x_rtc.h"
45 //#include "stm32f10x_sdio.h"
46 //#include "stm32f10x_spi.h"
47 //#include "stm32f10x_tim.h"
48 //#include "stm32f10x_usart.h"
49 //#include "stm32f10x_wwdg.h"
50 //#include "misc.h" /* High level
```

Abbildung 12: Einbinden benötigter Bibliotheks Header-Dateien (stm32f10x_conf.h)

! Die Schreibrechte der Headerfiles müssen (z. B. mit dem Explorer des Betriebssystems) gesetzt werden um die Dateien verändern zu können.

Da es sich bei dem hier verwendeten Mikrocontroller um ein „Connectivity line device“ handelt wird Zeile 0073 der Datei **stm32f10x.h** einkommentiert. Abbildung 13 verdeutlicht dies.

```
0061 /* Uncomment the line below according to the target STM32 device used in your
0062    application
0063  */
0064
0065 #if !defined (STM32F10X_LD) && !defined (STM32F10X_LD_VL) && !defined (STM32F10
0066   /* #define STM32F10X_LD */     /*!< STM32F10X_LD: STM32 Low density devices *
0067   /* #define STM32F10X_LD_VL */  /*!< STM32F10X_LD_VL: STM32 Low density Value
0068   /* #define STM32F10X_MD */     /*!< STM32F10X_MD: STM32 Medium density device
0069   /* #define STM32F10X_MD_VL */  /*!< STM32F10X_MD_VL: STM32 Medium density Val
0070   /* #define STM32F10X_HD */     /*!< STM32F10X_HD: STM32 High density devices
0071   /* #define STM32F10X_HD_VL */  /*!< STM32F10X_HD_VL: STM32 High density value
0072   /* #define STM32F10X_XL */     /*!< STM32F10X_XL: STM32 XL-density devices */
0073 #define STM32F10X_CL           /*!< STM32F10X_CL: STM32 Connectivity line devi
0074 #endif
```

Abbildung 13: Auswählen des verwendeten Mikrocontrollers (stm32f10x.h)

In Abbildung 14 sieht man, dass durch Einkommentieren der Zeile 105 der Datei **stm32f10x.h** abschließend festgelegt wird, dass in diesem Projekt die Standard Peripherals Library verwendet werden soll. Nach diesem letzten Schritt lässt sich jetzt die Bibliothek verwenden und es kann begonnen werden ein minimales Beispielprogramm zu implementieren.

```
0099 #if !defined  USE_STDPERIPH_DRIVER
0100 /**
0101   * @brief Comment the line below if you will not use the peripherals drivers.
0102     In this case, these drivers will not be included and the application code will
0103     be based on direct access to peripherals registers
0104     */
0105 #define USE_STDPERIPH_DRIVER
0106 #endif
```

Abbildung 14: stm32f10x.h Konfiguration der Standard Peripherals Library

3.2 Minimale Beispielanwendung

Der Quellcode des hier ausgearbeiteten Templates wird in der Form abgebildet, dass er 1:1 kopiert und in ein reales Projekt eingefügt werden kann.

3.2.1 Initialisierung und Konfiguration des Mikrocontrollers (Template)

Im diesem Kapitel werden die Taktfrequenzen und die GPIOs des Mikrocontrollers konfiguriert und initialisiert.

3.2.1.1 Konfiguration der Taktfrequenzen

Der erste Schritt bei der Implementierung jeder Anwendung ist die Konfiguration der verschiedenen Taktfrequenzen des Mikrocontrollers und der benötigten Peripherie. Zur Konfiguration der Taktfrequenzen wird die Reset and Clock Control (RCC) des Prozessors verwendet. Hierbei zu beachten ist, dass alle Teile der Peripherie mit einem Takt versorgt werden müssen bevor man diese verwenden kann. Aus diesem Grund werden im ersten Block der Prozessorinitialisierung wie in Abbildung 15 zu sehen, die Grundtakte des Mikrocontrollers konfiguriert. So lange man nicht genau weiß, was dort passiert, sollte dieser Block nicht verändert werden, da auch die Reihenfolge der einzelnen Initialisierungsschritte wichtig ist.[30]

[30] [STM_REF_MAN] Seite 87

Die dort eingestellten Taktfrequenzen sind:

- Systemtakt (SYSCLK) 72MHz
- Hardwaretakt (HCLK) 72MHz
- Peripherietakt 1 (PCLK1) 36MHz
- Peripherietakt 2 (PCLK2) 72MHz

Welche maximalen Frequenzen möglich sind und welche Funktionen die im Quellcode zu sehenden Phasenregelschleifen (PLL[31]) haben, kann im Clock Tree des Reference Manuals[32] und/oder einfacher im Blockdiagramm[33] des Mikrocontrollers nachgeschlagen werden. In der hier beschriebenen Konfiguration wird der auf dem Entwicklungsboard befindliche externe Quarz (HSE[34]) zur Taktgenerierung verwendet.

[31] Engl. Phase-locked loop
[32] Weiterführende Literatur [STM_REF_MAN] Seite 123
[33] Weiterführende Literatur [STM_BLOCKDIA]
[34] Abk. High Speed External Clock Signal

```
/**
  ******************************************************************************
  * @file    Mein_Projekt\Projekt\app\setup.c
  * @author  Hauke Pfaffelmoser
  * @date    11 October 2011
  * @brief   This file provides all the Mikrocontroller initialization
  ******************************************************************************
  */

/* Includes ------------------------------------------------------------------*/
#include "stm32f10x.h"
#include "setup.h"

/* Private functions ---------------------------------------------------------*/

/**
  * @brief  Configures the different system clocks..
  * @param  None
  * @retval None
  */
void Setup_Clock(void)
{
   /* Since the MCBSTM32C eval board provides a 25MHz HSE clock which is much more
    * precise than the controller-internal HSI clock the HSE clock is used instead
    * of the HSI clock. The system clock will be set to max value (72MHz). The
    * STM32F107VC is a connectivity line device, so it has two PLLs (PLL and PLL2).
    * Both of them must be configured to make the controller work! */

   ErrorStatus HSEStartUpStatus;

   /* Set RCC system to default values */
   RCC_DeInit();

   /* Enable HSE */
   RCC_HSEConfig(RCC_HSE_ON);

   /* Wait till HSE is ready */
   HSEStartUpStatus = RCC_WaitForHSEStartUp();

   /* If the HSE clock could be started successfully, try to initialize the
      controllers clock tree. */
   if (HSEStartUpStatus == SUCCESS)
   {
      /* Enable Prefetch Buffer */
      FLASH_PrefetchBufferCmd(FLASH_PrefetchBuffer_Enable);
      /* Flash 2 wait state */
      FLASH_SetLatency(FLASH_Latency_2);

      /* Configure Clocks ********************************************************/
      /* HCLK = SYSCLK = 72MHz */
      RCC_HCLKConfig(RCC_SYSCLK_Div1);
      /* PCLK2 = HCLK = 72MHz */
      RCC_PCLK2Config(RCC_HCLK_Div1);
      /* PCLK1 = HCLK/2 = 36MHz */
      RCC_PCLK1Config(RCC_HCLK_Div2);

      /* Configure PLLs **********************************************************/
      /* PLL2 configuration: PLL2CLK = (HSE / 5) * 8 = 40 MHz */
      RCC_PREDIV2Config(RCC_PREDIV2_Div5);
      RCC_PLL2Config(RCC_PLL2Mul_8);

      /* Enable PLL2 */
      RCC_PLL2Cmd(ENABLE);

      /* Wait till PLL2 is ready */
      while(RCC_GetFlagStatus(RCC_FLAG_PLL2RDY) == RESET);

      /* PLL configuration: PLLCLK = (PLL2 / 5) * 9 = 72 MHz */
      RCC_PREDIV1Config(RCC_PREDIV1_Source_PLL2, RCC_PREDIV1_Div5);
      RCC_PLLConfig(RCC_PLLSource_PREDIV1, RCC_PLLMul_9);

      /* Enable PLL */
      RCC_PLLCmd(ENABLE);

      /* Wait till PLL is ready */
      while(RCC_GetFlagStatus(RCC_FLAG_PLLRDY) == RESET);

      /* Select PLL as system clock source */
```

```
      RCC_SYSCLKConfig(RCC_SYSCLKSource_PLLCLK);

      /* Wait till PLL is used as system clock source */
      while(RCC_GetSYSCLKSource() != 0x08);
```

Abbildung 15: Konfiguration und Initialisierung der Grundtaktfrequenzen (setup.c)

Nachdem die Grundtaktfrequenzen eingestellt sind müssen die Takte für die benötigte Peripherie wie in Abbildung 16 zu sehen eingeschaltet werden. In diesem Fall ist es lediglich der Takt des GPIOE welcher mit den anzusteuernden LEDs verbunden ist.[35]

```
      /* Enable Peripheral Clocks: */
      RCC_APB2PeriphClockCmd( RCC_APB2Periph_GPIOE, ENABLE);
   }
```

Abbildung 16: Einschalten des Taktes für GPIOE (setup.c)

Das Ende der Taktkonfiguration bildet die Fehlerbehandlung in der nun folgenden Abbildung 17.

```
   else
   { /* If HSE fails to start-up, the application will have wrong clock
        configuration. User can add here some code to deal with this error */

       /* Go to infinite loop */
       while (1)
      {
         /* do nothing */
      }
   }
}
```

Abbildung 17: Fehlerbehandlung bei misslungener Taktinitialisierung (setup.c)

3.2.1.2 Konfiguration der GPIO

Nachdem die Initialisierung der Taktfrequenzen abgeschlossen ist können die GPIO Pins konfiguriert werden. Dazu wird mit Hilfe der Bibliothek eine Struktur erzeugt in der die Einstellungen zu den entsprechenden GPIO Pins gespeichert werden. Die möglichen zur Verfügung stehenden Einstellungen lassen sich über einen **Rechtsklick → Go To Definition Of** auf eine der symbolischen Konstanten einsehen und deren Bedeutung im Reference Manual des Mikrocontrollers nachschlagen. Wie in Abbildung 18 zu erkennen, werden die GPIOE Pins 8 - 15 für den Modus[36] Output push-pull mit einer maximalen Schaltfrequenz von 50MHz konfiguriert.

[35] Vgl. [KEIL_EVAL_SCHALTPL] Seite 2
[36] [STM_REF_MAN] Seite 154

```
/**
  * @brief  Configures the different GPIO.
  * @param  None
  * @retval None
  */
void Setup_GPIO (void)
{
   GPIO_InitTypeDef  GPIO_InitStructure;

   /* GPIOE Configuration: Pin 8 - 14 as output push-pull for LED panel ports */
   GPIO_InitStructure.GPIO_Pin =  GPIO_Pin_8  | GPIO_Pin_9  | GPIO_Pin_10 |
                                  GPIO_Pin_11 | GPIO_Pin_12 | GPIO_Pin_13 |
                                  GPIO_Pin_14 | GPIO_Pin_15 ;

   GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
   GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
   GPIO_Init(GPIOE, &GPIO_InitStructure);
}
```

Abbildung 18: Konfiguration des GPIOE (setup.c)

Abschließend werden die Initialisierungsprototypen noch in einer Header-Datei (Aufbau einer Header-Datei siehe Kapitel 4.2.8) deklariert. Siehe dazu Abbildung 19.

```
/**
  ******************************************************************************
  * @file    Mein_Projekt\Projekt\app\setup.h
  * @author  Hauke Pfaffelmoser
  * @date    11 October 2011
  * @brief   Setups all necessary system components.
  ******************************************************************************
  */
#ifndef SETUP_H
#define SETUP_H

/* Private function prototypes -----------------------------------------------*/
void Setup_Clock(void);
void Setup_GPIO (void);

#endif // SETUP_H
```

Abbildung 19: Deklaration der Initialisierungsprototypen (setup.h)

3.2.2 Implementierung der Main-Funktion

Nachdem die Initialisierungsroutinen fertiggestellt sind ist man jetzt in der Lage über die Bibliotheksfunktionen einzelne GPIO Pins zu setzen. Abbildung 20 zeigt, dass dazu die Initialisierungsroutinen aufgerufen und im Anschluss daran die Bits mit der entsprechenden Bibliotheksfunktion gesetzt werden. Alle zur Verfügung stehenden Bibliotheksfunktionen einschließlich der symbolischen Konstanten können in den entsprechenden Header-Dateien (hier: **stm32f10x_gpio.h**) oder in der Hilfedatei der Bibliothek nachgeschlagen werden.

```
/**
  ******************************************************************************
  * @file    main.c
  * @author  Hauke Pfaffelmoser
  * @brief   Template for Keil MCBSTM32C Evaluation Board
  *          LED 8 and LED 9 will be switched on
  ******************************************************************************
  */

/* Includes ------------------------------------------------------------------*/
#include "stm32f10x.h"
#include "setup.h"

/**
  * @brief  Main program
  * @param  None
  * @retval None
  */
int main(void)
{
/* System Init---------------------------------------------------------------*/
   /* System Clocks Configuration */
   Setup_Clock();
   /* Configure the GPIO ports */
   Setup_GPIO();
   /* Set LED8 */
   GPIO_SetBits( GPIOE, GPIO_Pin_8 );
   /* Set LED9 */
   GPIO_SetBits( GPIOE, GPIO_Pin_9 );

   while(1)
   {
      /* do nothing */
   }
}
```

Abbildung 20: Aufruf der Initialisierungsroutinen und Setzen der GPIOE-Pins 8 und 9 (main.c)

Nun steht ein fertiges Grundgerüst für weitere Anwendungen zur Verfügung und das Projekt sollte sich über die Menüleiste **Project → Rebuild all target files** erstellen und anschließend über **Flash → Download** flashen lassen (zur Konfiguration des Programmers siehe Kapitel 7.2).

! Sollte sich das Projekt jetzt nicht fehlerfrei erstellen lassen, so kann es notwendig sein, den Ordner der veralteten Bibliothek umzubenennen. Die von Keil mitgelieferte Bibliothek ist in folgendem Ordner zu finden, und kann gelöscht oder der Ordner umbenannt werden.

C:\Keil\ARM\INC\ST → C:\Keil\ARM\INC\ST_AL

4 Implementierung der Lüftersteuerung

In diesem Kapitel soll Schritt für Schritt gezeigt werden, wie eine PWM Lüftersteuerung auf Basis des zuvor in Kapitel 3 erstellten Templates auf einem STM32F107V Mikrocontroller implementiert werden kann.
Mit dieser Implementierung soll es möglich sein, bei einem Lüfter die Drehzahl zu regeln. Durch das Einbinden der Standard Peripherals Library ist es jetzt möglich Auszüge aus den dort zur Verfügung stehenden Beispielen zu verwenden. Es hat sich herausgestellt, dass die Bibliothek den Zugang zum Mikrocontroller stark vereinfacht. Durch die von STMicroelectronics zur Verfügung gestellten Beispielprogramme wird ein Großteil der Funktionalität des Prozessors gut lesbar beschrieben. Daher empfiehlt es sich, diese Beispielprogramme bei Folgeprojekten primär zu Rate zu ziehen. Der nachfolgend abgebildete Code wird wieder in der Reihenfolge beschrieben, dass er Stück für Stück in das zuvor erstellte Template eingefügt werden kann. Das Resultat wird ein kompilier- und flashbares Programm ergeben.
Zu Beginn sollte eine Kopie des Template-Ordners **..\Mein_Projekt** angefertigt werden und diese in **..\Lueftersteuerung** umbenannt werden.

4.1 Aufgabenstellung

Die zu implementierende Lüftersteuerung soll folgende Funktionalität aufweisen:

1. Drehzahl über das Potentiometer des Entwicklungsboards regeln
2. Drehzahl des Lüfters mit Hilfe der PWM steuerbar machen
3. Einlesen einzelner Tachosignale (interruptgesteuert)
4. Drehzahl (rpm) berechnen
5. Ausgabe und grafische Darstellung der Werte auf dem Display

4.1.1 Vorbereitung und Lösungsvorschlag

Zu Beginn sollte festgestellt werden, welche Funktionalität von Seiten des Mikrocontrollers zur Verfügung gestellt wird, um die in der Aufgabenstellung beschriebenen Anforderungen realisieren zu können.

Lösungsvorschlag zu

1. Die Drehzahlregelung soll mit Hilfe des Potentiometers erfolgen. Mit dessen Hilfe kann eine am ADC anliegenden Spannung verändert werden. Diese Veränderung kann man in Abhängigkeit zur Drehzahl setzen und diese somit steuern. Zum Auslesen der Spannung wird ein **ADC** des Mikrocontrollers verwendet.
2. Die Drehzahl lässt sich über einen speziellen PWM Timer des Mikrocontrollers realisieren.
3. Die Tachosignale lassen sich über einen externen Interrupt (EXTI[37]) einlesen.
4. Mit Hilfe einer Timerfunktion können die Tachosignale über einen bestimmten Zeitraum gezählt und dann auf rpm umgerechnet werden.
5. Für die Ausgabe auf dem Display stellt die Bibliothek keine Funktionalität zur Verfügung. Das Display wird über einen Displaycontroller angesteuert, welcher von der Bibliothek nicht direkt unterstützt wird. Der Einfachheit halber wird deshalb hier auf den von Keil zur Verfügung gestellten Display-Treiber zurückgegriffen.

4.2 Initialisierung und Konfiguration des Mikrocontrollers (Lüftersteuerung)

In diesem Kapitel wird die für die Lüftersteuerung notwendige Initialisierung des Mikrocontrollers und der Peripherie angesprochen. Dazu gehört die Initialisierung und Konfiguration des Takts, der GPIO, des PWM Timers, des DMA Controllers, des ADCs der externen Interrupts und des NVICs[38]. Im ersten Schritt wird eine Variable angelegt, welche in der Lage ist, die vom DMA Controller übertragenen ADC Werte

[37] Abk. Externer Interrupt
[38] Abk. Nested Vectored Interrupt Controller

aufzunehmen. Diese haben eine maximale Größe von 12 Bit, zu sehen in Abbildung 21.

```
/* Private variables ---------------------------------------------------------*/
__IO uint16_t ADC1ConvertedValue = 0;
```

Abbildung 21: Speicherstelle der konvertierten ADC Werte (setup.c)

4.2.1 Konfiguration der für die Peripherie benötigten Takte

Wie in Abbildung 22 zu erkennen muss bei weiteren GPIOs und alternativen Funktionen (AFIO[39]) wie den externen Interrupts, dem ADC1 und Timer1 der Takt eingeschaltet werden. Dazu muss der folgende Code dem Template aus Abbildung 16 hinzugefügt werden. Des Weiteren muss noch der Takt des Timer3 eingeschaltet werden. Da dieser über den APB1 angebunden ist, muss dies äquivalent zu APB2 erfolgen. Im Anschluss daran wird der Takt vom ADC (max. 14MHz) konfiguriert und der Takt des DMA Controllers eingestellt.

```
    /* Enable Peripheral Clocks:  */
    RCC_APB2PeriphClockCmd( RCC_APB2Periph_ADC1  |
                            RCC_APB2Periph_TIM1  |
                            RCC_APB2Periph_GPIOA |
                            RCC_APB2Periph_GPIOC |
                            RCC_APB2Periph_GPIOD |
                            RCC_APB2Periph_GPIOE |
                            RCC_APB2Periph_AFIO, ENABLE);

    RCC_APB1PeriphClockCmd( RCC_APB1Periph_TIM5, ENABLE);

    /* ADCCLK = PCLK2/6 */
    RCC_ADCCLKConfig(RCC_PCLK2_Div6);

    /* Enable DMA1 clock */
    RCC_AHBPeriphClockCmd(RCC_AHBPeriph_DMA1, ENABLE);
```

Abbildung 22: Anpassen der Setup_Clock() zum Konfigurieren und Einstellen des Takts (setup.c)

4.2.2 Konfiguration der benötigten GPIOs und alternativer Funktionen (AFIO)

Im folgenden Schritt müssen weitere GPIOs konfiguriert und das Template aus Abbildung 18 durch den in der folgenden Abbildung 23 dargestellten Code erweitert werden. Dieser Programmcode konfiguriert alle benötigten Pins des Mikrocontrollers ihrer Aufgabe entsprechend.

[39] Abk. Alternative Function Input/Output

- GPIOA Pin 8 (PWM Timer 1)
  Alternative function open-drain
- GPIOC Pin 4 (Potentiometer)
  Analog input
- GPIOD Pin 11 (Tachosignal)
  Input pull-up

```
/* GPIOA Configuration: Pin 8 as alternate function open-drain for Tim1_Ch1 PWM */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_8;
GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_OD;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOA, &GPIO_InitStructure);

/* GPIOC Configuration: Pin 4 as analog input for Poti -> ADC */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4;
GPIO_InitStructure.GPIO_Mode =  GPIO_Mode_AIN;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOC, &GPIO_InitStructure);

/* GPIOD Configuration: Pin 11 as input pull-up for Tachometer */
GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11;
GPIO_InitStructure.GPIO_Mode =  GPIO_Mode_IPU;
GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
GPIO_Init(GPIOD, &GPIO_InitStructure) ;
```

Abbildung 23: Anpassen der Setup_GPIO() zum Konfigurieren der GPIO (setup.c)

4.2.3 Konfiguration des PWM Timers

Nachdem die GPIOs konfiguriert wurden, kann der Timer für die PWM Steuerung des Lüfters konfiguriert werden. Dazu wird eine neue Funktion **Setup_PWM_Timer1()** definiert. Der Mikrocontroller stellt zur Erzeugung von PWM Signalen einen speziellen PWM Timer (**TIM_OCMode**) zur Verfügung.[40] Dieser Timer wird so konfiguriert, dass er in einer Grundfrequenz von 25KHz läuft. Im hier beschriebenen Beispiel läuft der Timer mit einem Takt von 72MHz. Zum Inkrementieren des Timer Registers benötigt er einen Takt. Um den Timer auf die für die PWM Steuerung erforderliche Frequenz (Periodendauer) von 25KHz zu takten muss das Auto Reload Register (**TIM_Period**) des Timers so angepasst werden, dass es den Timer Counter 25000/Sek zum Überlaufen bringt. Innerhalb jeder Timer Periode hat man nun die Möglichkeit das Tastverhältnis (**TIM_Pulse**) zu steuern. Dazu legt man mit Hilfe des Capture and Compare Registers fest, wie lange innerhalb der Timer Periode ein High Pegel an den

[40] [STM_REF_MAN] Seite 304

PWM Ausgang gelegt wird. Der hier errechnete Wert für das Auto Reload Register ist

$(\frac{72000000}{25000})-1=2879$ für eine Timer Frequenz von 25KHz.

Durch Beschreiben des Capture and Compare Registers ist man jetzt in der Lage das Tastverhältnis analog zu steuern. Dabei liegt grundsätzlich ein Pegel am PWM Ausgang an welcher auf logisch low gezogen wird (**TIM_OCPolarity**), sobald der Wert des Timer Counters den Wert des Capture and Compare Registers übersteigt. Siehe dazu Abbildung 24.

```
/**
  * @brief  Initialization of Timer 1 for PWM Output
  * @param  None
  * @retval None
  */
void Setup_PWM_Timer1(void)
{
   /* TIM1 Configuration ----------------------------------------------------
       Generate a CCR1 controlled PWM signal with variable duty cycles:
       TIM1CLK = SystemCoreClock, Prescaler = 0, TIM1 counter clock = SystemCoreClock
       SystemCoreClock is set to 72MHz
       The objective is to generate a PWM signal at 25KHz:
       The Timer periode is computed as following:
       TIM1_Period = (SystemCoreClock / 25000) - 1
   ----------------------------------------------------------------------------*/

   /*Specifies the period value to be loaded into the active
     Auto-Reload Register (ARR)at the next update event. */
   uint16_t TimerPeriod = 0;

   /* Specifies the pulse value to be loaded into the Capture Compare Register */
   uint16_t Channel1Pulse = 0;

   TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;
   TIM_OCInitTypeDef  TIM_OCInitStructure;

   /* Compute the value to be set in ARR regiter to generate signal frequency at
      25KHz */
   TimerPeriod = (SystemCoreClock / 25000 ) - 1;

   /* Time Base configuration */
   TIM_TimeBaseStructure.TIM_Prescaler = 0;
   TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
   TIM_TimeBaseStructure.TIM_Period = TimerPeriod;
   TIM_TimeBaseStructure.TIM_ClockDivision = 0;
   TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;

   TIM_TimeBaseInit(TIM1, &TIM_TimeBaseStructure);

   /* PWM Mode configuration: Channel1 */
   TIM_OCInitStructure.TIM_OCMode = TIM_OCMode_PWM2;
   TIM_OCInitStructure.TIM_OutputState = TIM_OutputState_Enable;
   TIM_OCInitStructure.TIM_Pulse = Channel1Pulse;
   TIM_OCInitStructure.TIM_OCPolarity = TIM_OCPolarity_Low;

   TIM_OC1Init(TIM1, &TIM_OCInitStructure);

   /* TIM1 counter enable */
   TIM_Cmd(TIM1, ENABLE);

   /* TIM1 Main Output Enable */
   TIM_CtrlPWMOutputs(TIM1, ENABLE);
}
```

Abbildung 24: Konfiguration des PWM Timers (setup.c)

4.2.4 Konfiguration des DMA Controllers und des ADC

Wie in der Aufgabenstellung beschrieben, soll das Tastverhältnis der PWM mit Hilfe des auf dem Entwicklungsboard befindlichen Potentiometers analog von 0% - 100% gesteuert werden können. Die konvertierten Spannungswerte werden im Anschluss daran über den DMA Controller in der Variablen **ADC1ConvertedValue** gespeichert. Dazu wird der DMA Controller, wie in Abbildung 25 zu sehen ist, konfiguriert.

Über die Basisadresse der Peripherie (**DMA_PeripheralBaseAdr**) erhält der DMA Controller die Position der Speicherstelle[41] [42] des konvertierten ADC Wertes. Die Basisadresse des Speichers (**DMA_MemoryBaseAdr**) enthält eine Referenz auf eine Speicherstelle (**ADC1ConvertedValue)**, in die der DMA Controller den zu übertragenden Wert schreiben soll. Weiterhin muss festgelegt werden, ob der Controller Daten von der Peripherie empfangen oder an sie senden soll (**DMA_DIR_PeripheralSRC**). Mit der Puffergröße (**DMA_BufferSize**) wird festgelegt, wie groß der hier verwendete Puffer sein soll. Im Falle dieses Beispiels ist es nur eine Speicherstelle. Aus diesem Grund werden keine Speicheradressen, weder bei der Peripherie (**DMA_PeripheralInc**), noch im RAM (**DMA_MemoryInc**) inkrementiert. Da der zu übertragende ADC Wert eine 12Bit Variable ist, wird als Datengröße (**DMA_PeripheralDataSize** und **DMA_MemoryDataSize**) eine 16 Bit Variable (**Half_Word**) gewählt und als Übertragungsmodus ein Ringpuffer (**DMA_Mode**). Die Datenübertragung erfolgt mit höchster Priorität (**DMA_Priority**). Da keine Daten von RAM zu RAM übertragen werden, kann die Memory to Memory Funktion (**DMA_M2M**) des DMA Controllers deaktiviert werden.

Nachdem der DMA Controller so konfiguriert ist, dass er die konvertierten ADC-Werte übertragen kann, muss im Anschluss der ADC konfiguriert werden. Sind zwei ADCs vorhandenen, ist es möglich diese im Dual Mode zu betreiben. Dabei übernimmt ein ADC die Rolle des Masters und kann den Start der Konvertierung des anderen ADCs in verschiedenen Modi[43] triggern. Im hier beschriebenen Projekt wird ein ADC unabhängig (**ADC_Mode)** von dem anderen ADC verwendet und es wird nur einer der 18 Channel (**ADC_NbrOfChannel)** für diesen ADC benötigt (**ADC_ScanConvMode[44]**).

[41] [STM_DATASHEET] Seite 31, Memorymap für Speicherbereich
[42] [STM_REF_MAN] Seite 242, Speicheradresse des ADC (offset)
[43] [STM_REF_MAN] Seite 210
[44] ENABLE für Multi-Channel Mode; DISABLE für Single-Channel Mode

Die Konvertierung soll fortlaufend erfolgen (**ADC_ContinuousConvMode**) und es wird kein externes Signal (**ADC_ExternalTrigConv**) benötigt um die Konvertierung zu starten. Im Anschluss daran werden folgende Initialisierungs- und Kalibrierungsfunktionen ausgeführt:

- **ADC_RegularChannelConfig()**
  weist den dazugehörigen Channel[45] [46] und den Rang[47] zu und legt die gewünschte Konvertierungszeit fest.
- **ADC_DMACmd()**
  lässt den ADC nach Beendigung seiner Konvertierung einen DMA Request auslösen, welcher dem DMA Controller mitteilt, dass neue Daten zur Übertragung vorhanden sind. Hierbei ist zu beachten, dass DMA nur bei ADC1 oder im Dual Mode möglich ist.
- **ADC_Cmd()**
  schaltet den ADC ein.

Die folgenden Funktionen dienen zur Kalibrierung des ADCs. Dieser hat einen eingebauten Kalibrierungsmodus[48], welcher immer auf die in Abbildung 25 beschriebene Weise für alle ADCs durchgeführt werden kann.
Abschließend wird die Konvertierung über die Funktion **ADC_SoftwareStartConvCmd()** gestartet. Der ADC wird beim ersten Einschalten mit der Funktion **ADC_Cmd()** nur aus dem Ruhezustand geweckt und beginnt erst beim zweiten Einschalten mit der Konvertierung.

[45] Weiterführende Literatur [KEIL_EVAL_SCHALTPL] Seite 1, Anschluss des Potentiometers
[46] Weiterführende Literatur [STM_DATASHEET] Seite 27, ADC und Channel PC4 → ADC1_Channel14
[47] Legt die Reihenfolge der Konvertierung fest, sollte ein ADC mehrere Channel konvertieren.
[48] Weiterführende Literatur [STM_REF_MAN] Seite 214

```
/**
  * @brief  Initialization of the ADC1. Used to compute the Poti output to set the
  *         PWM duty cycle. Converted values are transferd with the DMA controller
  *          - DMA1 Channel1
  * @param  None
  * @retval None
  */
void Setup_ADC1_Poti(void)
{
   /* DMA1 Channel1 configuration ----------------------------------------------*/
   ADC_InitTypeDef ADC_InitStructure;
   DMA_InitTypeDef DMA_InitStructure;

   DMA_DeInit(DMA1_Channel1);
   DMA_InitStructure.DMA_PeripheralBaseAddr = ADC1_DR_Address;
   DMA_InitStructure.DMA_MemoryBaseAddr = (uint32_t)&ADC1ConvertedValue;
   DMA_InitStructure.DMA_DIR = DMA_DIR_PeripheralSRC;
   DMA_InitStructure.DMA_BufferSize = 1;
   DMA_InitStructure.DMA_PeripheralInc = DMA_PeripheralInc_Disable;
   DMA_InitStructure.DMA_MemoryInc = DMA_MemoryInc_Disable;
   DMA_InitStructure.DMA_PeripheralDataSize = DMA_PeripheralDataSize_HalfWord;
   DMA_InitStructure.DMA_MemoryDataSize = DMA_MemoryDataSize_HalfWord;
   DMA_InitStructure.DMA_Mode = DMA_Mode_Circular;
   DMA_InitStructure.DMA_Priority = DMA_Priority_High;
   DMA_InitStructure.DMA_M2M = DMA_M2M_Disable;
   DMA_Init(DMA1_Channel1, &DMA_InitStructure);

   /* Enable DMA1 channel1 */
   DMA_Cmd(DMA1_Channel1, ENABLE);

   /* ADC1 configuration ------------------------------------------------------*/
   ADC_InitStructure.ADC_Mode = ADC_Mode_Independent;
   ADC_InitStructure.ADC_ScanConvMode = DISABLE;
   ADC_InitStructure.ADC_ContinuousConvMode = ENABLE;
   ADC_InitStructure.ADC_ExternalTrigConv = ADC_ExternalTrigConv_None;
   ADC_InitStructure.ADC_DataAlign = ADC_DataAlign_Right;
   ADC_InitStructure.ADC_NbrOfChannel = 1;
   ADC_Init(ADC1, &ADC_InitStructure);

   /* ADC1 regular channels configuration */
   ADC_RegularChannelConfig(ADC1, ADC_Channel_14, 1, ADC_SampleTime_239Cycles5);

   /* Enable ADC1 DMA */
   ADC_DMACmd(ADC1, ENABLE);

   /* Enable ADC1 */
   ADC_Cmd(ADC1, ENABLE);

   /* Enable ADC1 reset calibration register */
   ADC_ResetCalibration(ADC1);

   /* Check the end of ADC1 reset calibration register */
   while(ADC_GetResetCalibrationStatus(ADC1));

   /* Start ADC1 calibration */
   ADC_StartCalibration(ADC1);

   /* Check the end of ADC1 calibration */
   while(ADC_GetCalibrationStatus(ADC1));

   /* Start ADC1 Software Conversion */
   ADC_SoftwareStartConvCmd(ADC1, ENABLE);
}
```

Abbildung 25: ADC1 Konfiguration mit DMA Datenübertragung (setup.c)

4.2.5 Konfiguration des externen Interrupts

Um die Drehzahl des Lüfters berechnen zu können, wird mit Hilfe des externen Interrupt Controllers bei jedem Tachosignal ein Interrupt erzeugt. Der Mikrocontroller verfügt hierzu über 20 Flankendetektoren. 16 dieser 20 Flankendetektoren sind über ex-

terne Interruptlines (EXTI) mit den GPIOs verbunden um externe Interrupts zu generieren. Der hier konfigurierte externe Interrupt hat folgende Einstellungen, die in Abbildung 26 implementiert werden:

- Eingangssignal an Pin D11 (**EXTI_Line**)
- GPIO ist über EXTI Line 11[49] verbunden (**EXTI_Mode**)
- Interrupt bei fallender Tachosignal Flanke (**EXTI_Trigger**)
- Einschalten des externen Interrupts (**EXTI_LineCmd**)

```
/**
  * @brief  Initialization of external interrupt on GPIOD Pin 11 .
  *         Used to receive the tachometer output signal.
  * @param  None
  * @retval None
  */
void Setup_EXTI11_GPIOD_Pin11(void)
{
   EXTI_InitTypeDef   EXTI_InitStructure;

   /* Connect EXTI11 Line to GPIOD Pin 11 */
   GPIO_EXTILineConfig(GPIO_PortSourceGPIOD, GPIO_PinSource11);

   /* Configure EXTI11 Line */
   EXTI_InitStructure.EXTI_Line = EXTI_Line11;
   EXTI_InitStructure.EXTI_Mode = EXTI_Mode_Interrupt;
   EXTI_InitStructure.EXTI_Trigger = EXTI_Trigger_Falling;
   EXTI_InitStructure.EXTI_LineCmd = ENABLE;
   EXTI_Init(&EXTI_InitStructure);
}
```

Abbildung 26: Konfiguration des externen Interrupts (setup.c)

4.2.6 Konfiguration des Timer 5

Um die eingehenden Tachosignale des Lüfters über einen bestimmten Zeitraum zählen zu können muss ein Timer konfiguriert werden. Da der Lüfter bei geringster Drehzahl nur ca. 500 rpm (ca. 16 Signale/Sekunde) und bei maximaler Drehzahl nur ca. 2000 rpm (66 Signale/Sekunde) dreht, kann der Timer auf der geringst möglichen Taktfrequenz laufen.

Es sollen über einen Zeitraum von 0.5 Sekunden die Tachosignale gezählt werden. Hierzu wird der Prescaler des Timers auf den Maximalwert von 0XFFFF gesetzt (**TIM_Prescaler**), was den Timer bei einem Grundtakt von 72MHz auf 1098Hz taktet. Um zwei Update Interrupts pro Sekunde zu generieren muss die Periode des Timers auf 0X225 (**TIM_Period**) angepasst werden. Diese wird in das Auto-Reload Register eingetragen und lässt den Timer einen Update Interrupt auslösen (**TIM_ITConfig()**), sobald der Wert des Counter Registers beim Hochzählen (**TIM_CounterMode**) den

[49] Weiterführende Literatur [STM_REF_MAN] Seite 200, 201

Wert des Auto-Reload Registers erreicht. Der Timer wird so eingestellt, dass er endlos läuft (**TIM_RepetitionCounter**). Siehe dazu Abbildung 27.

```
/**
  * @brief  Initialization of Timer 5. Used to calculate tachometer output
  *         signals in RPM.
  * @param  None
  * @retval None
  */
void Setup_Tim5(void)
{
   /* TIM5 Configuration ---------------------------------------------------
      The objective is to count the tachometer output signals for 0.5 sec.:
      The Timer prescaler is computed as following:
      - TIM5_Prescaler = (SystemCoreClock / 65535)  - Timer set to 1098Hz
       The Timer period is set to 549   - Two update interrupts per second
   ----------------------------------------------------------------------- */
   TIM_TimeBaseInitTypeDef  TIM_TimeBaseStructure;

   TIM_TimeBaseStructure.TIM_Prescaler = 0XFFFF;
   TIM_TimeBaseStructure.TIM_CounterMode = TIM_CounterMode_Up;
   TIM_TimeBaseStructure.TIM_Period = 0X225;
   TIM_TimeBaseStructure.TIM_RepetitionCounter = 0;

   TIM_TimeBaseInit(TIM5, &TIM_TimeBaseStructure);

   /* TIM enable counter */
   TIM_Cmd(TIM5, ENABLE);

   /* Enable the TIM5 Update Interrupt Request */
   TIM_ITConfig(TIM5, TIM_IT_Update, ENABLE);
}
```

Abbildung 27: Konfiguration des Timer 5 (setup.c)

4.2.7 Konfiguration des Interruptcontrollers

Alle Interrupts, einschließlich der Prozessorkern-Interrupts, werden von dem NVIC verwaltet. In ihm wird über Ein- und Ausschalten der Interrupt Channels festgelegt, welche Peripherie Interrupts erzeugen darf. Die Vektortabelle[50] enthält 67 Einträge, wobei jede Interrupt-Quelle einen festen Vektor mit einem Eintrag der entsprechenden Interrupt Service Routine hat. Die Interrupts können in Gruppen mit unterschiedlichen Prioritäten verschachtelt (engl. nested) werden. Innerhalb dieser Gruppen ist es möglich den Interrupts weitere Prioritäten zuzuweisen. Sollten die einzelnen Interrupts innerhalb ihrer Gruppe die selbe Priorität haben, so wird ihre Priorität durch ihre Position in der Vektor Tabelle festgelegt. Aus jener lässt sich auch die Channel-Nummer (Position) entnehmen. Für Priorität und Subpriorität stehen jeweils 4 Bits zur Verfügung und je höher der jeweilige Wert, desto niedriger seine Priorität. Eine Ausnahme bei der Priorisierung bilden drei Core Exeptions (Reset, Clock Security System, Hard Fault). Diese haben in der angegebenen Reihenfolge die höchsten Prioritäten, welche

[50] Weiterführende Literatur [STM_REF_MAN] Seite 190 - 193

auch nicht verändert werden können. In dem hier gezeigten Beispiel in Abbildung 28, sollen die Channels[51] (**NVIC_IRQChannel**) des externe Interrupt an Pin D11, und der Interrupt des Timer 5 aktiviert werden. Beide Interrupts erhalten die selbe und niedrigst mögliche Priorität (**NVIC_IRQChannelPreemptionPriority**) und innerhalb dieser Gruppe auch dort die niedrigste (**NVIC_IRQChannelSubPriority**).

```
/**
  * @brief  Enable and set EXTI15_10 Interrupt to the lowest priority
  *         Used to enable external interrupts on GPIOD Pin11
  * @param  None
  * @retval None
  */
void Setup_NVIC_EXTI15_10(void)
{
   NVIC_InitTypeDef NVIC_InitStructure;

   NVIC_InitStructure.NVIC_IRQChannel = EXTI15_10_IRQn;
   NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0xF;
   NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0xF;
   NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
   NVIC_Init(&NVIC_InitStructure);
}

/**
  * @brief  Enable and set Timer5 Interrupt to the lowest priority
  *         Used to enable interrupts on Timer5
  * @param  None
  * @retval None
  */
void Setup_NVIC_TIM5(void)
{
   NVIC_InitTypeDef NVIC_InitStructure;

   NVIC_InitStructure.NVIC_IRQChannel = TIM5_IRQn;
   NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0XF;
   NVIC_InitStructure.NVIC_IRQChannelSubPriority = 0XF;
   NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;

   NVIC_Init(&NVIC_InitStructure);
}
```

Abbildung 28: Konfiguration der Interrupts (setup.c)

4.2.8 Header-Datei der setup.c

Um einen schnellen Überblick über die implementierte Funktionalität zu erhalten werden folgende Deklarationen und Definitionen immer in einer Header-Datei gespeichert:

- Globale Makros
- Deklarationen von globalen Konstanten
- Globale Typdefinitionen

[51] Siehe Lueftersteuerung\Libraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x\stm32f10x.h um die literale Konstante des entsprechenden Channels zu bestimmen.

- Deklaration von globalen Variablen
- Deklaration von globalen Funktionen

Vergleiche dazu Abbildung 29.

```
/**
  ******************************************************************************
  * @file    Lueftersteuerung\Projekt\app\setup.h
  * @author  Hauke Pfaffelmoser
  * @date    11 October 2011
  * @brief   Setups all necessary system components.
  ******************************************************************************
  */

#ifndef SETUP_H
#define SETUP_H

/* Private defines-----------------------------------------------------------*/
#define ADC1_DR_Address    ((uint32_t)0x4001244C)

/* Private global variables ------------------------------------------------*/
extern __IO uint16_t ADC1ConvertedValue;

/* Private function prototypes ---------------------------------------------*/
void Setup_Clock(void);
void Setup_GPIO(void);
void Setup_PWM_Timer1(void);
void Setup_ADC1_Poti(void);
void Setup_EXTI11_GPIOD_Pin11(void);
void Setup_Tim5(void);
void Setup_NVIC_EXTI15_10(void);
void Setup_NVIC_TIM5(void);

#endif // SETUP_H
```

Abbildung 29: Header-Datei der Prozessorinitialisierung (setup.h)

4.3 Implementierung der Interrupt Service Routinen (ISR)

Nachdem der Mikrocontroller initialisiert und die benötigte Peripherie konfiguriert wurde, können, wie in Abbildung 30 dargestellt, die ISRs implementiert werden. Hierzu werden zwei Dateien[52] aus einer Beispielanwendung[53] in den Projektordner[54] kopiert und dem Projekt in der Gruppe Application hinzugefügt. Diese Dateien enthalten ein Grundgerüst für die Exception Behandlung der Prozessor Exceptions[55] [56]. Dieser Teil des Quellcodes wurde herausgenommen so dass der hier abgebildete Code in die Datei integriert werden muss.

[52] **stm32f10x_it.c** und **stm32f10x_it.h**
[53] Lueftersteuerung\Examples\GPIO\IOToggle
[54] Lueftersteuerung\Projekt\app
[55] Weiterführende Literatur [STM_REF_MAN] Seite 190
[56] Weiterführende Literatur [ARM_TEC_REF_MAN] Seite 50

! Damit die ISR angesprungen werden kann, wenn ein Interruptrequest erfolgt, ist es von entscheidender Wichtigkeit, der Interruptserviceroutine den richtigen Namen zu geben. Dieser Name hat die Form PeripherieName_IRQHandler. Die Peripherie Namen lassen sich dem Startup-File[57] entnehmen.

Sobald der Interruptcontroller einen Interrupt Request erhält, setzt er für die dazugehörige Peripherie ein Pending-Bit. Abhängig von seiner Prioritätsstufe wird der Interrupt zur Bearbeitung der ISR an das CPU Interface weitergeleitet. Solange man das Pending Bit nicht löscht (**PeriphName_ClearITPendingBit()**), geht der Interruptcontroller davon aus, dass der Interrupt noch nicht bearbeitet wurde. Er wird ihn somit wieder an die CPU zur Bearbeitung der ISR weiterleiten. Sobald die CPU ein Interrupt erreicht, werden mit Hilfe von Mikrocode einige Register auf dem Stack gespeichert. Parallel dazu wird die Zieladresse aus der Vektortabelle gelesen und anschließend der Interrupt ausgeführt. Abschließend werden die Register vom Stack wiederhergestellt.

[57] Lueftersteuerung\Libraries\CMSIS\CM3\DeviceSupport\ST\STM32F10x\startup\arm\startup_stm32f10x_cl.s

```
/**
  ******************************************************************************
  * @file    \Luefterssteuerung\Projekt\app\stm32f10x_it.c
  * @author  MCD Application Team
  * @version V3.5.0
  * @date    08-April-2011
  * @brief   Main Interrupt Service Routines.
  *          This file provides template for all exceptions handler and
  *          peripherals interrupt service routine.
  ******************************************************************************
  */
/* Includes ------------------------------------------------------------------*/
#include "stm32f10x_it.h"

/* Private variables ---------------------------------------------------------*/
uint16_t tacho_irq_count =0;
uint32_t total_revolutions = 0;
uint16_t revolutions_per_minute = 0;

/******************************************************************************/
/*                 STM32F10x Peripherals Interrupt Handlers                   */
/*  Add here the Interrupt Handler for the used peripheral(s) (PPP), for the  */
/*  available peripheral interrupt handler's name please refer to the startup */
/*  file (startup_stm32f10x_xx.s).                                            */
/******************************************************************************/

/**
  * @brief  This function handles the EXTI15_10 interrupt request GPIOD Pin11.
  * @param  None
  * @retval None
  */
void EXTI15_10_IRQHandler(void)
{
   if(EXTI_GetITStatus(EXTI_Line11) != RESET)
   {
      /* Clear EXTI Line11 interrupt */
      EXTI_ClearITPendingBit(EXTI_Line11);

      /* Count revolution per irq update */
      tacho_irq_count += 1;
      total_revolutions += 1;
   }
}

/**
  * @brief  This function handles Timer5 interrupt request on timer update.
  * @param  None
  * @retval None
  */
void TIM5_IRQHandler(void)
{
   /* Clear TIM5 update interrupt */
   TIM_ClearITPendingBit(TIM5, TIM_IT_Update);

   /* Calculate revolutions per minute.
      2 tacho signals per revolution and
      2 TIM5 interrupts per second */
   revolutions_per_minute = tacho_irq_count * 60;
   tacho_irq_count = 0;
}
```

Abbildung 30: Interrupt Service Routinen (stm32f10x_it.c)

4.3.1 Header-Datei der stm32f10x_it.h

Abbildung 31 zeigt den Aufbau der Header-Datei welcher in Kapitel 4.2.8 beschrieben wurde.

```
/**
  ******************************************************************************
  * @file    C:\Luefttersteuerung\Projekt\app\stm32f10x_it.h
  * @author  MCD Application Team
  * @version V3.5.0
  * @date    08-April-2011
  * @brief   This file contains the headers of the interrupt handlers.
  ******************************************************************************
  */

/* Define to prevent recursive inclusion -------------------------------------*/
#ifndef __STM32F10x_IT_H
#define __STM32F10x_IT_H

/* Includes ------------------------------------------------------------------*/
#include "stm32f10x.h"

/* Private global variables
------------------------------------------------------------*/
extern uint32_t total_revolutions;
extern uint16_t revolutions_per_minute;

/* Private function prototypes -----------------------------------------------*/
void EXTI15_10_IRQHandler(void);
void TIM5_IRQHandler(void);

#endif /* __STM32F10x_IT_H */
```

Abbildung 31: Header-Datei der Interrupt Service Routinen (stm32f10x_it.h)

4.4 Implementierung der Main-Funktion

Abschießend muss noch die Main-Funktion implementiert werden. Zuvor sollten jedoch die für das Projekt benötigten Header-Dateien der Standard Peripheral Library über das Library configuration file (siehe Abbildung 12) eingebunden werden:

- stm32f10x_adc.h
- stm32f10x_dma.h
- stm32f10x_exti.h
- stm32f10x_tim.h
- misc.h

Weiterhin müssen die dazugehörigen *.c Dateien[58] dem Projekt hinzugefügt werden. Das gleiche gilt für die zur Ausgabe der Lüfterwerte benötigte Bibliothek des auf dem Evaluationsboard verbauten Displays. Der Displaycontroller des TFT-LCD ist über die SPI[59] Schnittstelle mit dem Mikrocontroller verbunden und wird hier über die Bibliothek von Keil initialisiert. Um die LCD-TFT Bibliothek einzubinden wird ein neuer Ordner **Luefttersteuerung\Projekt\lcd** und eine neue Projektgruppe **LCD** angelegt. In diesen Ordner sollen dem Projekt folgende Dateien aus dem von Keil zur Verfügung

[58] Luefttersteuerung\Libraries\STM32F10x_StdPeriph_Driver\src
[59] [STM_REF_MAN] Seite 672, Serial Peripheral Interface

gestellten Beispielprojekt LCD_Blinky[60] kopiert werden, bevor die **GLCD_SPI_STM32.c** Datei der Gruppe LCD hinzugefügt wird:

- GLCD_SPI_STM32.c
- GLCD.h
- Font_16x24_h.h
- Font_6x8_h.h

Nachdem die LCD Bibliothek dem Projekt hinzugefügt wurde, kann die eigentliche Main Funktion wie in Abbildung 32 grafisch dargestellt, implementiert werden. Die verschiedenen Initialisierungsprozesse werden in dem Flussdiagramm zu einem Prozess zusammengefasst um die Übersichtlichkeit zu erhalten. In der Main-Funktion werden die zuvor implementierten Initialisierungsfunktionen, einschließlich der Display Initialisierung, aufgerufen. Nach einmaligem Schreiben der Überschrift auf das Display geht die Main Funktion in die für die Mikrocontroller Programmierung typische Endlosschleife über. In dieser wird überprüft, ob sich der ADC-Wert geändert hat. Sollte dies der Fall sein, wird das PWM Tastverhältnis angepasst und eine neue Ausgabe in das Display geschrieben.

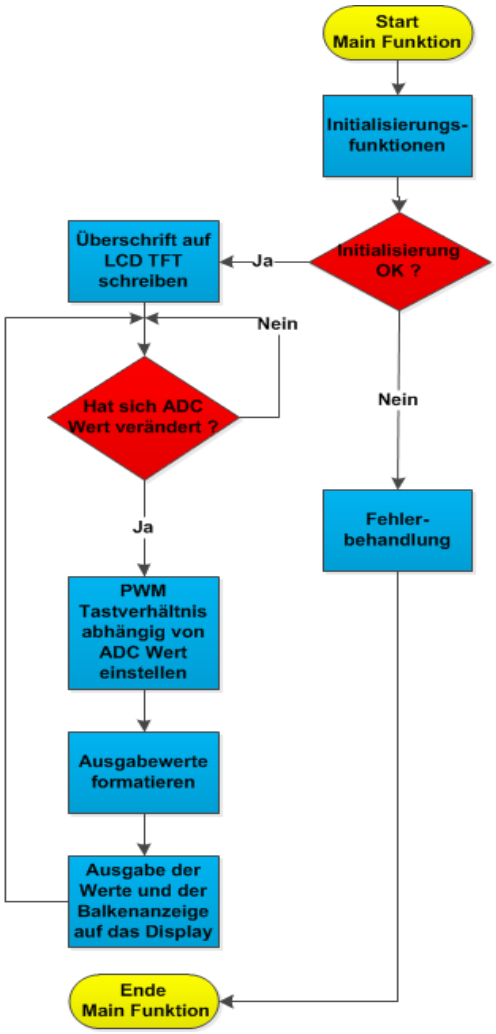


Abbildung 32: Flussdiagramm main.c

Abbildung 33 zeigt die dazugehörige Implementierung.

[60] Keil\ARM\Boards\Keil\MCBSTM32C\LCD_Blinky

```
/**
  ******************************************************************************
  * @file    Lueftersteuerung\Projekt\app\main.c
  * @author  Hauke Pfaffelmoser
  * @brief   Fan controll via potentiometer for Keil MCBSTM32C Evaluation Board.
  *          Using the STMicroelectronics STM32F10x Standard Peripherals Library.
  *          Values:
  *                 - rpm revolutions per minute
  *                 - potentiometer value 0 - 4095
  *                 - total revolutions
  *          will be displayed on LCD.
  ******************************************************************************
  */

/* Includes ------------------------------------------------------------------*/
#include <stdio.h>
#include "stm32f10x.h"
#include "setup.h"
#include "lcd/GLCD.h"

/* Private variables ---------------------------------------------------------*/
extern uint32_t total_revolutions;
extern uint16_t revolutions_per_minute;
extern __IO uint16_t ADC1ConvertedValue;

/**
  * @brief  Main program
  * @param  None
  * @retval None
  */
int main(void)
{
/* Private lokal variables ---------------------------------------------------*/
   char display_ADC1Converted_value[20];
   char display_revolutions_per_minute[20];
   char display_total_revolutions[20];
   char poti_value    = 0xFFFF;

/* System Init---------------------------------------------------------------*/
   /* System Clocks Configuration */
   Setup_Clock();
   /* Configure GPIO ports */
   Setup_GPIO();
   /* Configure PWM Timer 1 */
   Setup_PWM_Timer1();
   /* Configure ADC */
   Setup_ADC1_Poti();
   /* Configure external interrupt EXTI Line 11 */
   Setup_EXTI11_GPIOD_Pin11();
   /* Configure Timer 5 */
   Setup_Tim5();
   /* Configure Exti Line11 Interrupt */
   Setup_NVIC_EXTI15_10();
   /* Configure TIM5 Interrupt */
   Setup_NVIC_TIM5();

   /* Configure LCD Display */
   GLCD_Init();
   GLCD_Clear(White);
   GLCD_SetTextColor(White);
   GLCD_SetBackColor(Blue);

   /* Write heading on Display */
   GLCD_DisplayString(0, 0, 1, "   Drehzahlregler   ");
   GLCD_SetBackColor(White);
   GLCD_SetTextColor(Blue);

   while(1)
   {
      if (ADC1ConvertedValue ^ poti_value)
      {


         /*-------------------------------------------------------------------
          Write 12 Bit ADC conversion value in Timer 1 capture and compare
          register (CCR) to set the PWM duty cycle.
          CCR max value = 2880 ; ADC max value = 4095;  4095/2880=1,4218
         -------------------------------------------------------------------*/
```

```
        TIM1->CCR1 = (int)(ADC1ConvertedValue / 1.422);

        sprintf(display_revolutions_per_minute, "RPM       :%5i",
                revolutions_per_minute);
        sprintf(display_ADC1Converted_value, "ADC Value :%5i", ADC1ConvertedValue);
        sprintf(display_total_revolutions,"Umdr. Ges.:%5i", (total_revolutions/2));

        /* Show ADC value bargraph             */
        GLCD_Bargraph (115, (Line8), 160, 20, (ADC1ConvertedValue>>2));
        poti_value = ADC1ConvertedValue;

        /* Write values on LCD*/
        GLCD_DisplayString(3, 0, 1, (unsigned char *)
                                    display_revolutions_per_minute);
        GLCD_DisplayString(4, 0, 1, (unsigned char *) display_ADC1Converted_value);
        GLCD_DisplayString(5, 0, 1, (unsigned char *) display_total_revolutions);
        GLCD_DisplayString(8, 0, 1, "Geschw:");
      }
      /* Reset interrupt LEDs */
      GPIO_ResetBits( GPIOE, GPIO_Pin_8 );
      GPIO_ResetBits( GPIOE, GPIO_Pin_15 );
    }
}
```

Abbildung 33: Main-Funktion (main.c)

5 Keils Echtzeitbetriebssystem RTX

In den letzten Jahren hat die Komplexität und Leistungsfähigkeit der Mikrocontroller einen Grad erreicht, der den Einsatz eines Echtzeitbetriebssystems möglich macht. Traditionell mussten Entwickler von Applikationen im Bereich der Programmierung eingebetteter Systemen die Software komplett selbst entwerfen und schreiben. Diese Software beinhaltete einige Interruptbehandlungen und eine Endlosschleife, in welcher das eigentliche Programm ausgeführt wurde. Bei dieser Art der Programmierung ist es jedoch nur schwer möglich die heutigen Fortschritte in der Programmstruktur und im Design umzusetzen und deren Werkzeuge zu verwenden.
Mit der Einführung der ARM Cortex™-M3 32-Bit Mikrocontroller stehen kostengünstige, leistungsfähige Mikrocontroller zur Verfügung, welche große Mengen an internem SRAM und Flashspeicher adressieren und somit zur Verfügung stellen können. Dies hat zur Folge, dass die Komplexität und Größe der Software mit den Prozessoren mitwächst. Aus diesem Grund kann nach [MICROCONSULT] die Verwendung eines Echtzeitbetriebssystems folgende Vor- aber auch Nachteile in der Entwicklung von Software auf Mikrocontrollern bieten:

Vorteile:

- „Verbesserung der Portabilität“
- „Klarere Strukturierung im Quellcode“
- „Verbesserte Wartbarkeit“
- „Verbesserte Wiederverwendbarkeit“
- „Erhöhte Zuverlässigkeit“
- „Verbesserte Produktivität in der Entwicklung durch Modularisierung“
- „Kosteneinsparungen durch verkürzte Entwicklungszeiten“

Nachteile:

- „Erhöhter Ressourcenverbrauch und schlechtere Performance“
- „Längere Einarbeitungszeit und erhöhter Trainingsaufwand“
- „Abhängigkeit vom Betriebssystemhersteller“

5.1 RTX Kernel

Dieses Kapitel beschäftigt sich ausschließlich mit der Programmierung des Echtzeitbetriebssystem Kernels RTX sowie den dazu gehörigen Beispielanwendungen und Templates, rot gerahmt in Abbildung 34. Der RTX Kernel ist als minimales Echtzeitbetriebssystem zu klassifizieren. Diese Klasse von Betriebssystemen ist für die Verwendung auf Mikrocontrollern mit eingeschränkten Speicherressourcen gedacht. Dabei ist das Betriebssystem nur eine Bibliothek, welche dem Anwendungsprogramm hinzugebunden wird. Die Funktion beschränkt sich auf eine Taskverwaltung und einfache Ein- und Ausgabemechanismen. Des Weiteren wird keine Speicherverwaltung zur Verfügung gestellt, sondern nur physikalische Adressierung innerhalb eines Adressraumes unterstützt.

Abgesehen vom RTX Kernel bietet Keil noch eine Middleware namens RL-ARM an. Die RL-ARM ist eine Bibliothek, bestehend aus fünf Hauptkomponenten, die sich nahtlos in den RTX Kernel einbinden lässt und weitere Funktionen zur Verfügung stellt, blau gerahmt in Abbildung 34.[61]

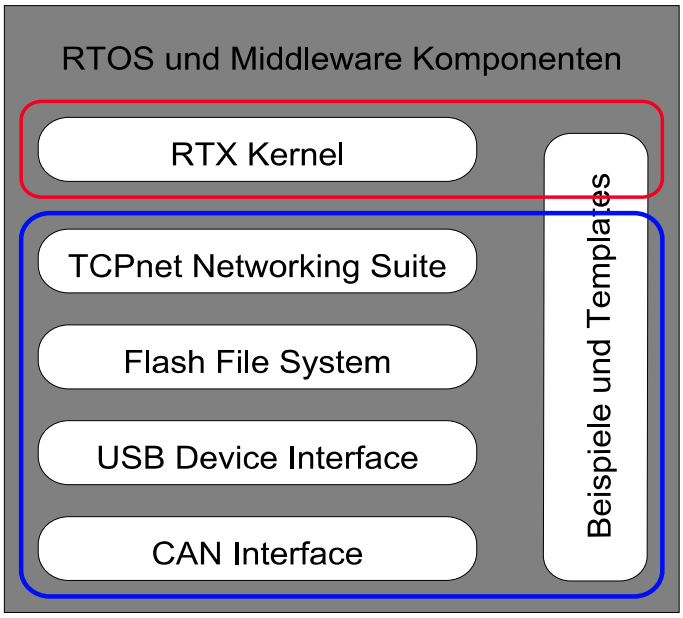


Abbildung 34: Keil RTX Middleware Komponenten

Quelle: [KEIL_RTX_MAN]

[61] Vgl. [KEIL_RTX_MAN], [WÖRN]

Abbildung 35 veranschaulicht, dass das RTOS[62] im Wesentlichen die selben Aufgaben wie ein Standard Betriebssystem erfüllen muss:[63]

- **Taskverwaltung (Scheduler)**
  Die Taskverwaltung ist für die Zuteilung des Prozessors an die Tasks und somit für die Organisation und Steuerung der auszuführenden Programme verantwortlich.
- **Speicherverwaltung (Memory Pool)**
  Die Speicherverwaltung teilt dem einzelnen Task den benötigten Speicher zu.
- **Interprozesskommunikation (Mailbox)**
  Die Interprozesskommunikation ist für die Kommunikation einzelner Tasks untereinander verantwortlich.
- **Synchronisation (Event, Mutex, Semaphore, Delay & Interval)**
  Unter Synchronisation versteht man die zeitliche Koordination der Tasks.

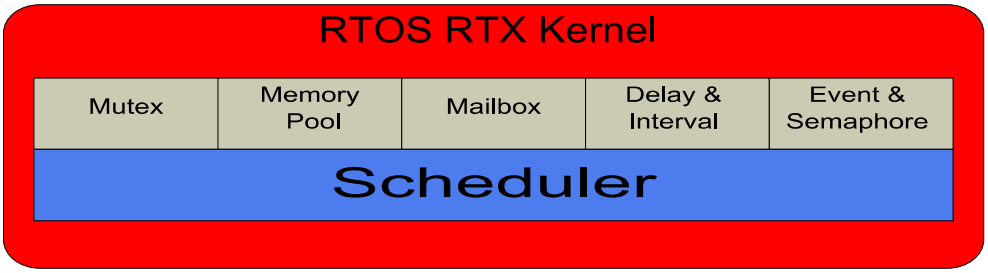


Abbildung 35: Aufbau des Keil RTX Kernel

Quelle: [KEIL_RTX_MAN]

Da es in Echtzeitbetriebssystemen in der Regel darauf ankommt ein möglichst schlankes Betriebssystem zu haben, sind Schutzmaßnahmen, wenn überhaupt, nur rudimentär vorhanden. Diese belasten den begrenzten Speicher und schränken somit die Verarbeitungsleistung ein. Der wesentliche Unterschied zwischen einem Standard Betriebssystem und einem Echtzeitbetriebssystem sind nach [WÖRN] zwei zusätzliche Aufgaben des Echtzeitbetriebssystems:

- „Wahrung der Rechtzeitigkeit und Gleichzeitigkeit“
- „Wahrung der Verfügbarkeit“.

[62] Abk. Real-Time Operating System
[63] Vgl. [KEIL_RTX_MAN], [WÖRN]

Darunter ist zu verstehen, dass ein Echtzeitbetriebssystem in der Lage sein muss, verschiedene Aufgaben zu einem vorgegebenen Zeitpunkt auszuführen und Ergebnisse innerhalb einer vorgegebenen Zeitspanne liefern zu können, sobald dies erforderlich wird. Aus diesem Grund sind Reorganisationspausen, wie sie bei Standard Betriebssystemen z. B. zur Speicherbereinigung in der Speicherverwaltung vorkommen, bei Echtzeitbetriebssystemen nicht zulässig. Diese Vorgänge würden die Performance und Anforderungen an die Verfügbarkeit verletzen.[64]

5.2 Task und Taskverwaltung

Die Taskverwaltung ist eine wesentliche Aufgabe des RTX Kernels und unterstützt Round Robin, Pre-emptive Scheduling, Round Robin Pre-emptive Scheduling und Cooperative Multitasking. Auf diese Verfahren wird in den Kapiteln 5.4.2, 5.4.3, 5.4.4 und 5.4.5 näher eingegangen.

Ein Task ist ein vom RTX verwalteter Rechenprozess. Er besitzt eine Aktionsfunktion und eine Zustandsvariable. Die Aktionsfunktion beinhaltet den Quellcode, welcher für die eigentliche Funktionalität dieses Tasks verantwortlich ist (z. B. Auslesen eines Temperatursensors). Die Zustandsvariable ist ein Tripel, welches den aktuellen Zustand des Tasks speichert. Sie besteht aus Programmzähler, Register und Variablen des Tasks. Diese Zustandsvariable ist in Form eines Stacks aufgebaut und beinhaltet alle Informationen über den aktuellen Zustand des Tasks. Sie wird benötigt um während der Ausführung einen Kontextwechsel vornehmen zu können. Das RTOS kann somit während der Laufzeit eines Tasks diesen unterbrechen, seinen aktuellen Zustand in der Zustandsvariable speichern und einen weiteren Task starten bzw. weiterlaufen lassen. Dies ermöglicht bei Einkern-Prozessoren eine quasi-parallele Bearbeitung und bei Mehrkernprozessoren eine echte parallele Bearbeitung von Tasks. Konzeptionell ist ein Task also ein Funktionsblock des Betriebssystems. Er ermöglicht ein objektorientierteres Design als zuvor. Für den Programmierer ergibt sich der Vorteil, einzelne Programmblöcke isolieren zu können um z. B. Fehler zu finden und zu beseitigen. Typischerweise werden in C-Programmen Funktionen zur Bearbeitung bestimmter Aufgaben aufgerufen, welche nach Bearbeitung zur aufrufenden Funktion zurückkehren. Wie in Tabelle 1 zu sehen, verhält es sich bei Tasks ähnlich, jedoch mit einigen wesentli-

[64] Vgl. [KEIL_RTX_MAN], [WÖRN]

chen Unterschieden. In der rechten Spalte ist der Aufbau eines Tasks in Form einer C Funktion zu sehen. Der wesentliche Unterschied im Vergleich zu einer normalen Funktion ist, dass im Falle eines Tasks der Funktion das Schlüsselwort **__task** voran gestellt wird. Dies verhindert, dass der Compiler den Ein- und Austrittscode hinzufügt. Hierbei handelt es sich in der Regel um die Stackbearbeitung. Diese Aufgabe wird im Falle eines Tasks vom Scheduler des RTOS übernommen und aus diesem Grund nicht mehr benötigt. Ein weiterer Unterschied ist, dass sich die eigentliche Endlosschleife, wie man es von der prozeduralen Programmierung eines Mikrocontrollers gewohnt ist, innerhalb des Tasks befindet.[65]

Funktion	Task

`unsigned int Funktion (void)`
`{`
 	
`  return (val);`
`}`
	`__task void task (void)`
	`{`
	`  for(;;)`
	`  {`
	
	`  }`
	`}`

Tabelle 1: Vergleich von Funktion und Task

Wie in Abbildung 36 zu sehen ist, können Tasks während der Laufzeit verschiedene Zustände einnehmen:

- Waiting
  Task wartet darauf, dass die für seine Ausführung benötigten Bedingungen erfüllt sind. Dies können zum Beispiel Zeitbedingungen sein.
- Ready
  Alle erforderlichen Bedingungen für die Ausführung des Tasks sind erfüllt, Betriebsmittel wurden zugeteilt und es wird auf die Zuteilung des Schedulers der Taskverwaltung gewartet.
- Running
  Der Task befindet sich in der Ausführung. Dies können bei Mehrkernprozessoren auch mehrere Tasks sein.
- Block

[65] Vgl. [KEIL_RTX_MAN], [WÖRN]

Ein Task gilt als blockiert, wenn er auf eintreffende Ereignisse wartet. Dies kann beispielsweise ein Eingabewert oder das Freiwerden eines Betriebsmittels sein.[66]

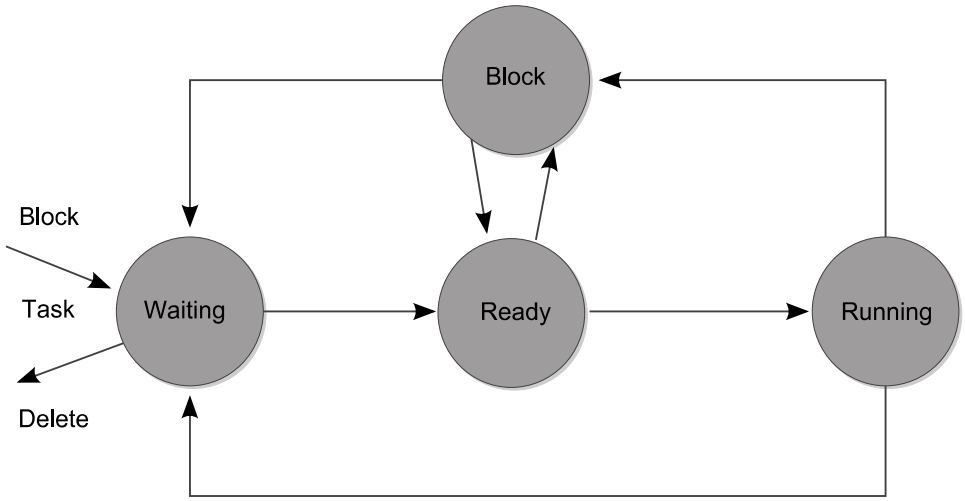


Abbildung 36: Mögliche Taskzustände während der Laufzeit

Quelle: [WÖRN]

In Tabelle 2 sind die beim Keil RTX RTOS möglichen Zustände eines Tasks aufgelistet und erläutert.

Task Status	Beschreibung
RUNNING	Task wird ausgeführt
READY	Task ist bereit zur Ausführung
WAIT DELAY	Task wartet eine (vom Entwickler) vorgegebene Zeit
WAIT INT	Task wird in bestimmten Intervallen ausgeführt
WAIT OR	Task wartet auf ein Event Flag
WAIT AND	Task wartet auf eine Gruppe von Event Flags
WAIT SEM	Task wartet auf ein freies SEMAPHOR
WAIT MUT	Task wartet auf ein SEMAPHOR MUTEX
WAIT MBX	Task wartet auf eine MAILBOX MESSAGE
INACTIVE	Task wurde nicht erkannt oder wurde noch nicht gestartet

Tabelle 2: Auflistung der möglichen Task-Zustände des Keil RTX RTOS

Quelle: [KEIL_RTX_MAN]

[66] Vgl. [WÖRN]

5.2.1 Erzeugen von Tasks

Jeder Task erhält, wie in Abbildung 37 dargestellt, bei seiner Erstellung eine einmalige Task-ID zugewiesen. Diese Variable fungiert als Handle und wird zur Verwaltung des Tasks während seiner Laufzeit verwendet.

```
OS_TID id1, id2, id3;
```

Abbildung 37: Deklaration einer Task ID

Um einen Task zu erstellen, definiert man wie in Abbildung 38 jeden Task als eine Standard C Funktion mit einer dazugehörigen Task-ID. Eine Ausnahme bildet hierbei der zuerst erstellte Task. Es gilt als guter Programmierstil, diesen hauptsächlich zur Erstellung weiterer Tasks zu verwenden und ihn sich danach selbst beenden zu lassen. Dieser erste Task erzeugt über den Systemaufruf **os_tsk_create(Taskname, Priorität)** zwei weitere Tasks und weist ihnen ihre Task-ID und ihre Priorität zu. Es können aber auch zu späteren Zeitpunkten weitere Tasks erzeugt werden.
Nach einem Reset des Mikrocontrollers würde dieser wie gewohnt das Programm über die Main-Funktion starten und den Initialisierungscode ausführen, bevor das RTOS gestartet wird. Erst nach dem Systemaufruf **os_sys_init(Taskname)** wird das RTOS initialisiert und mit dem ersten Task gestartet.[67]

```
/*Task prototypes ---------------------------------------*/
void Task1_Init(void);
void Task2(void);
void Task3(void);
/*Task IDs Task1_Init benötigt keine Task ID--------------*/

OS_TID TaskID_2;
OS_TID TsskID_3;

int main(void)
{
   /* Initialisiere RTX and starte Init */
   os_sys_init (Task0_Init);
}

__task void Task0_Init (void)
{
   /* Erstelle zweiten Task */
   TaskID_1 = os_tsk_create(Task1, 0X01);
   /* Erstelle dritten Task */
   TaskID_2 = os_tsk_create(Task2, 0X02);
   /* Task löscht sich selbst */
   os_tsk_delete_self ();
}

__task void Task1_Init (void){;}
__task void Task2_Init (void){;}
```

Abbildung 38: Erzeugen von 3 Tasks

[67] Vgl. [KEIL_RTX_MAN]

Weiterführend ist es auch möglich, einem Task bei seiner Erzeugung einen Parameter zu übergeben. Siehe dazu Abbildung 39. Dies ist sinnvoll, da man mit Hilfe dieses Parameters z. B. mehrere Instanzen eines Tasks erzeugen kann. Man hat beispielsweise so die Möglichkeit mit der selben Basis zwei Instanzen eines ADC Tasks laufen zu lassen und über den Parameter zwei verschiedene ADCs auf dem Mikrocontroller zu betreiben.

```
/* Erzeugen eines Tasks mit Parameterübergabe */
taskID4 = os_tsk_create_ex (Task4, prio, param);
```

Abbildung 39: Erzeugen eines Tasks mit Parameterübergabe

Beim Erzeugen eines Tasks bekommt dieser seinen eigenen Stack zugewiesen. Auf diesem speichert er alle für einen Kontextwechsel wichtigen Informationen. Der Stack hat eine fixe Menge an Speicher (RAM) und wird bei der Erstellung eines Projekts in den Grundeinstellungen (siehe Kapitel 6.3) des RTOS definiert. Sollte ein Task mehr Speicher für seinen Stack benötigen, so kann man die Größe einzelner Tasks, wie in Abbildung 40 zu sehen, anpassen.

```
/* Anlegen eines größeren Stacks.  Hier z. B. 400 Byte */
static uint32_t stack_name [400/8];
/* Erstellen des Tasks mit einem größeren Stack */
taskID5 = os_tsk_create_user(Task5, prio,&stack_name,sizeof(stack_name));
```

Abbildung 40: Anpassen der Stack Größe eines Tasks beim Erzeugen

Weiterhin ist es auch möglich beim Erzeugen des einzelnen Tasks diesem einen größeren Stack zur Verfügung zu stellen und einen Parameter zu übergeben. Dies geschieht wie in Abbildung 41.

```
/* Anlegen eines größeren Stacks. Hier z. B. 400 Byte */
static uint32_t stack_name [400/8];
/* Erstellen des Tasks mit einem größeren Stack und Parameter */
tskID6 = os_tsk_create_user (Task5, prio, &stack_name, sizeof(stackname), param);
```

Abbildung 41: Anpassen der Stack Größe und Übergabe eines Parameters bei Taskserzeugung

5.2.2 Priorisierung von Tasks

Um Fehler bei der Programmierung von Tasks zu vermeiden sollte man eine klare Vorstellung haben, wie und warum man seine Tasks priorisiert. In Abbildung 42 lässt sich erkennen, dass innerhalb des RTOS die höchst priorisierten Tasks jene Tasks sind, welche zum Bearbeiten der Interrupts verwendet werden. Danach folgen hoch priorisierte Progamm-Tasks, die niedrigst priorisierten Round Robin Tasks und der IDLE Demon[68]. Hierbei zu beachten ist, dass jeder Task mit einer höheren Priorität als die Round Robin Tasks sich zwingend selbst blockieren muss, da er sonst niedriger priorisierte Tasks dauerhaft blockiert.

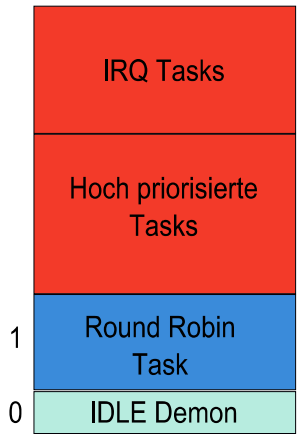


Abbildung 42: Priorisierung von Tasks

Quelle: [KEIL_RTX_MAN]

5.2.3 Taskverwaltung

Sobald ein Task erstellt ist, wird dieser abhängig vom verwendeten Scheduling Verfahren vom RTOS verwaltet. Es ist nur möglich ihn über die vom RTOS zur Verfügung gestellten Systemaufrufe, wie in Abbildung 43 gezeigt, zu manipulieren.

```
/* Ändern der Priorität des Tasks aus einer anderen Funktion.*/
   oder eines anderen Tasks */
OS_RESULT os_tsk_prio (tsskID7, priorität);
/* Task ändert seine Priorität selbst. Dies geschieht innerhalb des Tasks */
OS_RESULT os_tsk_prio_self (priorität);
/* Löschen eines Tasks über eine andere Funktion oder einen anderen Task */
OS_RESULT = os_tssk_delete (tskID1);
/* Task löscht sich selbst */
os_tsk_delete_self ();
/* Task unterbricht sich selbst und übergibt die Kontrolle an den nächsten Task */
os_tsk_pass ();
```

Abbildung 43: Systemaufrufe zur Taskverwaltung

[68] Spezieller Task der vom RTOS ausgeführt wird wenn kein anderer Task läuft. Zu finden in RTX_Config.c.

5.2.4 Time Management

RTOS stellt zwei Systemaufrufe zur Verfügung, welche es ermöglichen, Tasks für eine definierte Zeitspanne zu unterbrechen oder sie in gewissen Zeitabständen auszuführen. Die einzustellenden Timings sind abhängig von der SysTick Timer Configuration (siehe Kapitel 6.3) des RTOS. Die in Abbildung 44 gezeigten Parameter **delay_time** und **interval_time** legen fest, wie oft der SysTick Timer einen Interrupt erzeugen soll, bevor die Bearbeitung fortgesetzt bzw. das Warten beendet wird. Sobald diese Zeitspanne abgelaufen ist, wird der Task wieder in den Zustand READY versetzt und in die Warteschlange des Schedulers eingereiht.

```
/* Delay-Funktion zur einfachen Zeitverzögerung innerhalb des Tasks */
void os_dly_wait (unsigned short delay_time);
/* Periodische Taskausführung */
__task void task2 (void)
{
   os_itv_set (50);
   while(1)
   {
       /* Warte 50 Timer Ticks */
       os_itv_wait ();
   }
}
```

Abbildung 44: Systemaufrufe für das Time Management

5.2.5 Virtuelle Timer

RTOS ermöglicht es dem Programmierer über den Systemaufruf **os_tmr_create()** bis zu 255 virtuelle Timer zu erzeugen. Diese Timer sind abhängig vom SysTick-Timer des RTOS und verhalten sich wie Countdown-Timer. Sobald einer dieser Timer abgelaufen ist, ruft er eine Call-Back Funktion auf. In dieser Funktion steht der für diesen Timer auszuführende Code. Der Systemaufruf benötigt zwei Parameter. Der erste Parameter legt die Laufzeit über die Anzahl der SysTick-Timer Interrupts fest, der zweite Parameter ermöglicht der Call-Back Funktion diesen Timer zu identifizieren. Jeder dieser Timer hat seine eigene OS_ID, so dass man über Systemaufrufe auf den Timer zugreifen kann. Abbildung 45 zeigt dazu eine einfache Implementierung. Dabei ist zu beachten, dass der Programmcode in der Call-Back Funktion implementiert und die RTX Konfiguration angepasst werden muss (siehe Kapitel 6.3).[69]

[69] Vgl. [KEIL_RTX_MAN]

```
/* Erstellen eines virtuellen Timers */
OS_ID tmr1;
/* Zuweisen der OS_ID und Einrichten des Timers auf 100 System Ticks */
tmr1 = os_tmr_create(100,1);

/* Die folgende Call-Back Funktion zur Bearbeitung des Benutzer-Codes nach Ablauf
eines virtuellen Timers. Der Prototyp dieser Funktion befindet sich in der
RTX_Config.c Datei */

void os_tmr_call (U16 info)
{
   switch (info)
   {
      /* Code für Timer 1 */
      case 0x01:
      /* Benutzer Code */
      break ;
   }
}
```

Abbildung 45: Virtueller Timer mit Call-Back Funktion in RTX_Config.c

5.3 Intertask Kommunikation

Tasks sind in sich abgeschlossene Prozesse, auf welche von außen nur über bestimmte Systemaufrufe zugegriffen werden sollte. RTOS stellt dafür zur Intertask Kommunikation und Synchronisation folgende, nachfolgend erklärte, Funktionen zur Verfügung:

- Events
- Semaphoren
- Mutexe
- Mailboxen

5.3.1 Events

Jeder Task erhält bei seiner Erzeugung einen Block von 16 Event Flags welche im Kontrollblock des Tasks gespeichert werden. Mit diesen Flags ist es möglich einen Task bis zum Eintreffen eines Events zu unterbrechen. Der Task wird so lange unterbrochen, bis das erwartete Event Flag gesetzt wurde. Des Weiteren ist es auch möglich Tasks auf mehrere Event Flags, also eine Gruppe von Tasks, warten zu lassen. Der zweite Parameter des Systemaufrufs **os_evt_wait_..._(...)** definiert ein periodisches Timeout, nachdem der Task auch ohne eingetroffene Events wieder in den Zustand READY übergeht. Der Wert 0XFFFF bildet dabei eine Ausnahme. Er lässt den Task unbegrenzt auf ein oder mehrere Events warten ohne ein Timeout zu erzeugen. Jeder Task kann über die Task-ID Event Flags setzen und löschen. Der Systemaufruf **os_evt_get ()** ermöglicht es einem laufenden Task zu fragen, auf Grund welches Event Flags er

wieder ausgeführt wird. Das ermöglicht es den für das Ereignis verantwortlichen Task zu identifizieren. Die Systemaufrufe sind in Abbildung 46 zu sehen.[70]

```
/* Warten auf ein oder mehrere Events */
os_evt_wait_and (unsigned short wait_flags, unsigned short timeout);
os_evt_wait_or (unsigned short wait_flags, unsigned short timeout);
/* Setzen eines oder mehrerer Event Flags für einen Task */
void os_evt_set (unsigned short event_flags, OS_TID task);
/* Löschen eines oder mehrerer Event Flags für einen Task */
void os_evt_clr (unsigned short clear_flags, OS_TID task);
/* Task fragt bei erneuter Ausführung die eingetretenen Events ab */
which_flag = os_evt_get ();
```

Abbildung 46: Systemaufrufe der Event Funktionen

5.3.2 Interrupt-Behandlung

Der hier verwendete Mikrocontroller besitzt einen Nested Vectored Interrupt Controller. Mit diesem ist es möglich den Interrupts bis auf wenige Ausnahmen Prioritäten zuzuweisen. Das verschafft den auf Cortex™-M basierenden Mikrocontrollern einen wesentlichen Vorteil gegenüber den auf ARM7/9 basierenden Mikrocontrollern. Bei diesen Modellen hat das RTOS keine Möglichkeit Interrupts zu unterbrechen. Umfangreichere ISRs konnten hier den Interrupt des SysTick Timers unterbrechen und so den Systemclock des RTOS Kernels stören. Daher gilt es bis heute als guter Programmierstil, die ISRs so klein wie möglich zu halten und den eigentlichen Programmcode der Interrupt Service Routine innerhalb eines hoch priorisierten Tasks bearbeiten zu lassen. Innerhalb der Interrupt Service Routine soll lediglich ein spezielles Event Flag für den Task gesetzt und das entsprechende Interrupt Flag gelöscht werden. Für das Erzeugen eines Events aus einer ISR wird ein spezieller, in Abbildung 47 beschriebener, Systemaufruf benötigt. Diese Vorgehensweise ist nicht immer förderlich für die Prozessorauslastung und die Geschwindigkeit der Interruptbehandlung, da durch die Weiterleitung der ISR in einen Task ein Kontextwechsel erfolgt. Dieser hat zur Folge, dass die Prozessorbelastung steigt und sich die Bearbeitung der ISR verzögert. Beide Folgen ließen sich durch direkte Bearbeitung innerhalb der ISR vermeiden.[71]

[70] Vgl. [KEIL_RTX_MAN]
[71] Vgl. [Keil_RTX_MAN]

```
/* Typischer Aufbau einer Interrupt Service Routine einschließlich des dazugehörigen
Tasks zur Bearbeitung des Programm Codes der Interrupt Service Routine */
void Task3 (void)
{
   while (1)
   {
      /* Warte auf Event */
      os_evt_wait_or (0x0001, 0xffff);
      /* Eigentliche Interruptbehandlung */

      /* Löschen des ISR Events */
      os_evt_clr (0X0001, ID_tsk3);
   }
}

void IRQ_Handler (void) __irq
{
   /* Lösche Pending Bit */
   ...
   /* Sende Event an Task 3 */
   isr_evt_set (0x0001, ID_tsk3);
}
```

Abbildung 47: ISR mit Task zur Bearbeitung der Interruptbehandlung

5.3.3 Semaphore

Semaphoren sind wichtige und sehr komplexe Bestandteile eines RTOS. Genauer auf diese einzugehen würde den Rahmen dieser Arbeit sprengen. Aus diesem Grund werden nur die wesentlichen Grundlagen besprochen. Bei Interesse können die am Ende dieses Kapitels angegebenen Quellen genutzt werden.

Semaphoren werden verwendet um parallele oder quasi parallele Prozesse zu synchronisieren, wenn diese von einander abhängig sind. Dies ist der Fall, wenn Prozesse gemeinsame Betriebsmittel nutzen. Betriebsmittel können **Daten** (z. B. Variablen, Tabellen) oder **Programme** (z. B. Gerätetreiber) sein. Das Problem der Synchronisation kann man mit Semaphoren lösen. Hierbei wird eine gewisse Menge an Token in einem Container erzeugt. Jeder zu synchronisierende Abschnitt innerhalb des Tasks muss ein solches Token erwerben um den Prozessor verwenden zu dürfen. Dies geschieht in der Regel an einem bestimmten Punkt innerhalb des Tasks, an dem es notwendig wird, dass z. B. ein anderer Task vorher einen bestimmten Punkt innerhalb seines Task sicher erreicht hat. So lang kein weiteres Token zur Verfügung steht, blockiert ein Task an diesem Punkt, bis ein anderer Task einen Token freigibt.[72]

[72] Weiterführende Literatur [DOWNEY], [KEIL_RTX_MAN]

5.3.4 Mutex

Mutual Exclude[73] oder kurz Mutex ist eine Sperrsynchronisation und eine vereinfachte Form der Semaphoren. Im Gegensatz zu den Semaphoren gibt es beim Mutex nur einen Token und die Hauptaufgabe des Mutex ist, in bestimmten Situationen den Zugriff auf z. B. die Peripherie des Mikrocontrollers zu beschränken. Mutexe stellen also sicher, dass nur ein Task auf das gemeinsame Betriebsmittel zugreift. Abbildung 48 zeigt ein Beispiel, in dem die Temperatur über Sensoren ausgelesen wird um diese Werte danach in eine Tabelle zu schreiben. Zum Schutz dieser Tabelle wird ein Mutex definiert, welcher einen gleichzeitigen Zugriff von Tasks auf diese Tabelle verhindert. Vor dem Zugreifen muss jeder Task versuchen ein Token zu akquirieren. Ist dies nicht möglich, so blockiert der Task und wartet bis der Mutex wieder freigegeben wurde.

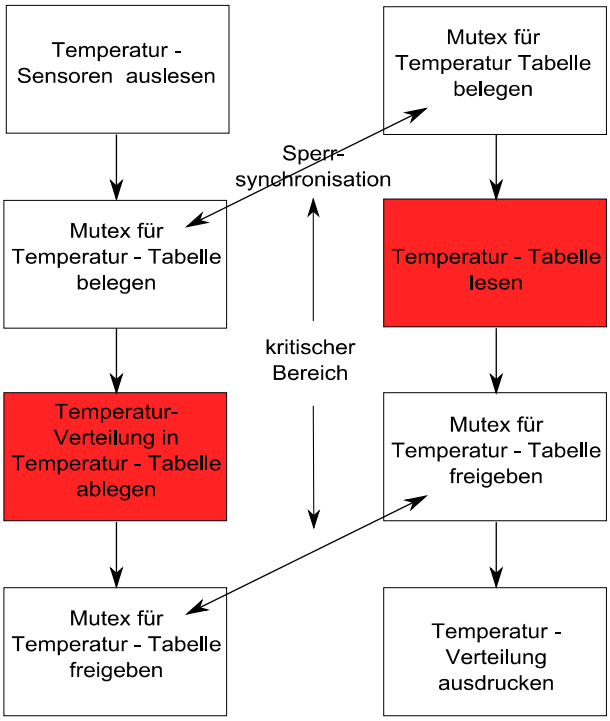


Abbildung 48: Sperrsynchronisation bei Zugriff auf Temperaturtabelle

Quelle: [WÖRN]

Die Verwendung von Mutexen und Semaphoren sind sehr fehleranfällig, da immer sicher gestellt werden muss, dass die Token wieder freigegeben werden. Aus diesem Grund sollten Systemaufrufe wie **os_task_delete()** mit großer Vorsicht verwendet wer-

[73] Wechselseitiger Ausschluss

den, wenn ein Task Mutexe oder Semaphoren kontrolliert. Abbildung 49 zeigt dazu ein Beispiel der Implementierung zur Verwendung von Mutexen.[74]

```
/* Deklaration eines Mutex Containers */
OS_MUT mutex_container;

__task void task1 (void)
{
   while(1)
   {
      os_mut_wait(mutex,0xffff);
      /* Eintritt in Synchronisationsbereich */

      /* Verlassen des Synchronisationsbereichs */
      os_mut_release(mutex);
   }
}
```

Abbildung 49: Mutex Sperrsynchronisation

5.3.5 Mailbox

Der letzte zu besprechende Punkt innerhalb der Intertask Kommunikation ist der eigentliche Datenaustausch zwischen Tasks. Im Vergleich zur Mailbox kann man Events, Semaphoren und Mutexe als informationslose Kommunikation betrachten. Sie werden im Wesentlichen zur Synchronisation verwendet und tauschen keine Daten aus. Es gibt zwei Varianten der Task Kommunikation:[75]

- Gemeinsamer Speicher
  Im Fall eines gemeinsamen Speichers wird die Synchronisation mit Hilfe von Semaphoren gelöst. Über diese wird gesteuert, wer wann auf den Speicher zugreifen darf.
- Nachrichtensystem (Mailbox)
  Beim Nachrichtensystem werden Nachrichten an einen Empfänger verschickt. Hierüber erfolgt auch der Datenaustausch und seine Synchronisation.

Die Methode des Datenaustauschs über einen gemeinsamen Speicher ist im allgemeinen die schnellere Variante im Vergleich zu einem Nachrichtensystem. Sollte jedoch kein gemeinsamer Speicher zur Verfügung stehen, weil z. B. die Systeme räumlich ge-

[74] Vgl. [Keil_RTX_MAN]
[75] Vgl. [WÖRN]

trennt sind, muss die Kommunikation über Nachrichten erfolgen. Ein weiteres Unterscheidungsmerkmal ist die zeitliche Koordination der Kommunikation:

- Synchrone Kommunikation
  Durch Blockieren eines Teilnehmers wird ein Zustand erzeugt, in welchem Sender und Empfänger gleichzeitig eine bestimmte Stelle erreichen, an der sie dann die Daten austauschen.
- Asynchrone Kommunikation
  Der Datenaustausch erfolgt über ein Kommunikationssystem, welches die zu sendenden Daten puffert. Der Empfänger der Daten kann jederzeit überprüfen, ob Daten für ihn eingegangen sind.

Im Echtzeitbetriebssystem von Keil wird ein Asynchrones Mailbox System verwendet, welches Nachrichten in sogenannten Mail Slots ablegt. Die Reihenfolge der eingehenden Nachrichten erfolgt nach dem FIFO[76] Prinzip. Über das Mailboxsystem ist ein Task in der Lage Daten mit fester und variabler Länge zu versenden. Im folgenden Beispiel soll eine Nachricht, bestehend aus 8 Bit, übertragen werden. Um diese Daten zu versenden, muss für die Daten eine entsprechende Mailbox deklariert werden. Jedes Mailbox Objekt besteht aus einem Blockspeicher, welcher in eine Reihe von Mail Slots unterteilt ist und einem dazugehörigen Feld, bestehend aus Zeigern gerichtet auf jeden Mail Slot.
Für die Konfiguration einer Mailbox müssen zuerst die Nachrichten-Zeiger deklariert werden. Im hiesigen Beispiel werden 16 Mail Slots verwendet. Die Zahl der Slots kann den Anforderungen entsprechend gewählt werden. Der in Abbildung 50 beschriebene Systemaufruf deklariert ein 16 Felder großes unsigned Integer Array und ein Array, welches die eigentlich zu versendenden Daten enthalten soll. Sobald das Format des Message Slots definiert wurde, muss ein Speicher reserviert werden, welcher in der Lage ist, die entsprechende Anzahl Message Slots aufzunehmen. Dieser Speicherblock wird dann innerhalb der Main-Funktion mit Hilfe eines Systemaufrufs in die erforderlichen 16 Mail Slots formatiert.

[76] Abk. First In First Out

```
/* Deklaration der Nachrichten-Zeiger */
os_mbx_declare(message,16);

/* Deklaration Mailbox */
unsigned int mpool[16*sizeof(unsigned int)/4+3];
int main (void)
{
   /* Formatieren der Mailbox in 16 Mail Slots*/
   _init_box(mpool,sizeof(mpool),sizeof(unsigned int));

   os_sys_init (task1);

}
```

Abbildung 50: Implementierung eines Mailbox-Systems 1. Teil. Initialisierung

Abbildung 51 zeigt den Aufbau des sendenden Tasks, in welchem ein Zeiger deklariert wird, welcher auf die Speicheradresse der Nachricht zeigt. Bei Initialisierung der Mailbox wird diesem dann ein Mail Slot zugewiesen. Anschließend wird der Mail Slot mit den zu übertragenden Daten gefüllt und die Nachricht über einen weiteren Systemaufruf versandt. Weitere Nachrichten können auf die selbe Weise versendet werden.

```
__task void task1 (void)
{
/* Zu versendende Nachricht */
unsigned int MssgData = 0x000000AA ;
/* Zeiger auf den Nachrichtenspeicher */
unsigned int  *mptr;
/* Initialisierung der Message Box */
os_mbx_init(message,sizeof(message));

while(1)
   {
      /* Adresse des Nachrichtenspeichers */
      mptr = _alloc_box(mpool);
      /* Schreiben der Nachricht in den Nachrichtenspeicher */
      *mptr = MssgData;
      /* Versand der Nachricht */
      os_mbx_send(message,mptr,0xffff);
   }
}
```

Abbildung 51: Implementierung eines Mailbox-Systems 2. Teil. Versand

Der Empfang einer Nachricht ist in Abbildung 52 beschrieben. Im Wesentlichen wird ein weiterer Zeiger deklariert, welcher in der Lage ist die Adresse der Mail-Slots aufzunehmen. Über den empfangenen Zeiger kann dann auf den Mail Slot zugegriffen werden. Zusätzlich benötigt der empfangende Task die Systemaufrufe um auf eingehende Nachrichten zu warten und einen Mail Slot wieder freizugeben.[77]

[77] Vgl. [Keil_RTX_MAN]

```
__task void task2 (void)
{
/* Zeiger auf den Nachrichtenspeicher */
unsigned int receive, *rptr;

while(1)
   {
      os_mbx_wait(message,(void **)&rptr,0xffff);
      receive = *rptr;
      /* Speicher freigeben */
      _free_box(mpool,rptr);
   }
}
```

Abbildung 52: Implementierung eines Mailbox-Systems 3. Teil. Empfang

5.4 Echtzeitscheduling Verfahren

Die wichtigste Aufgabe der Taskverwaltung ist die Zuteilung des Mikrocontrollers an ablaufbereite Tasks. Diese Aufgabe übernimmt der Scheduler innerhalb der Taskverwaltung und seine Aufgabe ist, den Prozessor so aufzuteilen, dass alle Echtzeitbedingungen eingehalten werden.

5.4.1 System Timer

Ein für ein RTOS entwickeltes Programm besteht in der Regel aus einer variablen Anzahl von Tasks. Die zeitliche Ausführung dieser Tasks wird vom Scheduler gesteuert. Dieser erhält seinen Takt über den SysTick Timer des Mikrocontrollers, welcher Timer Interrupts erzeugt und dem Betriebssystem damit seinen Grundtakt liefert. Er stellt unter anderem Tasks bei einem entsprechenden Scheduling Verfahren eine bestimmte Zeitscheibe zur Verfügung. Diese Zeitscheibe wird, wie auch das entsprechende Scheduling Verfahren, in der Konfigurationsdatei des RTOS eingestellt (siehe Kapitel 6.3).[78]

5.4.2 Round Robin (Zeitscheibenverfahren)

Wie in Abbildung 53 zu sehen, wird beim Verfahren des Round Robin Scheduling in der Betriebssystemkonfiguration eine Zeitscheibe festgelegt. Diese Zeitscheibe stellt jedem Task pro Durchlauf einen Zeitrahmen zur Verfügung, in dem er den Prozessor nutzen darf. Der einzelne Task bleibt so lange aktiv, bis die Zeitspanne abgelaufen ist

[78] Vgl. [Keil_RTX_MAN]

oder der Prozess durch einen Betriebssystemaufruf blockiert wird. Alle Tasks besitzen dabei die gleiche Priorität.[79]

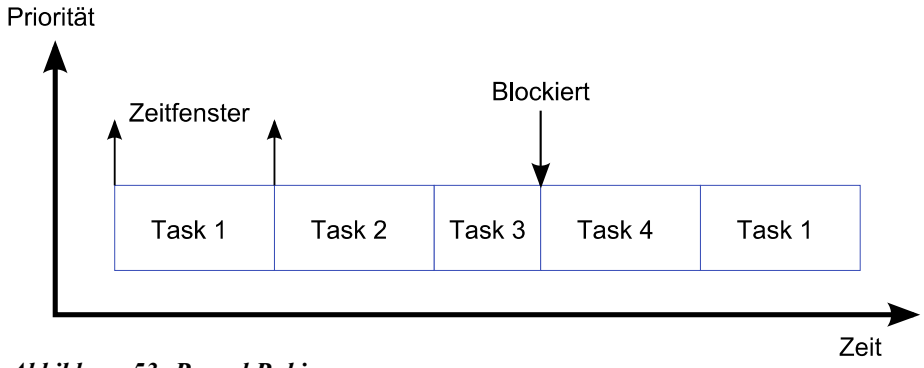


Abbildung 53: Round Robin

Quelle: [KEIL_RTX_MAN]

5.4.3 Pre-emptive Scheduling

Beim Pre-emptive Scheduling wird jedem Task eine Priorität, unterschiedlich zu den Prioritäten der anderen Tasks, zugewiesen. Der Task mit der höchsten Priorität beginnt nach seiner Erzeugung mit der Ausführung seines Programms. Im Falle von Abbildung 54 würde Task 3 nach der Erzeugung den Prozessor übernehmen und ihn erst dann wieder freigeben, wenn er sich selbst durch einen Systemaufruf blockiert oder ein Prozess mit einer höheren Priorität in den Ready Status wechselt. In diesem Fall würde dann dieser Task den Prozessor übernehmen und mit der Ausführung seines Programms beginnen. Im Vergleich zum Round Robin erzeugt man beim Pre-emptive Scheduling eine Struktur, in der Tasks auf Grund ihrer Prioritäten den Prozessor in unterschiedlich großen Zeitscheiben beanspruchen können.[80]

[79] Vgl [WÖRN]
[80] Vgl [WÖRN]

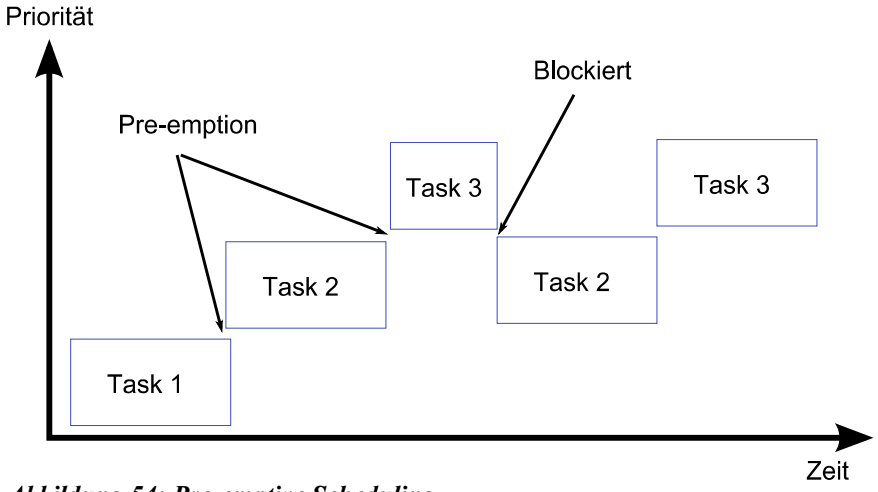


Abbildung 54: Pre-emptive Scheduling

Quelle: [KEIL_RTX_MAN]

5.4.4 Round Robin Pre-emptive Scheduling

In seiner Grundeinstellung ist das RTOS auf Round Robin Pre-emptive Scheduling eingestellt. Diese Art von Scheduling ist, wie der Name schon sagt, eine Kombination der Verfahren Round Robin und Pre-emptive Scheduling. So lange es nur Tasks gibt, welche die gleiche Priorität besitzen, kommt das Verfahren Round Robin zur Anwendung. Sobald ein Task mit einer höheren Priorität erzeugt wird, bekommt dieser vom Scheduler den Prozessor zugeteilt. Dieser höher priorisierte Task wird so lange ausgeführt, bis er blockiert oder wiederum ein höher priorisierter Task dem Prozessor zugeteilt wird. Wie in Abbildung 55 zu sehen ist, läuft das Programm im Round Robin Verfahren, bis Task 3 unterbrochen wird, da dann ein neuer, höher priorisierter Task zur Ausführung bereit ist. Es kommt zum Kontextwechsel, Task 3 pausiert und Task 4 beginnt mit der Ausführung. Der Scheduler befindet sich jetzt im Pre-emptive Modus. Sobald Task 4 seine Arbeit abgeschlossen hat (und so lange kein höher priorisierter Task den Prozessor anfordert), wird zurück in den Round Robin Modus gewechselt.[81]

[81] Vgl [WÖRN]

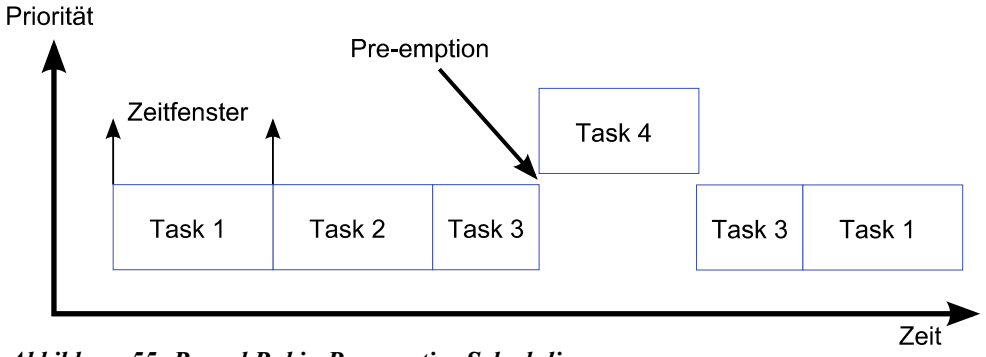


Abbildung 55: Round Robin Pre-emptive Scheduling

Quelle: [KEIL_RTX_MAN]

5.4.5 Co-operatives Multitasking

Das Co-operative Multitasking, dargestellt in Abbildung 56, ist ein Verfahren, in dem jeder Task den Prozessor so lange zugeteilt bekommt, bis er ihn selbst wieder freigibt. Round Robin ist bei diesem Verfahren ausgeschaltet. Sollte ein laufender Prozess den Prozessor freigeben, so wird die Ausführung beim nächsten Task, der bereit zur Ausführung ist, fortgesetzt.[82]

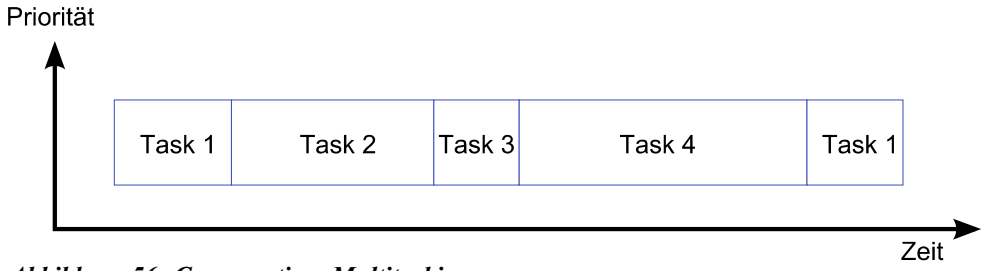


Abbildung 56: Co-operatives Multitasking

Quelle: [KEIL_RTX_MAN]

[82] Vgl [WÖRN]

6 Einbinden der Lüftersteuerung in das Echtzeitbetriebssystem

In diesem Kapitel sollen die Themen

- Konfiguration der Entwicklungsumgebung, Anpassung der Standard Peripherals Library und Konfiguration des Echtzeitbetriebssystems
- Einbinden der Lüftersteuerung in das Echtzeitbetriebssystem

besprochen werden.

Das zuvor implementierte Projekt der Lüftersteuerung soll Schritt für Schritt in das Echtzeitbetriebssystem eingebunden werden. Ausgehend von einer Kopie des Projektes der zuvor programmierten Lüftersteuerung werden in den folgenden Kapiteln nur Änderungen innerhalb des Projektes besprochen.

6.1 RTX Konfiguration der Entwicklungsumgebung

Um das RTX verwenden zu können muss eine Grundeinstellungen vorgenommen werden welche den Prozessor für die Verwendung mit einem RTOS konfiguriert (siehe auch Kapitel 2.1.1 Handler/Thread). Innerhalb der Entwicklungsumgebung muss, wie in Abbildung 57 zu sehen ist, in der Menüleiste **Flash→Configure Flash Tools...** im Register Target auf das Echtzeitbetriebssystem umgestellt werden.

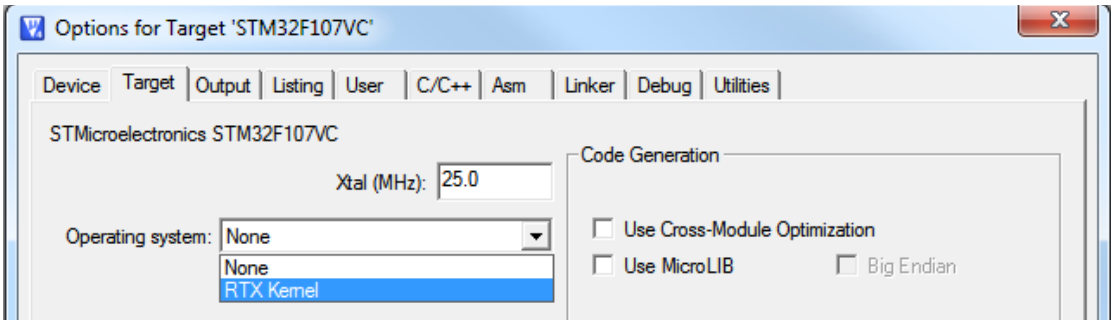


Abbildung 57: Konfiguration des Mikrocontrollers für Betrieb mit RTX Kernel

6.2 Anpassen der STM32F10x Standard Peripherals Library

Bei Verwendung des RTOS werden folgende Interrupts vom RTX-Kernel verwendet:

- SVCall
- PendSV
- SysTick

Um Mehrfachdefinitionen bei den literalen Konstanten zu vermeiden, müssen die folgenden Funktionen einschließlich der dazugehörigen Prototypen in der Datei[83] der ISR auskommentiert werden:

- SVC_Handler()
- PendSV_Handler()
- SysTick_Handler()

6.3 Konfiguration des Echtzeitbetriebssystems

Zur Konfiguration des Echtzeitbetriebssystems wird die Datei **RTX_Conf_CM.c**[84] in den Projektordner kopiert, eine neue Gruppe RTX innerhalb des Projektes erzeugt und diese dort dem Projekt hinzugefügt. Beim Öffnen dieser Datei innerhalb des Projekts kann man jetzt das Betriebssystem mit Hilfe des Configuration Wizzard, wie in Abbildung 58 zu sehen, konfigurieren. Folgende Einstellungen können hier vorgenommen werden:

- Task Configuration
  - **Number of concurrent running tasks**: Maximale Anzahl der im Projekt verwendeten Tasks.
  - **Number of tasks with user-provided stack**: Anzahl der Tasks mit angepasster Stack Größe.
  - **Task stack size [bytes]**: Reguläre Stack Größe der Tasks.

[83] stm32f10x_it.c und stm32f10x_it.h
[84] Keil\ARM\Boards\Keil\MCBSTM32C\RTX_Blinky\RTX_Conf_CM.c

- **Check for stack overflow**: Fügt den Programm Code hinzu, der den Stack bzgl. Überlauf überwacht. Sollte es einen Überlauf geben, führt der Systemkern eine System-Error Funktion aus und verbleibt dort in einer Endlosschleife. Wird für das Debugging nicht benötigt.
- **Run in privileged mode**: Lässt die Tasks im privilegierten Modus laufen

• SysTick Timer Configuration
- **Timer clock value [Hz]**: Angeben des aktuellen System Taktes.
- **Timer tick value [us]**: Einstellen der Frequenz des System Tick Timers zur Erzeugung von RTOS Interrupts.

• System Configuration
- **Round-Robin Task switching**: Aktivieren des Round Robin, bei deaktiviertem Feld läuft kooperatives Multitasking.
  - **Round-Robin Timeout [ticks]**: Anzahl der maximalen Timer-Ticks bis zum Kontextwechsel in den nächsten Task.
- **Number of user timers:** Anzahl der verwendeten virtuellen Timer[85].
- **ISR FIFO Queue size:** Größe des ISR FIFO Pufferspeichers.

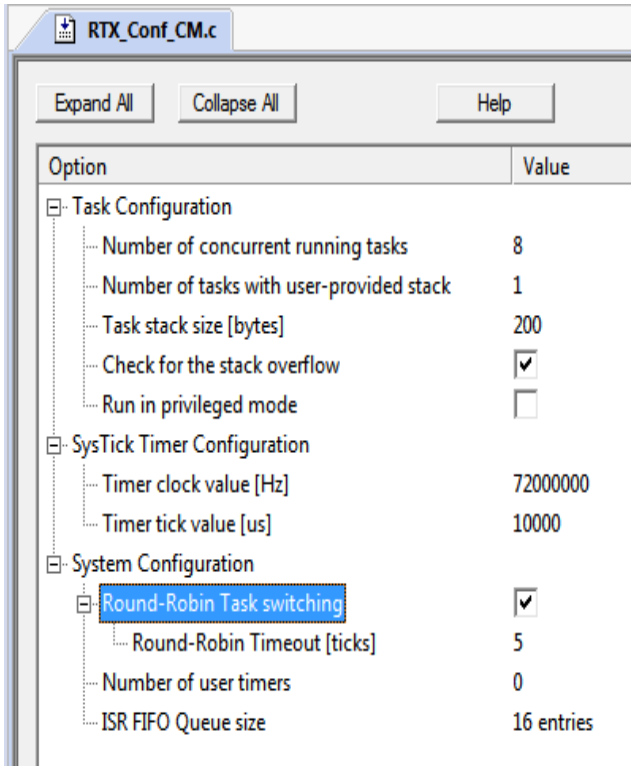


Abbildung 58: Konfiguration des RTX Kernels über den Configuration Wizzard

[85] Weiterführende Literatur [KEIL_RTX_MAN]

6.4 Einbinden der Lüftersteuerung in das Echtzeitbetriebssystem

Nachdem das Betriebssystem konfiguriert wurde, kann damit begonnen werden, das Projekt Lüftersteuerung in das Echtzeitbetriebssystem einzubinden. Die Initialisierung[86] muss nicht verändert werden.

6.4.1 Anpassen der Interrupt Service Routinen

Beim Anpassen der ISR[87] können die globalen Variablen gelöscht werden. Diese werden nicht weiter benötigt, da die eigentliche Berechnung innerhalb eines Tasks geschieht. Aus den ISR wird jeglicher Code zur Behandlung des Interrupts entfernt. Innerhalb der Routine wird lediglich das Pending Bit gelöscht und ein Event für einen hoch priorisierten Task generiert, welcher die eigentliche Bearbeitung des Interrupts erledigt. Dazu müssen noch die Task IDs extern definiert werden. Siehe dazu Abbildung 59. (siehe auch Kapitel 5.3.2).

[86] setup.c und setup.h
[87] stm32f10x_it.c

```
/* Includes ------------------------------------------------------------------*/
#include "stm32f10x_it.h"
#include "RTL.h"

/* Private variables ---------------------------------------------------------*/
//uint16_t tacho_irq_count =0;
//uint32_t total_revolutions = 0;
//uint16_t revolutions_per_minute = 0;
extern OS_TID TskID_2_EXTI15_10_IRQ;
extern OS_TID TskID_3_TIM5_IRQ;

/******************************************************************************/
/*            RTX STM32F10x Peripherals Interrupt Handlers                    */
/******************************************************************************/

/**
  * @brief  This function handles the EXTI15_10 interrupt request GPIOD Pin11
  *         and send an event to Task2_EXTI15_10_IRQ to compute the actual
  *         interrupt service routine
  * @param  None
  * @retval None
  */
void EXTI15_10_IRQHandler(void)   __irq
{
   if(EXTI_GetITStatus(EXTI_Line11) != RESET)
   {
      /* Clear EXTI Line11 interrupt */
      EXTI_ClearITPendingBit(EXTI_Line11);
      /* Set ISR event to high priority task TskID_3_TIM5_IRQ */
      isr_evt_set (0X0001, TskID_2_EXTI15_10_IRQ);
   }
}

/**
  * @brief  This function handles Timer5 interrupt request on timer update
  *         and send an event to Task3_TIM5_IRQ to compute the actual
  *         interrupt service routine
  * @param  None
  * @retval None
  */
void TIM5_IRQHandler(void)   __irq
{
   /* Clear TIM5 update interrupt */
   TIM_ClearITPendingBit(TIM5, TIM_IT_Update);
   /* Set ISR event to high priority task TskID_3_TIM5_IRQ */
   isr_evt_set (0X0002, TskID_3_TIM5_IRQ);
}
```

Abbildung 59: Anpassen der Interrupt Service Routinen zum Betrieb mit dem RTX Kernel (stm32f10x_it.c)

Abschließend (hier ohne Abbildung) müssen noch die Prototypen in der Header-Datei[88] angepasst (**void TIM5_IRQHandler(void) __irq**) werden.

6.4.2 Implementieren der main.c

Die Veränderungen der Datei sind so gravierend, dass diese hier zum besseren Verständnis nochmals vollständig in einzelnen aufeinander folgenden Abbildungen dargestellt und erklärt werden. Um das Echtzeitbetriebssystem verwenden zu können muss die in Abbildung 60 beschriebene Header-Datei[89] in das Betriebssystems eingebunden werden. Die Intertask Kommunikation wird hier mit Hilfe eines gemeinsamen Spei-

[88] stm32f10x_it.h
[89] RTL.h

chers und einer Sperrsynchronisation implementiert (siehe dazu Kapitel 5.3.4 Mutex). Es wäre auch möglich dies über ein Mailbox-Systems zu realisieren was aber die hier gezeigte Implementierung zu sehr aufblähen und unübersichtlich machen würde. Es sollte jedoch kein Problem darstellen dieses System im Nachhinein in das Projekt zu integrieren.

Zu Beginn werden einige Variablen für die Lüftersteuerung angelegt sowie eine weitere, welche Speicherplatz für einen größeren benutzerdefinierten Stack für den Ausgabe Task zur Verfügung stellt.

```
/**
  ******************************************************************************
  * @file    C:\Lueftersteuerung_RTX\Projekt\app\main.c
  * @author Hauke Pfaffelmoser
  * @brief  Fan control via potentiometer for Keil MCBSTM32C Evaluation Board.
  *         Using the Keil RTX Kernel (RTOS) and the
  *         STMicroelectronics STM32F10x Standard Peripherals Library
  *         Values:
  *                - rpm revolutions per minute
  *                - potentiometer value 0 - 4095
  *                - total revolutions
  *         will be displayed on LCD.
  ******************************************************************************
  */

/* Includes ------------------------------------------------------------------*/
#include <stdio.h>
#include "stm32f10x.h"
#include "setup.h"
#include "lcd/GLCD.h"
#include "RTL.h"

/* Private variables ---------------------------------------------------------*/
uint16_t tacho_irq_count = 0;
uint32_t total_revolutions = 0;
uint16_t revolutions_per_minute = 0;
extern __IO uint16_t ADC1ConvertedValue;

static uint32_t stack_Task1_Write_LCD [600/4];
```

Abbildung 60: Definition der benötigten Variablen (main.c)

In Abbildung 61 werden die Task Prototypen mit den dazugehörigen Task-IDs deklariert (siehe Kapitel 5.2.1).

```
/*Task prototypes ------------------------------------------------*/
void Task0_Init (void);

void Task1_Write_LCD(void);
void Task2_EXTI15_10_IRQ(void);
void Task3_TIM5_IRQ(void);
void Task4_PWM_Duty(void);

/*Task IDs -------------------------------------------------------*/
OS_TID TskID_1_Write_LCD;
OS_TID TskID_2_EXTI15_10_IRQ;
OS_TID TskID_3_TIM5_IRQ;
OS_TID TskID_4_PWM_Duty;

/* Declare Mutex Container */
OS_MUT mutex;
```

Abbildung 61: Definition der benötigten Task Prototypen und Task-IDs

In Abbildung 62 werden, wie auch schon in dem Projekt ohne RTX, die Peripherie und der Prozessor initialisiert. Der wesentliche Unterschied ist, dass am Ende der main() Funktion nicht in die Endlosschleife übergegangen wird, sondern durch die Initialisierung des RTOS die Prozessverwaltung dem Betriebssystem übergeben und der erste Task gestartet wird.

```
/**
  * @brief  Main program
  * @param  None
  * @retval None
  */
int main(void)
{
/* System Init-----------------------------------------------*/
   /* System Clocks Configuration */
   Setup_Clock();
   /* Configure GPIO ports */
   Setup_GPIO();
   /* Configure PWM Timer 1 */
   Setup_PWM_Timer1();
   /* Configure ADC */
   Setup_ADC1_Poti();
   /* Configure external interrupt EXTI Line 11*/
   Setup_EXTI11_GPIOD_Pin11();
   /* Configure Timer 5 */
   Setup_Tim5();
   /* Configure Exti Line11 Interrupt */
   Setup_NVIC_EXTI15_10();
   /* Configure TIM5 Interrupt */
   Setup_NVIC_TIM5();

   /* Configure LCD Display */
   GLCD_Init();
   GLCD_Clear(White);
   GLCD_SetTextColor(White);
   GLCD_SetBackColor(Blue);

   /* Write heading on Display */
   GLCD_DisplayString(0, 0, 1, "   Drehzahlregler   ");
   GLCD_SetBackColor(White);
   GLCD_SetTextColor(Blue);

   /* Initialize RTX and start init */
   os_sys_init (Task0_Init);
}
```

Abbildung 62: Initialisierung Prozessor und Peripherie (main.c)

Der Task in Abbildung 63 besitzt keine eigene Task-ID, da er nur dazu verwendet wird die anderen Tasks zu erzeugen. Nach Abarbeitung dieser Aufgaben löscht sich der Task selbst. Hierzu siehe Abbildung 63.

```
/**
  * @brief  Task0 Init Task
  * @param  None
  * @retval None
  */
__task void Task0_Init (void)
{
   TskID_1_Write_LCD = os_tsk_create_user(Task1_Write_LCD, 0X01,
                                          &stack_Task1_Write_LCD,
                                          sizeof(stack_Task1_Write_LCD));
   TskID_2_EXTI15_10_IRQ = os_tsk_create(Task2_EXTI15_10_IRQ, 0X02);
   TskID_3_TIM5_IRQ = os_tsk_create(Task3_TIM5_IRQ, 0X02);
   TskID_4_PWM_Duty = os_tsk_create(Task4_PWM_Duty, 0X01);

   /* Initialize the Mutex */
   os_mut_init(mutex);

   os_tsk_delete_self ();
}
```

Abbildung 63: Erzeugen der Programmtasks durch ersten Initialisierungstask (main.c)

Der folgende Task wird über die delay Funktion alle 10 Timer Ticks in den Zustand READY versetzt. Dies veranschaulicht Abbildung 64. Bei Ausführung des Tasks wird bei Änderung des ADC Wertes das Display mit den neuen Drehzahlwerten beschrieben. Er läuft im Round Robin und erwartet keine Events.

```
/**
  * @brief  Task 1 to write fan revolution values on display
  * @param  None
  * @retval None
  */
__task void Task1_Write_LCD(void)
{
   char display_ADC1Converted_value[20];
   char display_revolutions_per_minute[20];
   char display_total_revolutions[20];
   char poti_value = 0xFFFF;

   while(1)
   {
      if (ADC1ConvertedValue ^ poti_value)
      {

          /* Acquire Mutex */
          os_mut_wait(mutex,0xffff);

         sprintf(display_revolutions_per_minute, "RPM       :
                %5i",revolutions_per_minute);
         sprintf(display_ADC1Converted_value, "ADC Value :%5i", ADC1ConvertedValue);
         sprintf(display_total_revolutions,"Umdr. Ges.:%5i", (total_revolutions/2));

         /* Release Mutex */
         os_mut_release(mutex);

         /* Show ADC value bargraph */
         GLCD_Bargraph (115, (Line8), 160, 20, (ADC1ConvertedValue>>2));
         poti_value = ADC1ConvertedValue;

         /* Write values on LCD*/
         GLCD_DisplayString(3, 0, 1,(unsigned char *)
                                    display_revolutions_per_minute);
         GLCD_DisplayString(4, 0, 1, (unsigned char *) display_ADC1Converted_value);
         GLCD_DisplayString(5, 0, 1, (unsigned char *) display_total_revolutions);
         GLCD_DisplayString(8, 0, 1, "Geschw:");
      }

     /* Wait 10 Timer Ticks to reschedule */
      os_dly_wait(10);
   }
}
```

Abbildung 64: Definition des Tasks zum Schreiben des Displays (main.c)

In Abbildung 65 wird der Task zur Berechnung der externen Interrupts definiert. Er ist ein hoch priorisierter Task und dient zum Zählen der eingehenden Tachosignale des Lüfters. Bei jedem eingehenden Signal wird ein hier erwartetes Event Flag gesetzt. Sobald dieses Event ausgelöst wurde, werden zwei Variablen inkrementiert. Diese werden zur Berechnung der rpm und Gesamtdrehzahl verwendet. Abschließend wird das Event Flag gelöscht.

```
/**
  * @brief  Task 2 to compute EXTI15_10 interrupts on GPIOD Pin11
  * @param  None
  * @retval None
  */
__task void Task2_EXTI15_10_IRQ(void)
{
   while(1)
   {
      /* Wait for event */
      os_evt_wait_or (0X0001,0Xffff);
      /* Reset all LEDs */

      /* Acquire Mutex */
      os_mut_wait(mutex,0xffff);

      /* Count revolution per irq update */
      tacho_irq_count++ ;
      total_revolutions++ ;

      /* Release Mutex */
      os_mut_release(mutex);

      /* Clear event flag */
      os_evt_clr (0X0001, TskID_2_EXTI15_10_IRQ);
   }
}
```

Abbildung 65: Definition des Interrupt Tasks zum Zählen der Tachosignale des Lüfters über externe Interrupts

Der Task in Abbildung 66 berechnet die rpm und gehört auch zu den hoch priorisierten Interrupt Tasks. Auch dieser Task erwartet ein Interrupt Event, berechnet daraufhin die in Task 2 eingegangenen Tachosignale des Lüfters und löscht anschließend das Event Flag.

```
/**
  * @brief  Task 3 to compute  Timer5 interrupts
  * @param  None
  * @retval None
  */
__task void Task3_TIM5_IRQ(void)
{
   while(1)
   {
      /* Wait for event */
      os_evt_wait_or (0X0002,0Xffff);

      /* Acquire Mutex */
      os_mut_wait(mutex,0xffff);

      /* Calculate revolutions per minute.
         2 tacho signals per revolution and
         2 TIM5 interrupts per second */
      revolutions_per_minute = tacho_irq_count * 60;
      tacho_irq_count = 0;

      /* Release Mutex */
      os_mut_release(mutex);

      /* Clear event flag */
      os_evt_clr (0X0002, TskID_3_TIM5_IRQ);
   }
}
```

Abbildung 66: Definition des Interrupt Tasks zum Berechnen der rpm mit Hilfe eines Timer Interrupts (main.c)

In diesem Task wird alle 10 Timer Ticks das Tastverhältnis der PWM eingestellt, er läuft im Round Robin und erwartet keine Events, Abbildung 67 beschreibt die Implementierung.

```
/**
  * @brief  Task 4 to set the PWM duty cycle
  * @param  None
  * @retval None
  */
__task void Task4_PWM_Duty(void)
{
   while(1)
   {
      /* Set PWM duty cycle */
      TIM1->CCR1 = ADC1ConvertedValue;
      /* Wait 10 Timer Ticks to reschedule */
      os_dly_wait(10);
   }
}
```

Abbildung 67: Task zum Einstellen des Tastverhältnisses

7 Debugging und Konfiguration des Programmers/Debuggers

Ein wesentlicher und nicht zu vernachlässigender Punkt bei der Mikrocontroller Programmierung ist das Debugging. ARM hat dazu den Cortex™-M3 mit der Serial Wire Debug und Serial Wire Viewer Technologie ausgestattet. Diese Technologien sind Bestandteil des CoreSight™ Debug Access Port, welcher von ARM entwickelt wurde und eine On-Chip debug und Real-Time tracing Lösung für den Mikrocontroller darstellt. Serial Wire Debug bietet die selbe Funktionalität wie JTAG, benötigt jedoch nur zwei, mit Serial Wire Viewer drei Anschlusspins. Zusätzlich ist es möglich, über die Serial Wire Viewer Schnittstelle den Inhalt des Systemspeichers während der Laufzeit zu beobachten ohne den Prozessor anhalten zu müssen.

In diesem Kapitel sollen folgende Punkte genauer betrachtet werden:

- Fehlersuche im Quellcode
- Konfiguration des Programmers
- Konfiguration des Keil ULINK2 Debuggers
- Initialisierung des Keil ULINK2 Debuggers
- Tracen einiger Variablen und Interrupts
- Tracen des Echtzeitbetriebssystems
- Verwendung des Logic Analyzers

7.1 Fehlersuche

Es ist nicht bei allen Fehlern möglich, diese mit Hilfe des Debuggers zu identifizieren. Aus diesem Grund werden in Tabelle 3 acht Fehlerquellen aufgelistet die während der Einarbeitungszeit in den Mikrocontroller am Häufigsten auftraten. Es hat sich als sehr hilfreich herausgestellt diese Punkte in der gegebenen Reihenfolge abzuarbeiten um die wichtigsten Fehlerquellen im Bereich der Initialisierung und Konfiguration auszuschließen.

Mögliche Fehlerquellen
1. Wurden alle benötigten Takte eingestellt?
2. Wurden die benötigten GPIOs (richtig) konfiguriert?
3. Wurden die benötigten externen Interrupts (richtig) konfiguriert?
4. Wurde der NVIC (richtig) konfiguriert?
5. Haben die ISRs den richtigen Namen?
6. Wird das Pending Bit innerhalb der ISR gelöscht?
7. Werden in der Main Funktion alle Initialisierungen ausgeführt?
8. Wurde die benötigte Peripherie (richtig) konfiguriert?

Tabelle 3: Mögliche Fehlerquellen

7.2 Konfiguration des Debuggers/Programmers

Im hiesigen Kapitel werden die wesentlichen Einstellungen des Debugger besprochen. Alle nicht ausdrücklich erwähnten oder markierten Einstellungen müssen nicht unbedingt konfiguriert werden und können auf den Standard Einstellungen verbleiben.

Der ULINK2 dient neben seiner Funktion als Debugger auch als Programmer zum Downloaden der Programme auf den Mikrocontroller. Um dies tun zu können muss er, wie in Abbildung 68 gezeigt, konfiguriert werden. In das Menü gelangt man über **Menüleiste → Flash → Configure Flash Tools...**.[90]

Folgende Einstellungen müssen im Register Utilitys vorgenommen werden:

1. Auswählen des Programmers
2. Untermenü Cortex™-M Target Driver Setup
3. Vor dem Debugging wird immer erst das aktuelle Programm auf den Mikrocontroller geflasht.

Siehe dazu auch Abbildung 68.

[90] [KEIL_DEBUG_CONF]

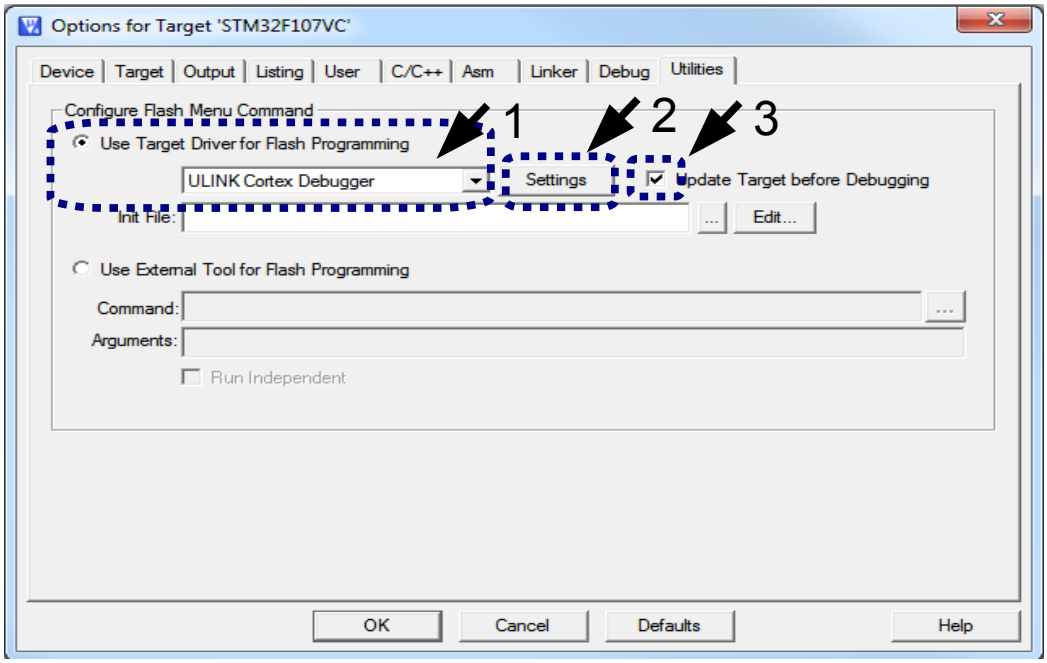


Abbildung 68: Options for Target 'STM32F107VC' / Utilities

Durch Anklicken des Button **Settings** öffnet sich das nächste Menü Cortex™-M Target Driver Setup. Im Register Flash Download werden die in Abbildung 69 zu sehenden Einstellung vorgenommen.[91]

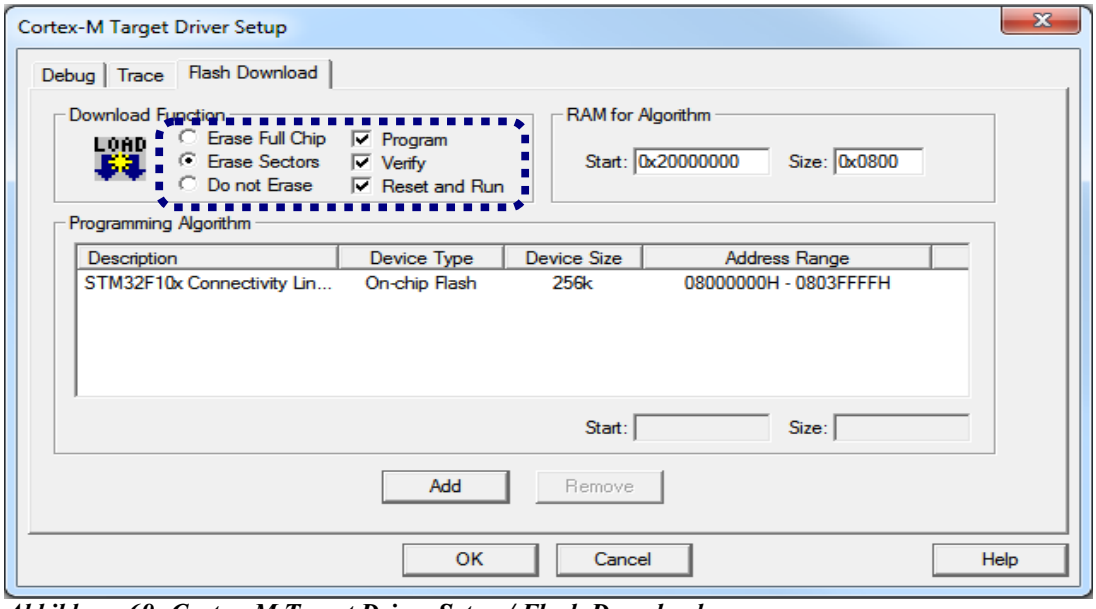


Abbildung 69: Cortex-M Target Driver Setup / Flash Download

[91] Weiterführende Literatur [KEIL_DEBUG_CONF]

Im Register Debug müssen die in Abbildung 70 markierten Felder konfiguriert werden:

1. Debugger auswählen
2. Debug Schnittstelle Taktfrequenz des Debuggers einstellen. Im Falle dieses Mikrocontrollers, bei dem sich JTAG und Serial Wire zwei Leitungen teilen, muss die Checkbox **SWJ** aktiviert werden.
3. Sobald der Debugger richtig erkannt wurde, wird er im Feld **SW Device** angezeigt.

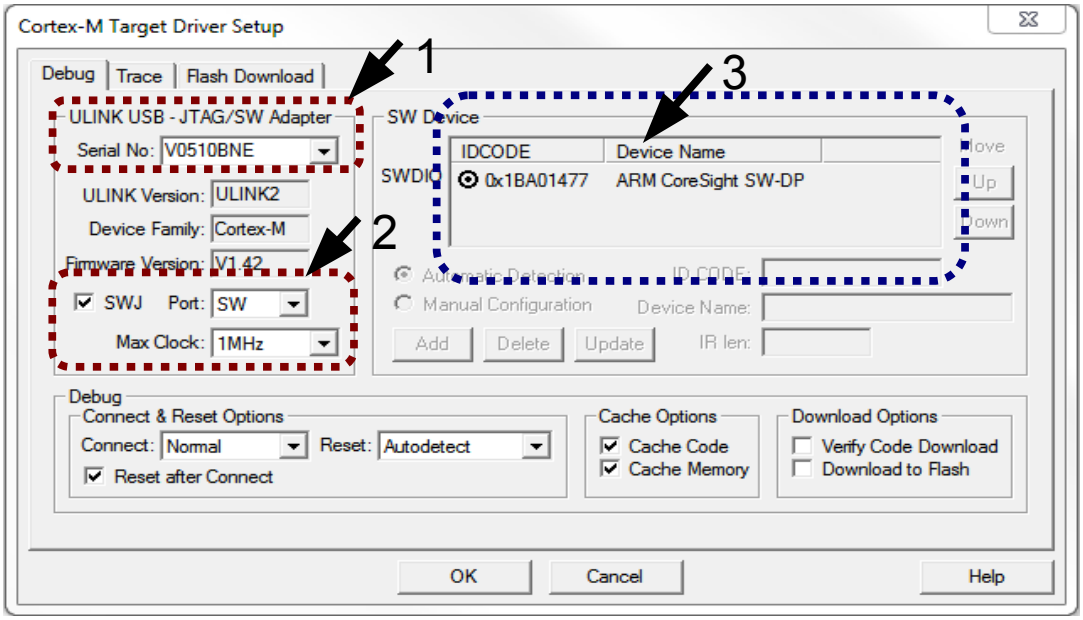


Abbildung 70: Cortex-M Target Driver Setup / Debug

Anschließend wird im Register Trace[92] das Tracing aktiviert und konfiguriert. Die Einstellungen sollten wie in Abbildung 71 übernommen werden. Hier ist zu beachten, dass der hier verwendete Debugger nur maximal 1MBit/s übertragen kann. Dies schränkt die Möglichkeiten des Tracings im Vergleich zum ULINK Pro[93] ein, da dieser sehr viel höhere Bandbreiten in der Datenübertragung unterstützt. Aus diesem Grund sollten nur einzelne Events überwacht und die höchst möglichen Vorteiler (Prescaler) gewählt werden.

[92] Weiterführende Literatur [KEIL_DEBUG_CONF] Trace Configuration
[93] Vgl. [KEIL_ULINK] comparison

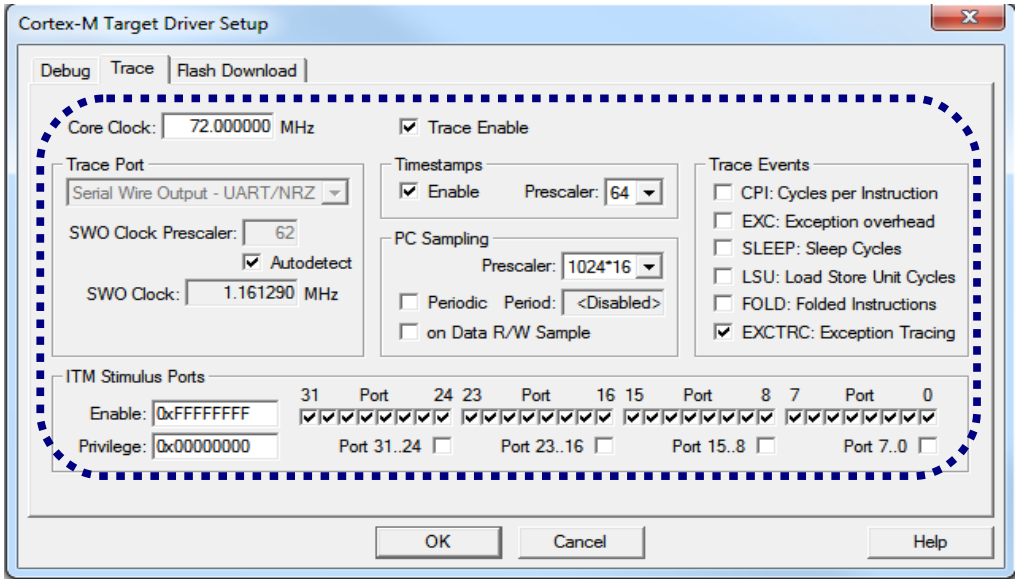


Abbildung 71: Cortex-M Target Driver Setup / Trace

Abschließend muss in dem Register Debug der richtige Debugger gewählt und die zur Initialisierung benötigte .ini-Datei hinzugefügt werden. Diese kann aus einem Beispielprojekt[94] in den Projektordner kopiert, über die Entwicklungsumgebung geöffnet und anschließend mit dem Configuration Wizzard konfiguriert werden. Siehe dazu die hervorgehobenen Felder in Abbildung 72.

Durch abschließendes Bestätigen mit OK ist der Debugger nun konfiguriert.

[94] Keil\ARM\Boards\Keil\MCBSTM32C\Blinky_ULp**STM32_SWO.ini**

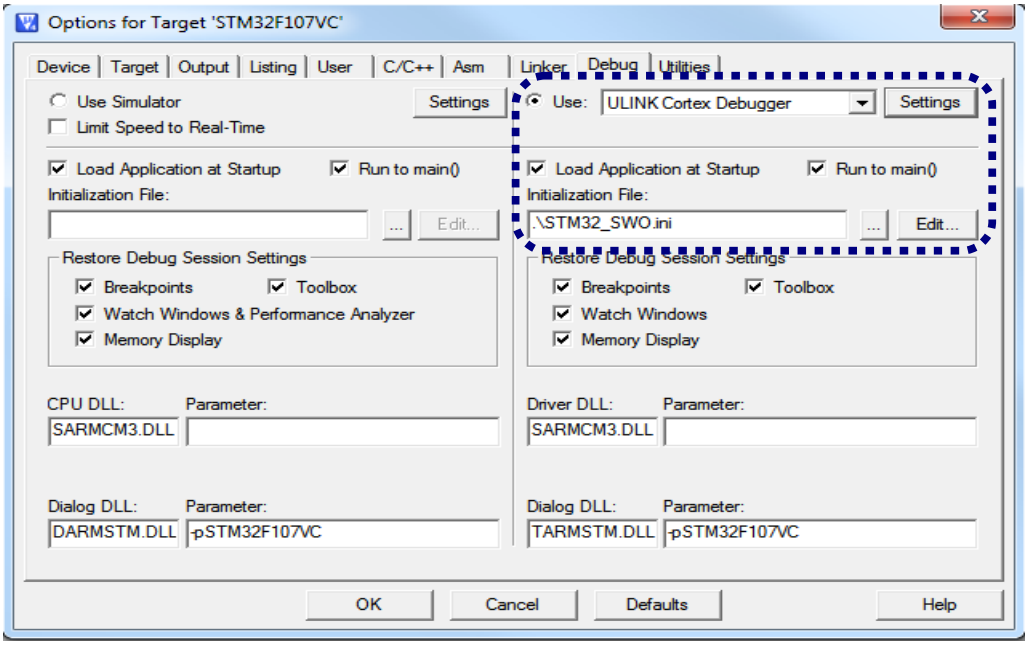


Abbildung 72: Options for Target 'STM32F107VC' / Debug

7.3 Debugging

Nachdem das hier vorgestellte Beispielprogramm compiliert und gelinkt wurde, kann es mit dem ULINK2 Debugger getestet werden. Über die Menüleiste **Debug → Start/Stop Debug Session** wird der Debugger initialisiert, die Debugfenster öffnen sich und das Beispielprogramm wird bis zum Eintritt in die main() ausgeführt. Nachdem dies geschehen ist, können über die Menüleiste die folgenden Fenster geöffnet werden:

- **View → Watch Window → Watch1 (Abbildung 74)**
- **View → Symbol Window (Abbildung 75)**
- **View → Analysis Window → Logic Analyzer (Abbildung 76)**
- **View → Trace → Exceptions (Abbildung 77)**

In Abbildung 73 sind die Control Buttons des Debuggers zu sehen. Mit ihnen kann die Programmausführung gesteuert werden.

Die Symbole stehen für folgende Funktionen:

1. Reset des Mikrocontrollers
2. Start Programmausführung
3. Abbruch der Programmausführung

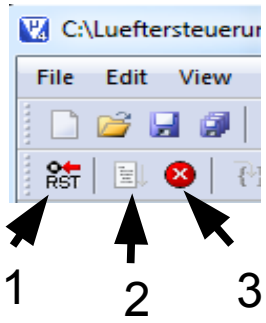


Abbildung 73: Control Buttons den Debuggers

7.3.1 Watch Window

Der Debugger besitzt zwei Watch Windows. In diesen kann man während der Ausführung Strukturen, Felder und Variablen beobachten. Dafür müssen diese jedoch im Quellcode global deklariert und die periodische Aktualisierung in der Menüleiste **View → Periodic Window Update** aktiviert sein. Weiterhin kann man über einen Rechtsklick auf den Wert die Zahlenbasis von Hexadezimal auf Dezimal umstellen. Watch 1 Window in Abbildung 74 zeigt die aktuellen Drehzahl- und ADC Werte des Lüfters.

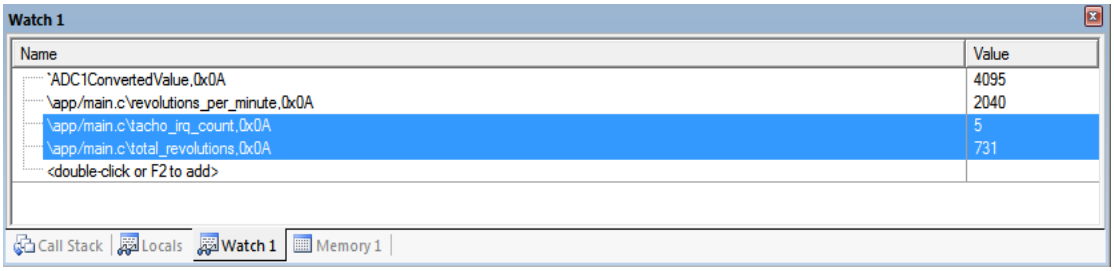


Abbildung 74: Watch1 Window mit Ausgabe der Lüftervariablen

Der einfachste Weg Variablen dem Watch Window hinzuzufügen ist, sie über das Symbols Window, zu sehen in Abbildung 75, per Drag and Drop in das Fenster zu ziehen.

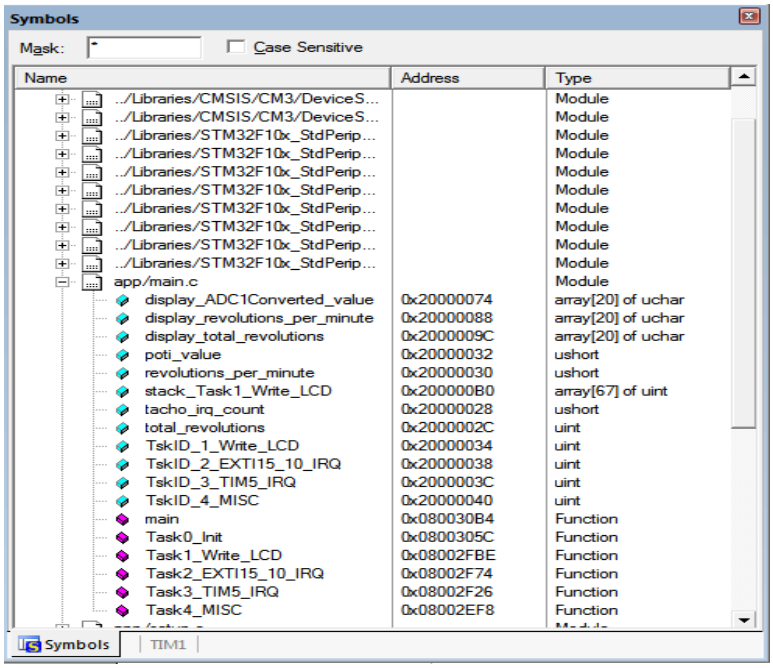


Abbildung 75: Symbol Window der µVision4 Debug Umgebung

7.3.2 Logic Analyzer

Der Logic Analyzer ermöglicht es Variablen und simulierte Signale während der Laufzeit zu plotten. Abbildung 76 stellt im Logic Analyzer den ADC Wert und die Drehzahl des Lüfters. grafisch dar. Die Variablen werden, wie schon in Kapitel 7.3.1 beschrieben, über Drag and Drop in das Logic Analyzer Fenster gezogen und anschließend konfiguriert.



Abbildung 76: Logic Analyzer Window mit grafischer Darstellung des ADC-Wertes und Lüfterdrehzahl

Über den Button Setup lässt sich nun der darzustellende Wertebereich und die Art des Signals einstellen. Siehe dazu Abbildung 77.

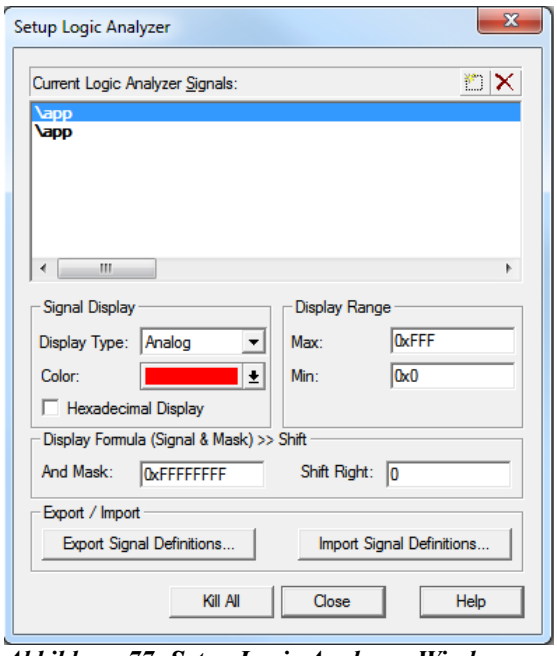
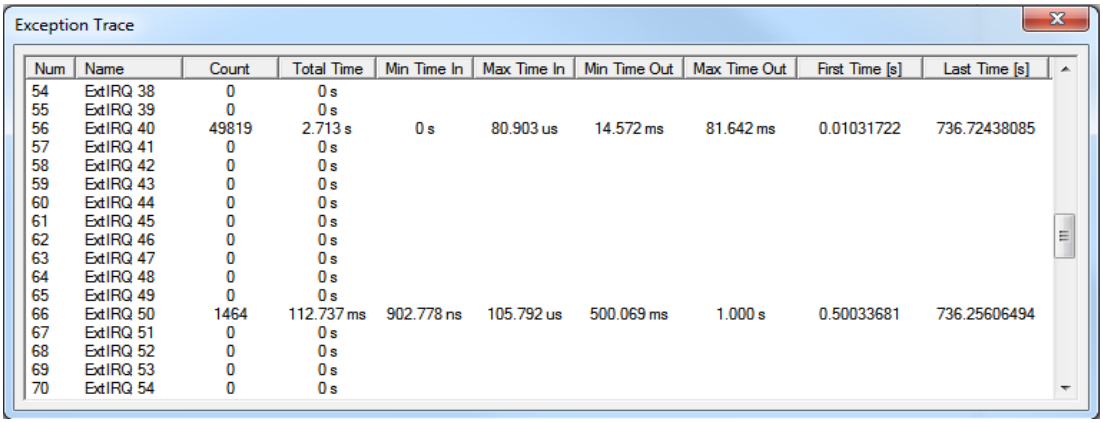


Abbildung 77: Setup Logic Analyzer Window

7.3.3 Exception Tracing

Durch Aktivierung des Exception Tracings und der Timestamps in der Debugger Konfiguration ist es jetzt möglich, die ausgelösten Interrupts während der Laufzeit zu beobachten bzw. festzustellen, ob diese überhaupt ausgelöst werden. Abbildung 78 zeigt IRQ40[95](EXTI15_10) und IRQ50(TIM5) und erzeugt die hier aufgeführten Informationen:

- Anzahl der ausgelösten Interrupts (Count)
- Gesamte Bearbeitungszeit aller Interrupts dieses Typs (Total Time)
- Minimaler Zeitraum innerhalb der ISR (Min Time In)
- Maximaler Zeitraum innerhalb der ISR (Max Time In)
- Minimaler Zeitraum bis zum erneuten Aufruf der ISR (Min Time Out)
- Maximaler Zeitraum bis zum erneuten Aufruf der ISR (Max Time Out)
- Zeitpunkt des ersten Aufrufs der ISR (First Time [s])
- Zeitpunkt des letzten Aufrufs der ISR (Last Time [s])



Num	Name	Count	Total Time	Min Time In	Max Time In	Min Time Out	Max Time Out	First Time [s]	Last Time [s]
54	ExtIRQ 38	0	0 s						
55	ExtIRQ 39	0	0 s						
56	ExtIRQ 40	49819	2.713 s	0 s	80.903 us	14.572 ms	81.642 ms	0.01031722	736.72438085
57	ExtIRQ 41	0	0 s						
58	ExtIRQ 42	0	0 s						
59	ExtIRQ 43	0	0 s						
60	ExtIRQ 44	0	0 s						
61	ExtIRQ 45	0	0 s						
62	ExtIRQ 46	0	0 s						
63	ExtIRQ 47	0	0 s						
64	ExtIRQ 48	0	0 s						
65	ExtIRQ 49	0	0 s						
66	ExtIRQ 50	1464	112.737 ms	902.778 ns	105.792 us	500.069 ms	1.000 s	0.50033681	736.25606494
67	ExtIRQ 51	0	0 s						
68	ExtIRQ 52	0	0 s						
69	ExtIRQ 53	0	0 s						
70	ExtIRQ 54	0	0 s						

Abbildung 78: Exeption Trace Window mit Ausgabe der EXTI15_10 und TIM5 Interrupts

7.3.4 Tracing des Echtzeitbetriebssystems und der Tasks

Über die Menüleiste **Debug → OS Support → RTX Tasks and System** kann das Tracing für das RTX aktiviert werden. Dadurch erscheint ein Fenster wie in Abbildung 79,

[95] [STM_REF_MAN] Seite 195, Position des Interrupts in der Vektortabelle

welches folgende Informationen über den aktuellen Zustand des Betriebssystems und der Tasks zur Laufzeit liefert:

- **Systeminformationen**
  - Verwendeter System Timer[96] des RTOS (Timer Number)
  - Taktrate des Betriebssystem Takts (Tick Timer)
  - Maximaler Zeitraum innerhalb eines Tasks bis zum Kontextwechsel beim Round Robin (Round Robin Time Out)
  - Stack Größe der Tasks zum Speichern der Laufzeitinformationen beim Kontextwechsel (Stack Size)
  - Anzahl[97] der benutzerdefinierten Stacks (Task with User-provided Stack)
  - Anzahl der verwendeten Tasks (Task Usage)
  - Anzahl der verwendeten virtuellen Timer (User Timers)

- **Taskinformationen**
  - Task-ID des aktuellen Tasks (ID)
  - Name das Tasks (Name)
  - Priorität des Tasks (Priority)
  - Aktueller Status (State)
  - Evtl. eingestellte Warteperioden (Delay)
  - Für diesen Task gesetzte Event Flags (Event Value)
  - Vom Task erwartete Event Flags (Event Mask)
  - Aktuelle Stack Auslastung (Stack Load)

[96] In diesem Fall wird der Systick Timer (SysTick) des Mikrocontrollers verwendet
[97] Dieser Wert wird von Hand in die **RTX_Conf_CM.c** eingetragen **#define OS_PRIVCNT 1**

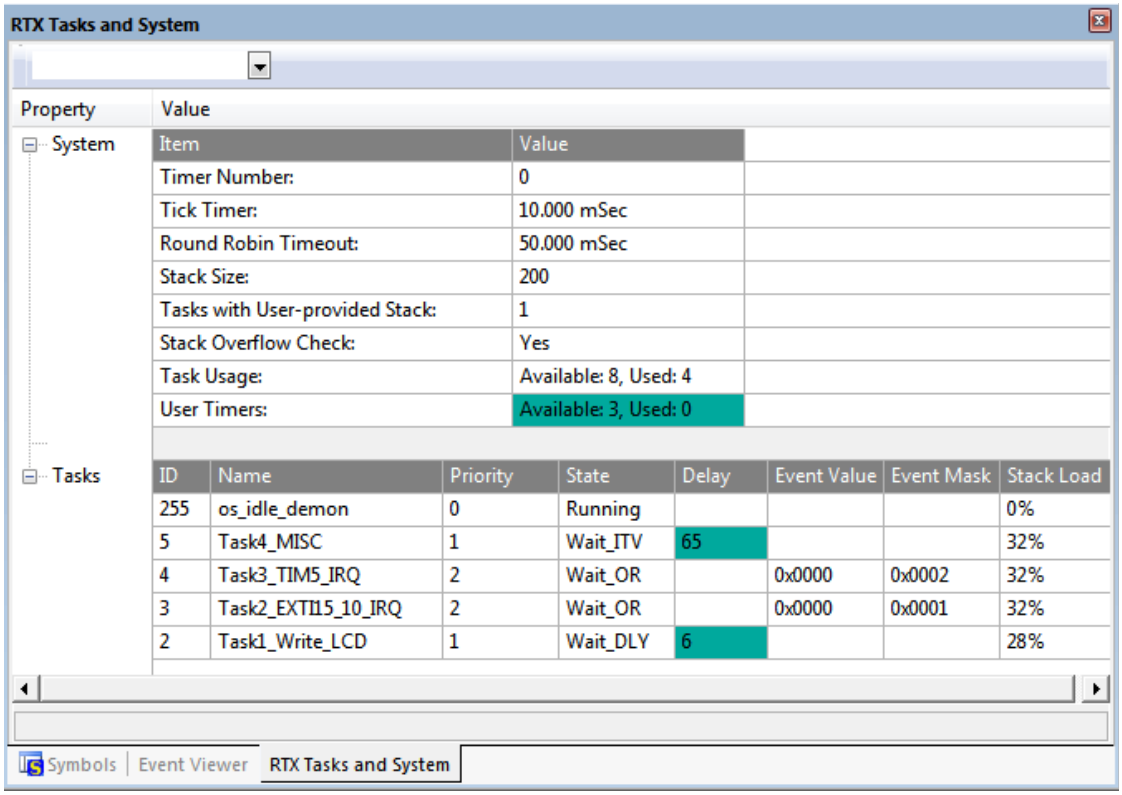


Abbildung 79: RTX Task and System Window mit Laufzeitinformationen des Echtzeitbetriebssystems

Durch das Tracing des Betriebssystems ist es sehr einfach Laufzeitfehler wie Stacküberläufe und Verklemmungen von Tasks zu identifizieren. Des Weiteren vereinfacht es die Anpassung der mittleren Stack Größe und ermöglicht es, diese zu Beginn einfach abzuschätzen.

8 Resumee

Das Ziel dieser Arbeit war, eine für einen Anfänger einfach zu verstehende Einführung in die Programmierung eines auf Cortex™-M3 Architektur basierenden Mikrocontrollers (STMikroelectronics STM32F107VC) unter Einbindung des Keil RTX Real-Time Operating Systems zu entwickeln. Zu diesem Zweck wurde mit Hilfe einer exemplarischen Anwendung in Form einer Lüftersteuerung die gesamte Implementierung in Einzelschritten bis hin zur Einbindung der Software in das Echtzeitbetriebssystem und das anschließende Debugging erklärt.

Diese Einführung stellt in ihrem Umfang nur ein Grundgerüst dar und hilft den Einstieg in die Programmierung dieses Prozessors zu erleichtern. Dadurch, dass mehr als ein Hersteller an der Entwicklung des Mikroprozessors beteiligt waren, gibt es verschiedene Bibliotheken und sehr viel Literatur (Datenblätter, Reference Manuals, Benutzerhandbücher), was die Suche nach Antworten auf sich ergebende Fragen teilweise sehr erschwert. Diese Arbeit ermöglicht auf Grund der didaktischen Reduktion der Informationen den logischen Aufbau des Prozessors, der Datenblätter und die Funktionsweise der Bibliothek besser zu verstehen. Dadurch und durch die methodische Entscheidung die Zusammenhänge am Beispiel „Lüftersteuerung“ zu veranschaulichen, ist es möglich, die weiterführenden Beispielanwendungen der Bibliothek zu benutzen und ist damit sehr schnell in der Lage funktionsfähige Software zu produzieren.

Bezüglich der Hardware musste festgestellt werden, dass das hier verwendete Entwicklungsboard für die Verwendung im Praktikum nur begrenzt geeignet ist, da die meisten Anschlusspins des Mikrocontrollers schon fest verdrahtet sind. Dies hat zur Folge, dass der Anschluss weiterer Module an den Mikrocontroller erschwert wird und eine falsche Konfiguration des Prozessors im ungünstigsten Fall zur Zerstörung des gesamten Entwicklungsboards führt.

Im Unterschied zu diesem Entwicklungsboard hat ein Board mit einem modularen Aufbau den Vorteil, dass man bei einem solchen die für den Versuchsaufbau benötigten Sensoren und Aktoren bezüglich Art und Anzahl variieren kann und bei Beschädigung wesentlich geringere Kosten entstehen. Die Firma TinkerForge liefert dazu einen sehr vielversprechenden Ansatz. Sie hat eine Art Hardware-Baukasten entwickelt, bei dem sich ein auf Cortex™-M3 basierender Baustein (Master Brick) mit einer Vielzahl von

weiteren Modulen (Bricks) erweitern lässt. Dazu gehören unter anderem verschiedene Motorsteuerungen, eine Debug-Schnittstelle, ein Potentiometer und verschiedene Displays. Außerdem ist die Hardware unter der Cern Open Hardware License und die Software unter GNU General Public License veröffentlicht.

Ähnlich wie bei den heutigen PCs wird die Komplexität und Leistungsfähigkeit der Mikrocontroller weiter zunehmen. Dies macht den Einsatz von Echtzeitbetriebssystemen mit gut beherrschbaren Hardwareabstraktionsschichten, Taskverwaltungen, Speicherverwaltungen und Dateisystemen unverzichtbar um produktiv entwickeln und die Leistungsfähigkeit der Mikrocontroller voll ausschöpfen zu können.

9 Verzeichnisse

9.1 Literaturverzeichnis

[DOWNEY] Downey, Allen B.: The Little Book of Semaphores Second Edition Version 2.1.5 : 2008, Green Tea Press. URL: http://greenteapress.com/semaphores/downey08semaphores.pdf

[MARTIN] Martin, Trevor: The Insider's Guide To The STM32 ARM® Based Microcontroller Version 1.8, Second Edition : 2009, Hitex UK Ltd.. - ISBN 0-9549988 8, URL: http://www.hitex.com/fileadmin/pdf/insiders-guides/stm32/isg-stm32-v18d-scr.pdf

[MARWEDEL] Marwedel, Peter: Eingebettete Systeme : 2008, Springer. - ISBN 978-3-540-34048-5

[WÖRN] Wörn, Heinz; Brinkschulte, Uwe: Echtzeitsysteme Grundlagen, Funktionsweisen, Anwendungen : 2005, Springer (eXamen.press). - ISBN 3-540-205883

[YIU] Yiu, Joseph: The Definitive Guide to the ARM Cortex™-M3 : 2009, Newnes. - ISBN 978-1-85617-963-8

9.2 Verzeichnis der Internetquellen

[ARM_GEN_USER_GUIDE] ARM Limited: Cortex™-M3 Devices Generic User Guide. URL: http://infocenter.arm.com/help/topic/com.arm.doc.dui0552a/DUI0552A_cortex_m3_dgug.pdf (abgerufen am 01.08.2011)

[ARM_SWD] Arm Limited: Serial Wire Debug. URL: http://www.arm.com/products/system-ip/debug-trace/coresight-soc-components/serial-wire-debug.php (abgerufen am 23.09.2011)

[ARM_SWV] Serial Wire Viewer (SWV) for Arm Cortex™-M3 Processors ARM Real-Time Trace aids debugging. URL: http://www.keil.com/download/files/swv_on_cortex-m3.pdf

[ARM_TEC_REF_MAN] ARM Limited: Cortex™-M3 Revision r2p1 Technical Reference Manual. URL: http://infocenter.arm.com/help/topic/com.arm.doc.ddi0337i/DDI0337I_cortexm3_r2p1_trm.pdf (abgerufen am 12.08.2011)

[BEQUIET] be quiet! SilentWings^{PWM} 80mm, Spezifikation, URL: http://www.be-quiet.net/be-quiet.net/index.php?StoryID=30&ProductID=58&websiteLang=de (abgerufen am)

[EETASIA] EE Times Asia: ARM Thumb-2 technology features enhanced performance. URL: http://www.eetasia.com (abgerufen am 25.10.2011)

[ELEKTR_NET] Dr. Böhringer, Kurt: Der Cortex™-M3 im Vergleich zu ARM7 – Performance und Speicherbedarf. URL: http://www.elektroniknet.de (abgerufen am 21.12.2011)

[KEIL_DEBUG_CONF] KEIL: Cortex-M Debugging. URL: http://www.keil.com/support/man/docs/ulink2/ulink2_cf_ctx_debugger.htm (abgerufen am 15.12.2011)

[KEIL_EVAL_FEAT] KEIL: MCBSTM32C Evaluationsboard Features. URL: http://www.keil.com/mcbstm32c/ (abgerufen am 07.10.2011)

[KEIL_EVAL_SCHALTPL] KEIL: MCBSTM32C base board schematics. URL: http://www.keil.com/mcbstm32c/mcbstm32c-base-board-schematics.pdf (abgerufen am: 14.08.2011)

[KEIL_EVAL_TEC_SPEC] KEIL: MCBSTM32C Evaluationsboard Technical Specifications. URL: http://www.keil.com/mcbstm32c/specs.asp (abgerufen am 22.09.2011)

[KEIL_RTX_MAN] KEIL: RL-ARM User's Guide. URL: http://www.keil.com/support/man/docs/rlarm/rlarm_ar_artxarm.htm (abgerufen am 30.08.2011)

[KEIL_UVISION4_MAN] KEIL: Getting Started Creating Applications with µVision®4. URL: http://www.keil.com/product/brochures/uv4.pdf (abgerufen am 20.10.2011)

[MICROCONSULT] Batt, Thomas: RTOS (Real Time Operating System) URL: http://www.-microconsult.de/ (abgerufen am 16.12.2011)

[MIKROCON_NET_PWM] Microcontroller.net: Pulsweitenmodulation URL: http://www.mikrocontroller.net (abgerufen am 02.09.2011)

[PWM-SPEC] Intel Corporation: 4-Wire Pulse With Modulation (PWM) Controlled Fans. URL: http://www.formfactors.org/developer/specs/4_Wire_PWM_Spec.pdf (abgerufen am 02.01.2012)

[RN_WISSEN] RoboterNetz: Pulsweitenmodulation. URL:http://www.rn-wissen.de (abgerufen am 02.09.2011)

[STM_BLOCKDIA] STMicroelectronics: Blockdiagramm STM32F107VC. URL: http://www.st.com/internet/com/TECHNICAL_RESOURCES/TECHNICAL_DIAGRAM/CIRCUIT_DIAGRAM/circuit_diagram_15274.pdf (abgerufen am 07.01.2012)

[STM_DATASHEET] STMicroelectronics: STM32F107xx Datasheet. URL: http://www.st.-com/internet/com/TECHNICAL_RESOURCES/TECHNICAL_LITERATURE/DATAS-HEET/CD00220364.pdf (abgerufen am 22.09.2011)

[STM_ERRATA_SHEET] STMicroelectronics: STM32F107xx Errata sheet. URL: http://ww-w.st.com/internet/com/TECHNICAL_RESOURCES/TECHNICAL_LITERATURE/ER-RATA_SHEET/CD00238166.pdf (abgerufen am 29.09.2011)

[STM_PROG_MAN] STMicroelectronics: RM0056 STM32F10xxx Programming manual. URL: http://www.st.com/internet/com/TECHNICAL_RESOURCES/TECHNICAL_LI-TERATURE/PROGRAMMING_MANUAL/CD00228163.pdf (abgerufen am 08.11.2011)

[STM_REF_MAN] STMicroelectronics: RM0008 STM32F107xx Reference manual. URL: http://www.st.com/internet/com/TECHNICAL_RESOURCES/TECHNICAL_LITERA-TURE/REFERENCE_MANUAL/CD00171190.pdf (abgerufen am 08.01.2012)

[TINKERFORGE] Tinkerforge: Bricks. URL: http://www.tinkerforge.com/ (abgerufen am 06.01.2012)

10Inhalt der CD

Ordner Dokumente

- Diplomarbeit im PDF Format

Ordner Literatur

- Keil Hardware
  - MCBSTM32C Schaltplan
- Keil Software
  - Getting Started Creating Applications with µVision4
- PWM
  - Intel 4 Wire PWM Spec
- Semaphore
  - The Little Book of Semaphores
- STM32F107xx
  - Cortex M3 Devices Generic User Guide
  - Cortex M3 Technical Reference Manual
  - STM32F10xxx Programming Manual
  - STM32F107VC Blockdiagramm
  - STM32F107xx Datasheet
  - STM32F107xx Errata sheet
  - STM32F107xx Reference manual
  - The Insiders Guide To The STM32
  - Serial Wire Viewer (SWV) for ARM Cortex™ M3

Ordner Projekte

- Lüftersteuerung
- Lüftersteuerung_RTX
- Template

11 Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der hier angegebenen Hilfsmittel angefertigt habe. Alle Stellen, die wörtlich oder sinngemäß aus veröffentlichten und nicht veröffentlichten Schriften entnommen wurden, sind als solche kenntlich gemacht. Die Arbeit ist in gleicher oder ähnlicher Form oder auszugsweise im Rahmen einer anderen Prüfung noch nicht vorgelegt worden.

Kirchhain, den 14.01.2012

_____

Hauke Pfaffelmoser